Project Report

Capstone project report: design, implementation, results and evaluation

Project title	IntelliMetrics AI: A Governed Semantic Layer with TARA, an AI Assistant for Trusted Business Metrics
Document	Project Report
Team members	Devi Priya, Priyanka, Akshay and Bhaskar
Institution	KL University
Academic year	2026-27
Date	26-Oct-2026 (IST)
Status	University project: academic capstone with synthetic demonstration data

Table of contents

- 1 Abstract
- 2 Introduction
  - 2.1 Background and motivation
  - 2.2 Why an Indian setting
- 3 Problem statement
- 4 Objectives
- 5 Scope
- 6 Background and literature review
  - 6.1 Semantic layers and metric governance
  - 6.2 Text-to-SQL and its risks
  - 6.3 Safe query execution
  - 6.4 Columnar analytics with DuckDB and Parquet
  - 6.5 Access control
  - 6.6 Indian business context
- 7 Existing system versus proposed system
- 8 Methodology
  - 8.1 Development approach
  - 8.2 Requirements to design
  - 8.3 Verification strategy
- 9 System design
- 10 Implementation
  - 10.1 Back-end services
  - 10.2 The formula language
  - 10.3 Conflict detection
  - 10.4 Version control
  - 10.5 Ask TARA
  - 10.6 Indian calendar and formatting
  - 10.7 Synthetic demonstration data
- 11 What we built: module by module
  - 11.1 Public pages
  - 11.2 1. Main dashboard
  - 11.3 2. Semantic layer: metrics catalog and metric detail
  - 11.4 3. Datasets: catalog, upload, connections, detail
  - 11.5 4. TARA metric generator
  - 11.6 5. Conflict detection center and conflict detail
  - 11.7 6. Formula Lab
  - 11.8 7. Version control
  - 11.9 8. Ask TARA
  - 11.10 9. Dashboards: list, templates and builder
  - 11.11 10. Performance evaluation
  - 11.12 11. Reports and exports, profile, administration, settings, activity
  - 11.13 Mobile layouts
- 12 Why we built it

    - 12.1 The business problem
    - 12.2 Who benefits
    - 12.3 Why a governed layer plus AI
- 13 What we used and why
- 14 Key design decisions and trade-offs
- 15 Results
    - 15.1 Test results
    - 15.2 Ingestion and query performance
    - 15.3 Dashboards and Ask TARA
    - 15.4 Quality measures from the Evaluation module
    - 15.5 Verified production topology
- 16 Limitations
- 17 Conclusion
- 18 Future scope
- 19 References

1 Abstract

Organisations routinely discover that the same business word, most often “revenue”, means different things to different departments. In a retail company the Sales team may report revenue as the taxable value of goods sold, the Finance team may subtract refunds, and the Marketing team may quote the customer invoice value that includes Goods and Services Tax (GST). Each number is defensible; together they erode trust in every dashboard.

This project, **IntelliMetrics AI**, is a full-stack business-intelligence platform built around a *governed semantic layer*: each business metric has exactly one published definition, every change is an immutable version, definitions are validated against the real data schema, conflicting definitions are detected automatically, and anyone can ask questions in plain English through **TARA**, an AI assistant that is architecturally prevented from inventing numbers. TARA only produces a structured *query plan*; the figures always come from executing governed metric formulas through a restricted formula language that is parsed to an abstract syntax tree and compiled to parameterised, read-only DuckDB SQL.

The system comprises a React/TypeScript front end (25 page components), a FastAPI back end (96 endpoints in 16 routers, 20 database tables, 19 role capabilities), Celery/Redis background ingestion, and PostgreSQL storage. It is demonstrated on a fictional Indian retailer, *Bharat Retail Pvt Ltd*, using 62,000 synthetic sales lines expressed in rupees with GST slabs, festival seasonality and an April-to-March financial-year calendar.

Measured results on the development machine: a 62,000-row CSV is ingested in the background in about 955.3 ms; governed queries run in a median of 6.4 ms (p95 12.8 ms); the dashboards load all widgets in a median of 174.9 ms; the deterministic conflict detector reaches precision and recall of 1.0 / 1.0 on 27 labelled cases, against 0.5 / 0.8 for a plain text comparison. 325 automated tests (136 pytest, 136 pytest re-run on real PostgreSQL, 40 Vitest, 13 Playwright end-to-end) pass with 0 failures. The Results and Limitations sections and the separate Testing Report state precisely what was and was not verified.

Keywords: semantic layer, metric governance, conflict detection, text-to-SQL, AI assistant, DuckDB, FastAPI, React, GST, financial year.

2 Introduction

2.1 Background and motivation

Business intelligence tools make it easy to draw a chart, but they do not decide *what a number means*. When definitions live inside individual dashboards and spreadsheets, two people can open two reports titled "Revenue" and see two different values. The cost is not only confusion: decisions are delayed while analysts reconcile figures, and trust in analytics falls.

A **semantic layer** addresses this by placing metric definitions in one governed place, outside any particular dashboard [1][2][3]. Recent large-language-model (LLM) assistants raise the stakes: an assistant that translates English to SQL can answer faster than any analyst, but it can also answer *wrongly with confidence* [5][6][7][8]. The combination studied here is therefore: a semantic layer for correctness, and an AI assistant that is constrained to use it.

2.2 Why an Indian setting

The demonstration organisations are fictional Indian businesses so that the example is concrete for an evaluator in India: amounts are in rupees with lakh/crore grouping (for example ₹12,45,300), GST slabs of 0, 5, 12, 18 and 28 per cent [13] determine the difference between "revenue excluding GST" and "revenue including GST", the financial year runs from 1 April to 31 March [14] so "last quarter" has a precise Indian meaning, sales spike around Diwali, Dussehra, Pongal/Sankranti, Eid and Holi, and payment methods such as UPI and cash-on-delivery (COD) behave differently in returns.

3 Problem statement

> Departments define the same business metric differently; there is no single governed definition, no automatic way to find the disagreement, no safe way to let non-technical users query data in natural language, and no audit trail of how a definition changed.

Concretely, in the demonstration company (see “Why we built it”):

- the **official** *Revenue* is `SUM(net_sales)`, the taxable value after discounts and **excluding GST**;
- **Finance** defines *Revenue* as `SUM(net_sales) - SUM(refund_amount)` so that it reconciles to the ledger;
- **Marketing** quotes *Revenue* as `SUM(net_amount)`, the invoice value **including GST**;
- Marketing also counts *Orders* with `COUNT(order_id)` (order lines) while the official definition is `COUNT_DISTINCT(order_id)`;
- Operations expresses *Gross Margin %* as a 0 to 1 ratio instead of a percentage.

4 Objectives

1. Provide a **governed semantic layer** with exactly one published definition per metric, an approval workflow, dependency tracking and immutable version history.
2. **Ingest large datasets** (CSV, Excel, JSON, Parquet, PostgreSQL, MySQL) in background workers, never in the web request, with schema detection and profiling.
3. **Detect conflicting definitions** deterministically across departments, with explanations, severity, recommended resolution and a full audit-logged workflow.
4. Let analysts **validate and execute formulas safely**: a real parser and AST compiled to parameterised SQL; raw SQL is never executed.
5. Offer **TARA**: a metric generator grounded in the real schema and catalog, and a natural-language BI chat that cannot fabricate data and never auto-publishes anything.
6. Support **dashboards** (KPI, line, bar, pie, table), exports (PNG/PDF/CSV, CSV/Excel/PDF reports) and a measured **evaluation** module.
7. Enforce **role-based access control and workspace isolation** on every endpoint, with an audit log of sensitive actions.
8. Deliver a **polished, accessible, themeable** interface (light/dark/system), and make the whole stack reproducible with Docker Compose, automated tests and documentation.
9. Adopt **Indian conventions** throughout: INR with lakh/crore grouping, April-March financial year, IST, GST.

5 Scope

In scope. Everything listed in the objectives, implemented as working software: 25 front-end page components (public pages, 20 application pages, 404 and 403), 96 API endpoints, two demonstration workspaces, four roles, synthetic Indian data.

Out of scope or not claimed.

- This is a university project, not a commercial product: there are no plans, pricing, billing or subscriptions anywhere in the product.
- All data is synthetic. No real person, company, phone number or transaction is used.
- E-mail delivery is not integrated; password-reset links are printed to the log and shown in a development-only banner.
- Cross-dataset joins in formulas are not supported (metric references must share a dataset).
- Live OpenAI/Gemini calls were not exercised because no API key was available; those code paths are tested with a mocked provider (see Limitations).

6 Background and literature review

6.1 Semantic layers and metric governance

A semantic layer is a business-facing abstraction over physical tables in which metrics (measures) and dimensions are defined once and reused by every consumer. Commercial and open-source implementations include Looker's LookML [3], dbt's MetricFlow-based semantic layer [1] and Cube [2]. They share the principle adopted here: *definitions are code-like, reviewable, versioned artefacts*. IntelliMetrics adds two governance capabilities that the surveyed tools treat as external processes: an in-product approval workflow with exactly-one-live-definition enforced by a database constraint, and automatic *conflict detection* between departmental variants.

6.2 Text-to-SQL and its risks

Benchmarks such as Spider [5] and BIRD [6] show that LLM-based text-to-SQL is useful but far from perfectly reliable on realistic schemas, and later evaluations study prompt design and error modes [7]. LLMs are also known to *hallucinate*, producing fluent but unsupported output [8]. Two further risks matter for enterprises: generated SQL can be unsafe (injection, expensive scans, data exfiltration) and can silently use a different definition of a metric than the organisation's. The design response in this project is to **never let a model write SQL or supply numbers**: the model (or an offline rule engine) emits a *plan* over catalog objects; the plan is validated; execution uses the same compiler as the Formula Lab.

6.3 Safe query execution

SQL injection remains a top web-application risk [9]. The standard defence is parameterised queries, and the stronger defence used here is to avoid accepting SQL at all: the user language is a small, typed expression grammar. Parsing and type-checking follow classical compiler construction: lexical analysis, recursive-descent parsing into an abstract syntax tree and semantic checks [12].

6.4 Columnar analytics with DuckDB and Parquet

DuckDB is an embeddable analytical database with vectorised execution [4]. Together with the Parquet columnar format it allows interactive aggregation over tens of thousands to millions of rows inside an ordinary application process, without operating a separate warehouse. This is why uploaded files are converted to compressed Parquet in a background job and queried through read-only DuckDB views.

6.5 Access control

Role-based access control assigns permissions to roles and roles to users [10]. Here the permission set is a single table of capabilities (`permissions.py`) shared by the back end (enforcement on every endpoint) and the front end (route guards and navigation), supplemented by tenant isolation: every row carries a workspace identifier and cross-workspace object access returns *not found* so that existence is never leaked.

6.6 Indian business context

GST has the rate slabs 0, 5, 12, 18 and 28 per cent [13]; whether a revenue figure includes GST changes it by up to a quarter, which is exactly the kind of silent definitional difference a semantic layer should expose. The financial year is defined in the Companies Act, 2013 as ending on 31 March [14]; planning and reporting therefore follow Q1 April-June, Q2 July-September, Q3 October-December and Q4 January-March, a convention TARA must understand when asked for “last quarter”.

7 Existing system versus proposed system

Aspect	Typical existing practice	IntelliMetrics AI (proposed)
Where metric logic lives	Inside dashboards, spreadsheets, ad-hoc SQL	One governed definition per metric in the semantic layer
Detecting disagreement	Noticed by a person when numbers do not match	Automatic structural comparison (formula, aggregation, filters, unit, window, dataset) with severity and recommendation
Change history	Overwritten files, undocumented edits	Immutable versions, diff view, rollback as a *new* version, audit timeline
Approval	E-mail or verbal	In-product submit / approve / reject / publish with comments
Natural-language access	Generic LLM text-to-SQL (risk of wrong or unsafe SQL, hallucinated numbers)	TARA builds a query plan over approved metrics only; the query plan is validated; same safe compiler; refuses when data cannot answer
Query safety	Trust in whoever writes SQL	Restricted grammar, AST, parameters, timeouts, row limits, read-only views
Large data	Manual exports, memory-bound tools	Background ingestion to Parquet, chunked, progress tracking
Tenancy and roles	Often shared dashboards	Workspaces with strict isolation and four roles
Fiscal calendar	Calendar-year assumptions	Indian April-March year, fiscal quarters, festival seasons, IST

8 Methodology

8.1 Development approach

The project followed an iterative, test-first-where-practical process in ordered stages: scaffold and containers, data model and migrations, authentication and RBAC, design system and layout, dashboard, dataset ingestion, semantic layer with versioning, formula compiler and lab, conflict detection, TARA generator, Ask TARA, dashboard builder, evaluation, reports, administration, automated tests, polish and documentation. A second iteration localised the whole product to India (data, formatting, calendar, copy), removed all commercial wording, and produced this report pack.

8.2 Requirements to design

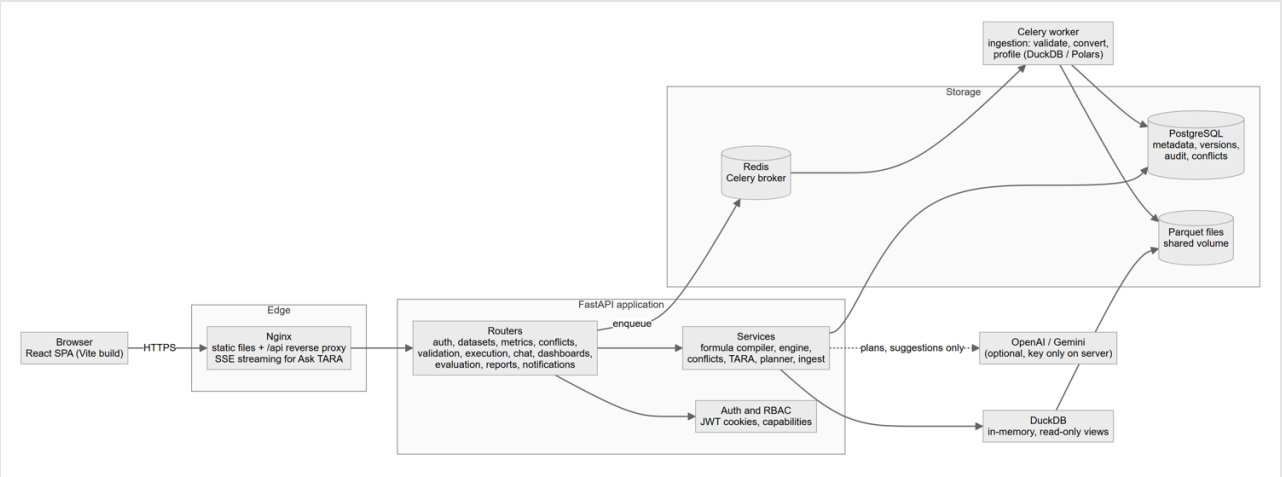
Requirements were captured as: the 20 application pages, the roles and permissions, the safety rules (no raw SQL, AI never auto-publishes, TARA never fabricates), and measurable quality targets. Each requirement was given at least one automated test, and each architectural decision was recorded (see Key design decisions and trade-offs).

8.3 Verification strategy

Four layers of automated tests (see Results and the Testing Report) plus manual and scripted verification of the production topology (PostgreSQL, Redis, Celery, MySQL connector) on the development machine. Screenshots in this report are produced by a Playwright script from the running application, not mocked.

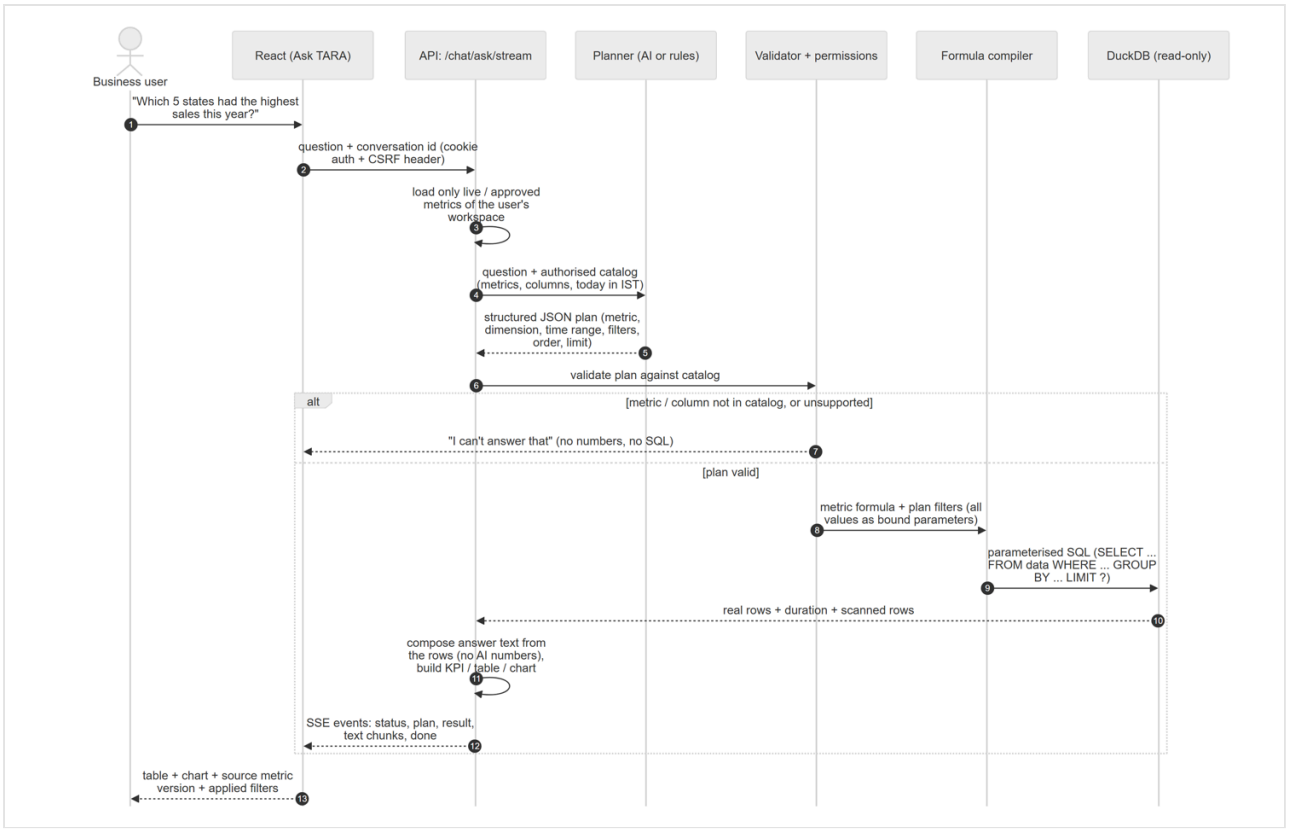
9 System design

The detailed diagrams are in the *System Architecture* document. The essentials:



High-level architecture

- **Frontend:** React 18 single-page application; TanStack Query for server state; Tailwind with CSS variables for the black-and-white brand and one violet accent; Framer Motion for transitions; Recharts for charts.
- **Backend:** FastAPI application with 15 routers; SQLAlchemy 2 models and Alembic migrations; consistent error envelope with request ids; structured JSON logs.
- **Data plane:** uploaded files become Parquet files on a shared volume; DuckDB opens them as read-only views per query; PostgreSQL stores metadata, versions, conflicts, audit and conversations.
- **Jobs:** Celery workers (Redis broker) perform ingestion; in single-process development a background thread plays the same role.
- **AI:** OpenAI or Gemini through a thin JSON client (keys only on the server), always optional; a deterministic offline mode is the default and is labelled wherever it speaks.



Request flow of an Ask TARA question

10 Implementation

10.1 Back-end services

Service (module)	Responsibility
`formula.py`	Lexer, recursive-descent parser, typed AST, schema/type/aggregation checks, metric-reference inlining, SQL generation with bound parameters
`engine.py`	Builds the complete query (dimensions, grains, time range, filters, order, limit), runs it on DuckDB with a timeout (interrupt) and `LIMIT n+1` truncation detection, records every execution
`metrics.py`	Create/update/submit/approve/reject/publish/deprecate/rollback; creates immutable versions; one live definition per key
`conflicts.py`	Pairwise structural diff (AST-normalised), severity, recommendation, fingerprint de-duplication, resolution actions, optional TARA semantic review
`tara.py`	Grounded metric generator: schema + catalog context, Pydantic validation, compiler validation, one self-repair round, offline deterministic fallback
`planner.py`	Natural-language question to validated query plan; Indian fiscal calendar; festival windows; comparisons; two-dimension splits
`ingest.py`	Validation, DuckDB/Polars conversion to Parquet, schema detection, profiling, PostgreSQL/MySQL chunked import
`evaluation.py`	Measures processing, latency, validation, success, conflict precision/recall, AI acceptance, dashboard load; stores runs
`reports.py`	CSV, Excel and PDF exports with INR and IST formatting
`calendar_in.py` , `formatting.py`	Financial year, fiscal quarters, festival windows, IST; lakh/crore and rupee formatting shared by exports and TARA text

10.2 The formula language

The restricted grammar (full text in the Architecture document) allows aggregates `SUM, AVG, MIN, MAX, COUNT, COUNT_DISTINCT, MEDIAN, STDDEV, SUMIF, COUNTIF, AVGIF`, scalar helpers `ABS, ROUND, COALESCE, IF, NULLIF, UPPER, LOWER, YEAR, MONTH, LENGTH`, arithmetic, comparisons, `IN`, `BETWEEN`, `LIKE`, `IS NULL`, dates as `DATE 'YYYY-MM-DD'`, metric references such as `[Revenue]`, and an optional trailing `WHERE`. Examples used in the demonstration:

```
SUM(net_sales)
SUM(net_sales) - SUM(refund_amount)
SUMIF(net_sales, payment_method = 'UPI') / SUM(net_sales) * 100
COUNTIF(refund_amount > 0 AND payment_method = 'COD') / COUNTIF(payment_method = 'COD') * 100
[Revenue] / [Orders]
```

Compilation guarantees (each covered by tests): comments and `;` are rejected; only whitelisted functions exist; every identifier is validated against the dataset schema and quoted; every literal becomes a `?` parameter; division is `x / NULLIF(y, 0)`; aggregates cannot be nested or mixed with row-level columns; metric references are resolved from live definitions with cycle and depth checks; queries run with a configurable timeout (default 20 seconds) and a row limit (default 10,000).

10.3 Conflict detection

For every group of non-deprecated metrics that share a governed key, the detector compares the published reference (or all pairs when nothing is published) across six dimensions: dataset mapping, aggregation functions, formula (after normalising case and operand order), row filters (as sets of conjuncts, merging the `WHERE` clause and the filters field), unit and time window / date column. Severity is derived from the differences and from whether the variant is already approved. A recommended action (rename the variant, align to published, deprecate, accept both) is attached, and affected metrics and dashboards are listed. A fingerprint prevents duplicates when detection is re-run. Optionally, TARA performs a *semantic* review of metric descriptions and stores suggestions clearly labelled **TARA suggestion**.

10.4 Version control

`metric_versions` rows are inserted once and never changed: an ORM listener and (on PostgreSQL and SQLite) a database trigger reject `UPDATE` and `DELETE`. Editing a published metric creates a new draft version while the previous version stays live, so dashboards never go dark. Rollback creates a *new* version carrying an older definition; for a live metric it becomes live immediately, an approver-only action. Dashboards follow a **version policy**: `latest_published` (default) or `pinned`, in which each widget reads a chosen version snapshot.

10.5 Ask TARA

The pipeline is: question → query plan (AI or deterministic rules) → plan validation against the *authorised catalog* (live or approved metrics of the user's workspace on ready datasets) → compile and execute through the same engine → compose a text answer **from the returned rows** → structured payload with KPI, table, chart, source metric version, applied filters and the parameterised SQL. Answers stream over server-sent events. Questions the data cannot support receive an explanation instead of a guess; a missing metric name may fall back to the governed *Revenue* metric only when the question is clearly a split or comparison, and that assumption is printed in the answer.

10.6 Indian calendar and formatting

- Financial year start year = `year if month >= 4 else year - 1`; labels such as "FY 2025-26"; quarters Q1 Apr-Jun, Q2 Jul-Sep, Q3 Oct-Dec, Q4 Jan-Mar.
- TARA understands *last/this quarter*, *the last quarter of this financial year*, *this/last financial year*, *FY 2025-26*, *FY26*, *Q3 FY26*, *last N days*, month names, and festival seasons (Diwali, Dussehra, Pongal/Sankranti, Eid, Holi, End-of-Season sale). *This year* means the financial year to date. "Compare A with the previous quarter" compares against the fiscal quarter before the one containing A and warns when the periods have unequal length by also showing per-day averages.

- Today is evaluated in IST; every date is displayed as DD-MMM-YYYY; timestamps as DD-MMM-YYYY HH:MM IST. One formatter on the front end (`utils.ts`) and one on the back end (`formatting.py`) implement lakh/crore grouping and compact forms (₹1.2 Cr, ₹45 L).

10.7 Synthetic demonstration data

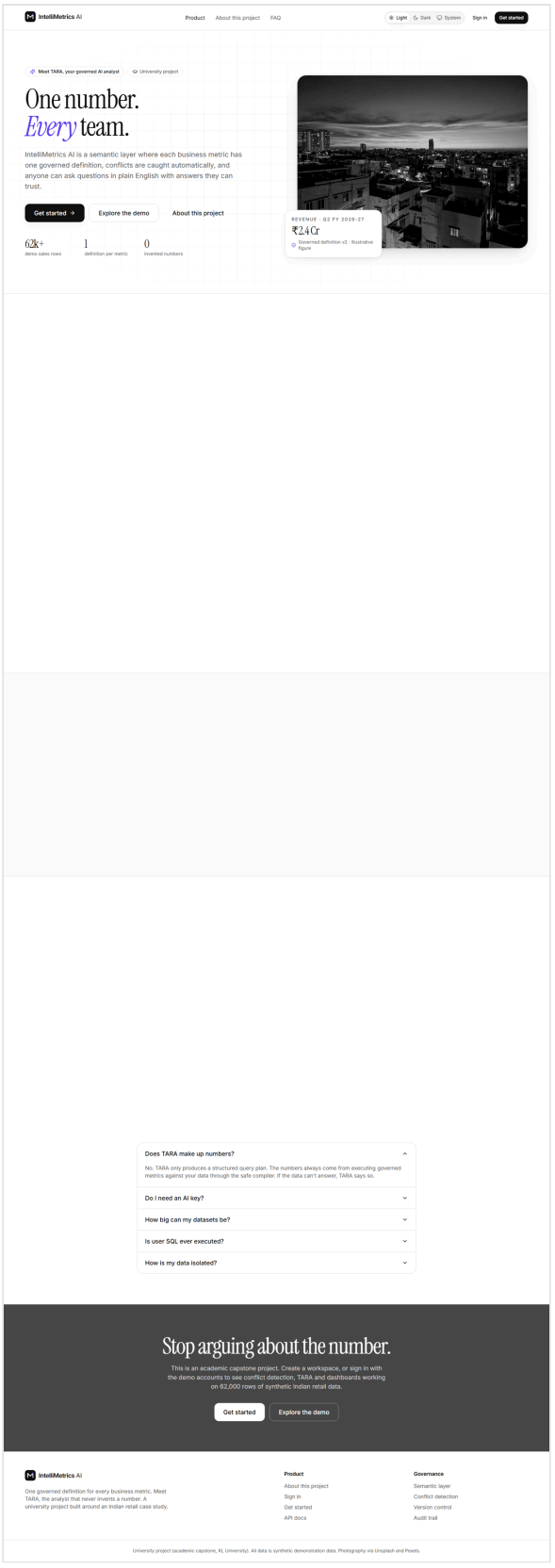
`seed_data.py` generates, reproducibly (fixed random seed), customers with Indian names, cities and states, PIN codes with the correct three-digit prefix of the city, fake `+91 90000 xxxxx` numbers and `example.com` e-mails; 200+ products across eight FMCG, electronics and apparel categories with realistic rupee amounts and GST slabs; 62,000 sales lines across financial years 2024-25 to date with festival-weighted demand, regional boosts (Pongal in the South, Holi in the North and Central), channel-dependent payment mix, higher COD return rates, refund reasons and discount behaviour. The data is labelled **Demonstration data (synthetic)** in the interface.

11 What we built: module by module

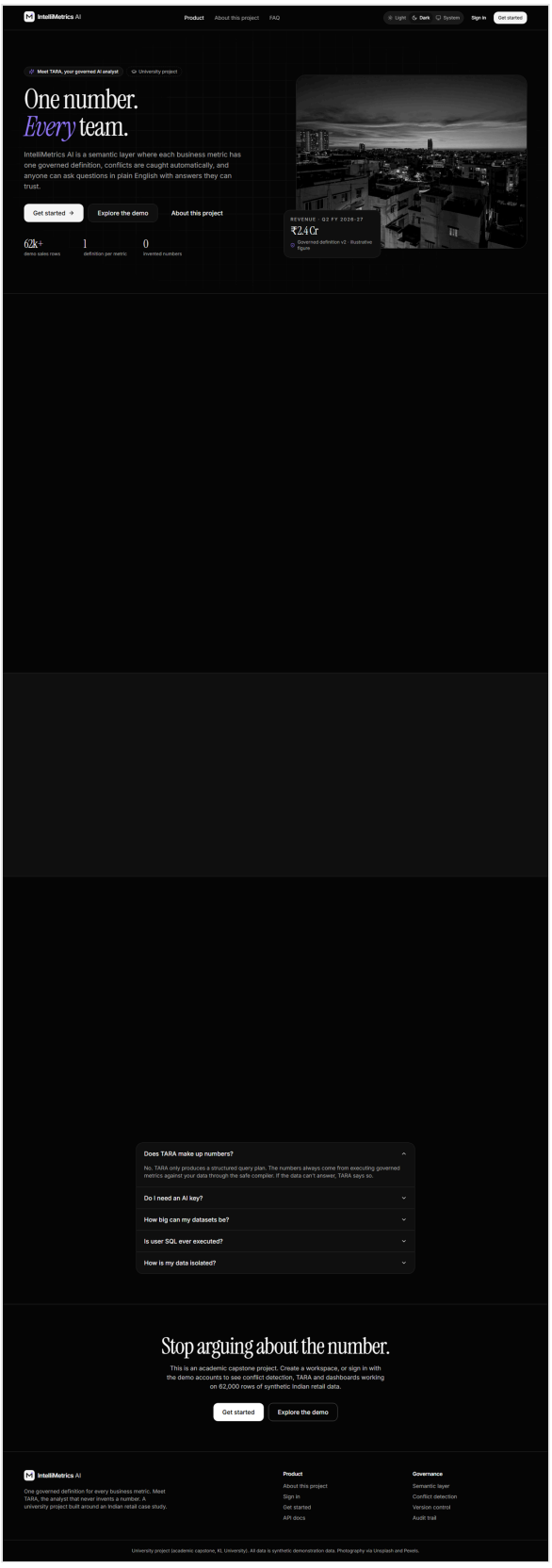
Each subsection lists the page, the problem it solves and what the user can do; screenshots in light and dark mode follow.

11.1 Public pages

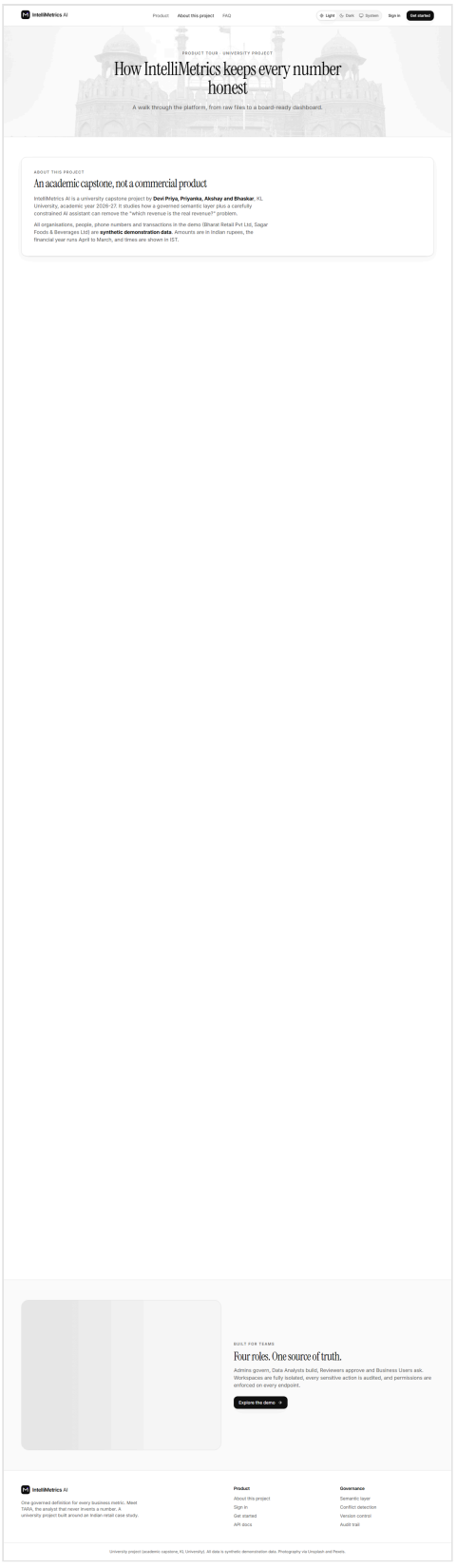
- **Landing page**: hero, platform features, how it works, a TARA showcase, FAQ and calls to action (“Get started”, “Explore the demo”, “About this project”), with a University Project badge and footer note.
- **About / product tour**: a walk through the seven capability areas and an “About this project” section naming the student, guide and institution.
- **Login** with a *Demo accounts* helper that fills the form, **Register**, **Forgot password** (development banner with the reset link) and **Reset password**; **404** and **403** pages.



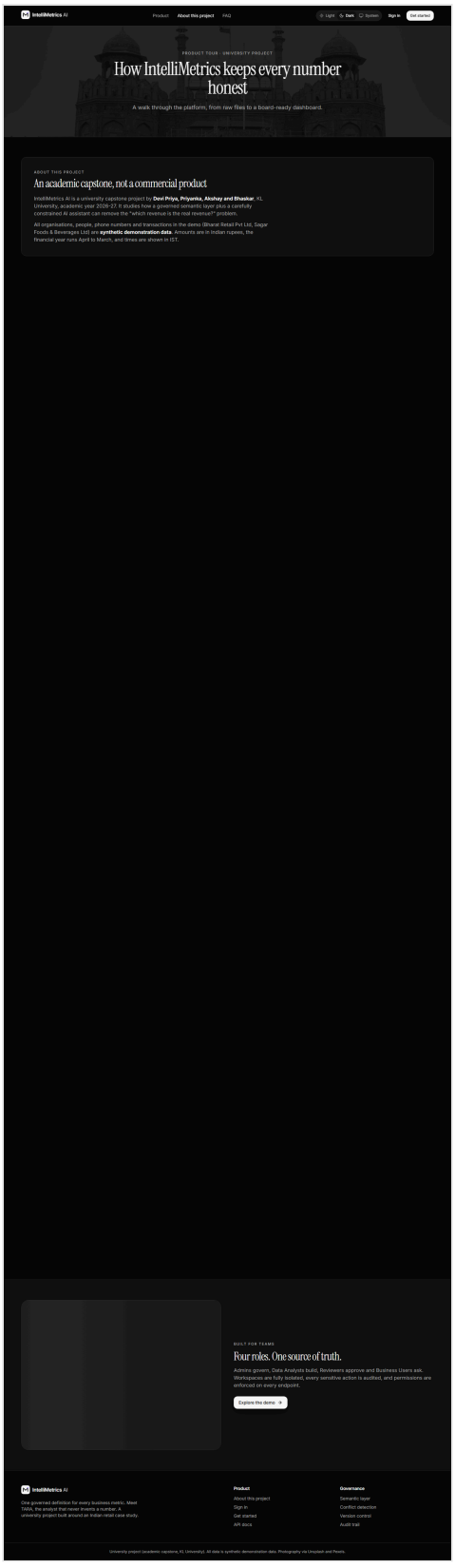
Landing page, light mode



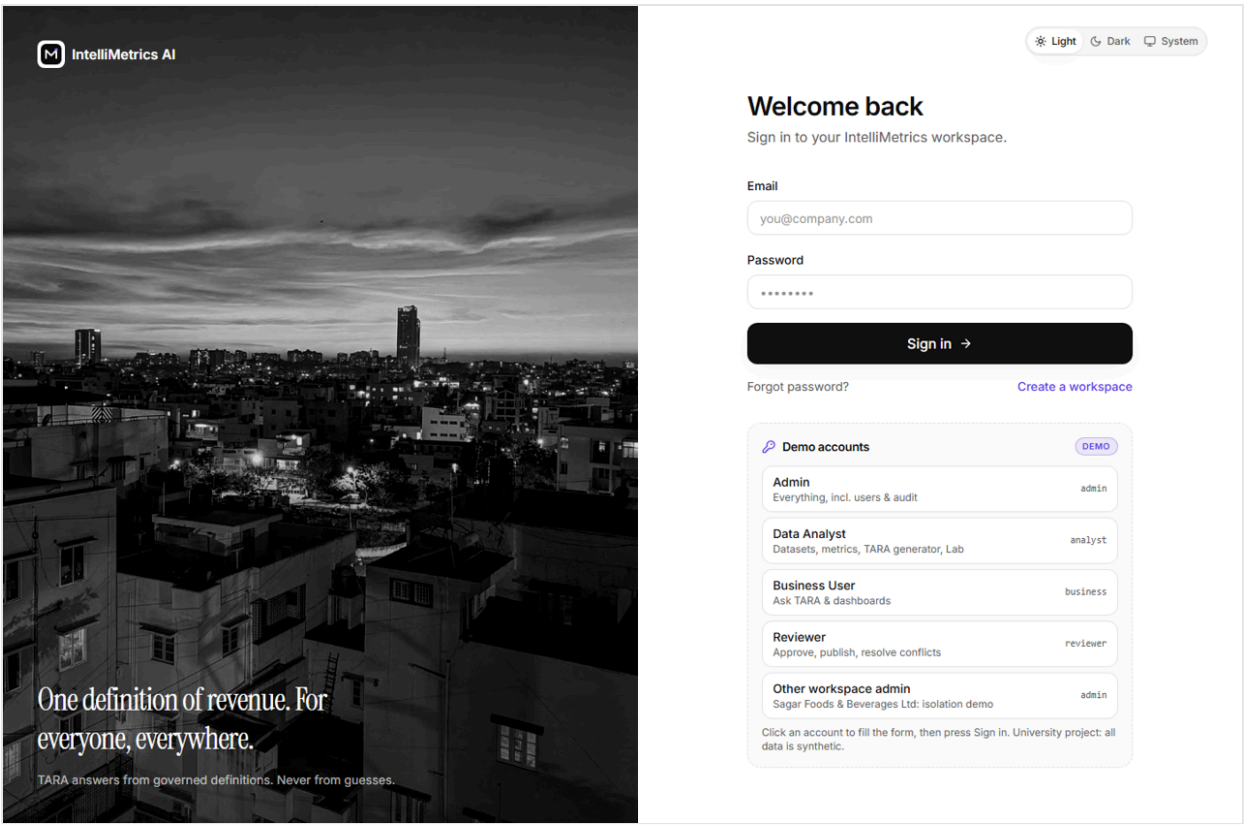
Landing page, dark mode



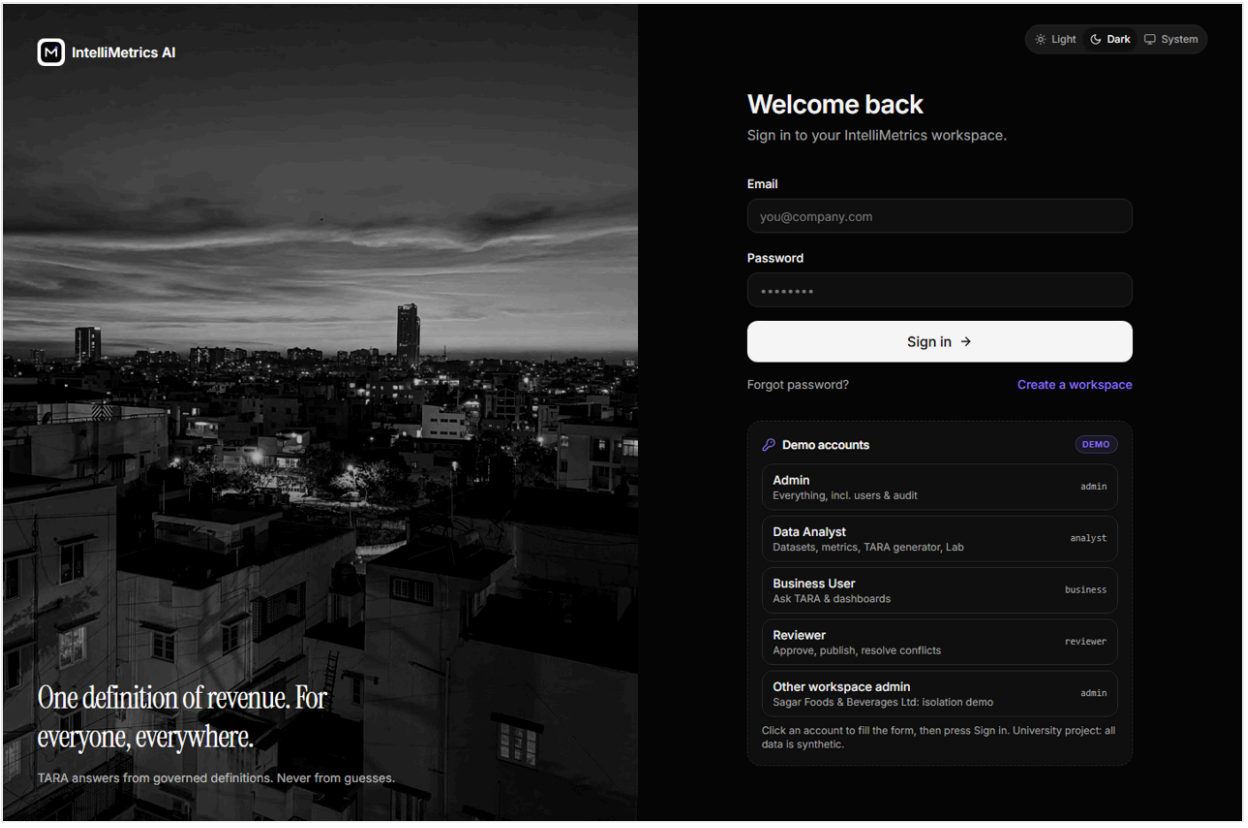
About / product tour, light mode



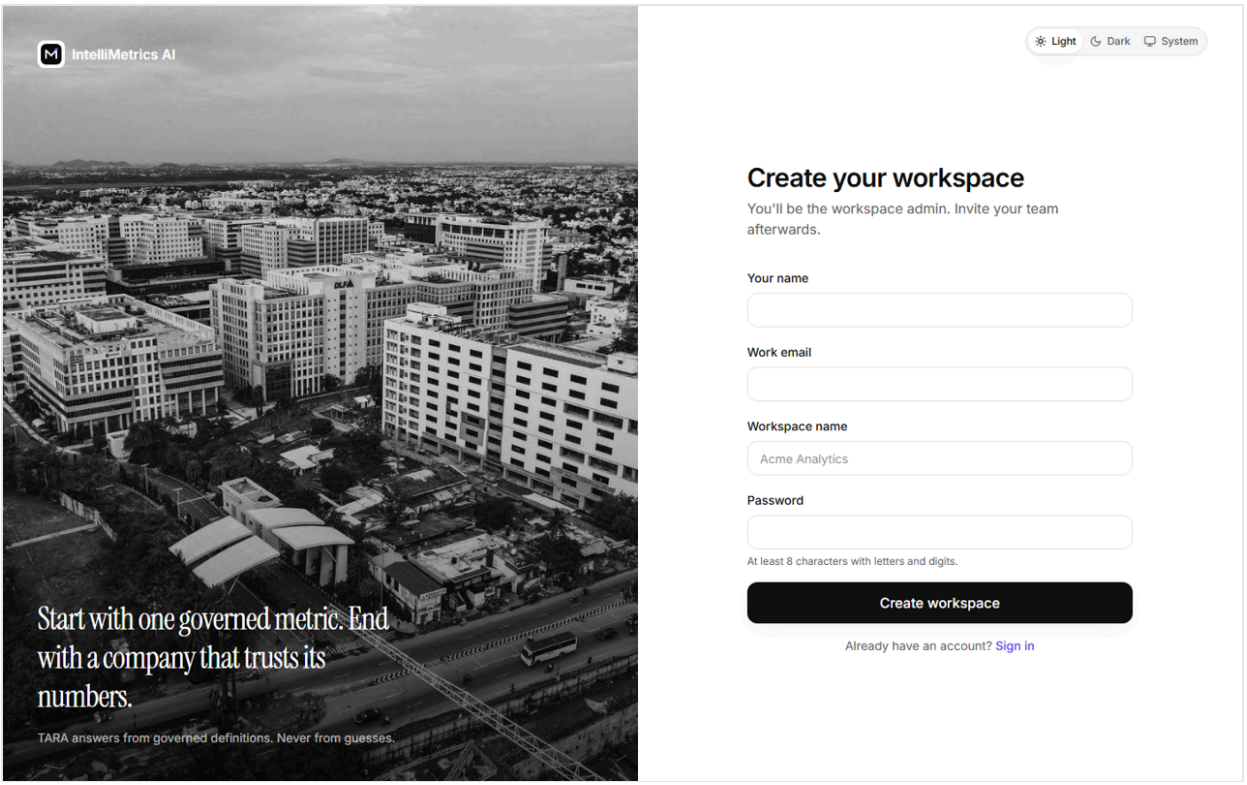
About / product tour, dark mode



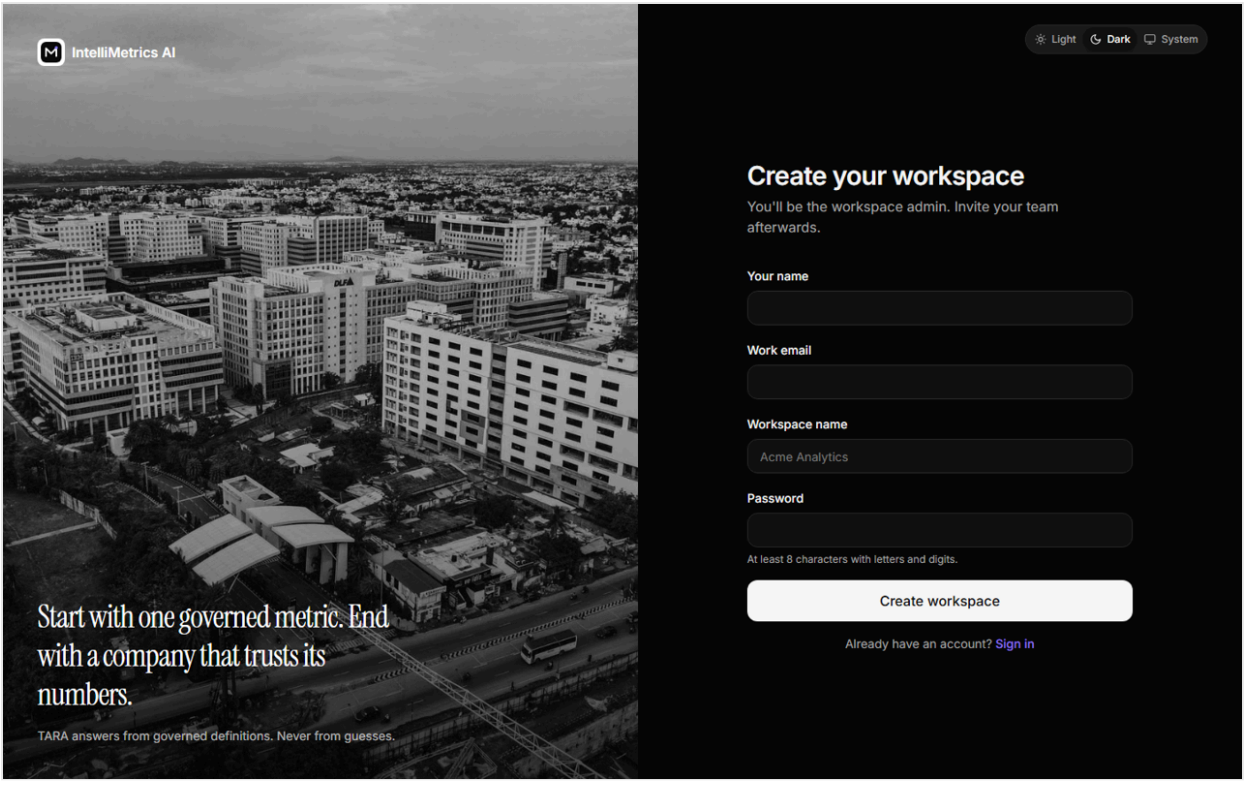
Login with demo accounts panel, light mode



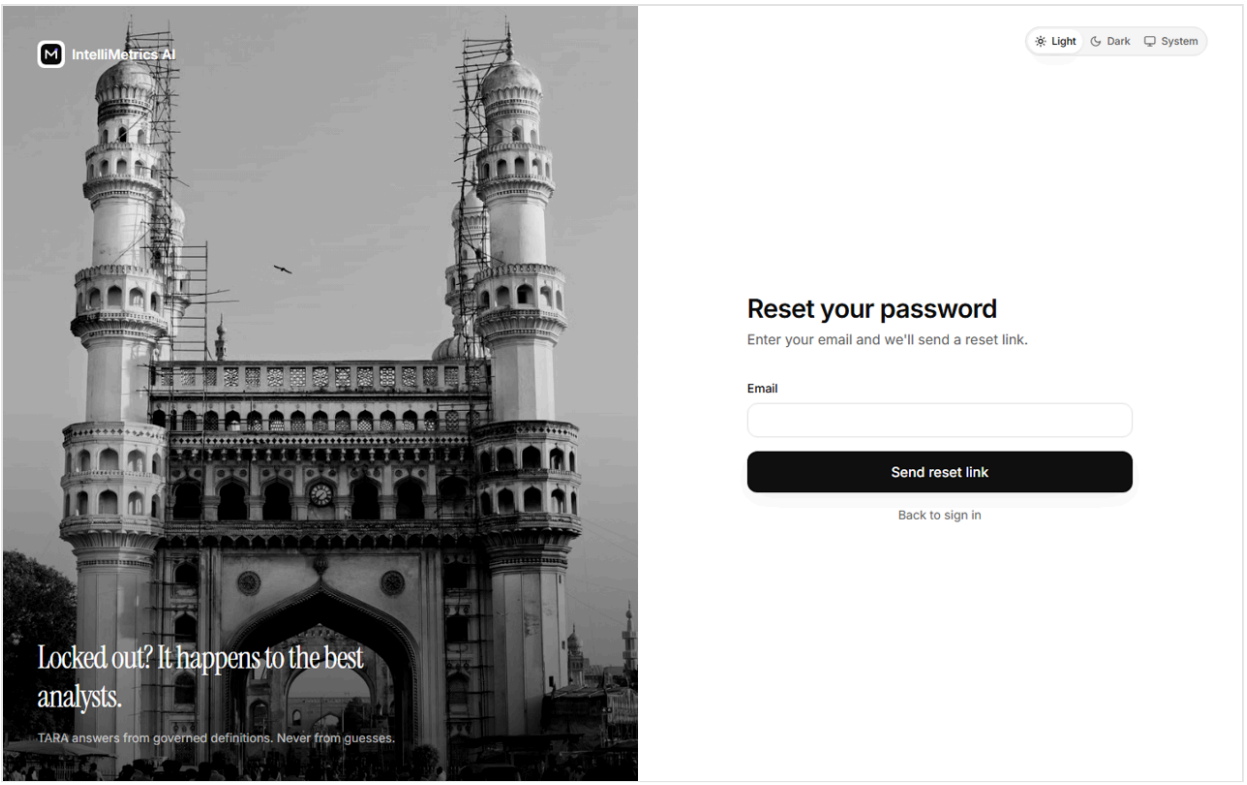
Login with demo accounts panel, dark mode



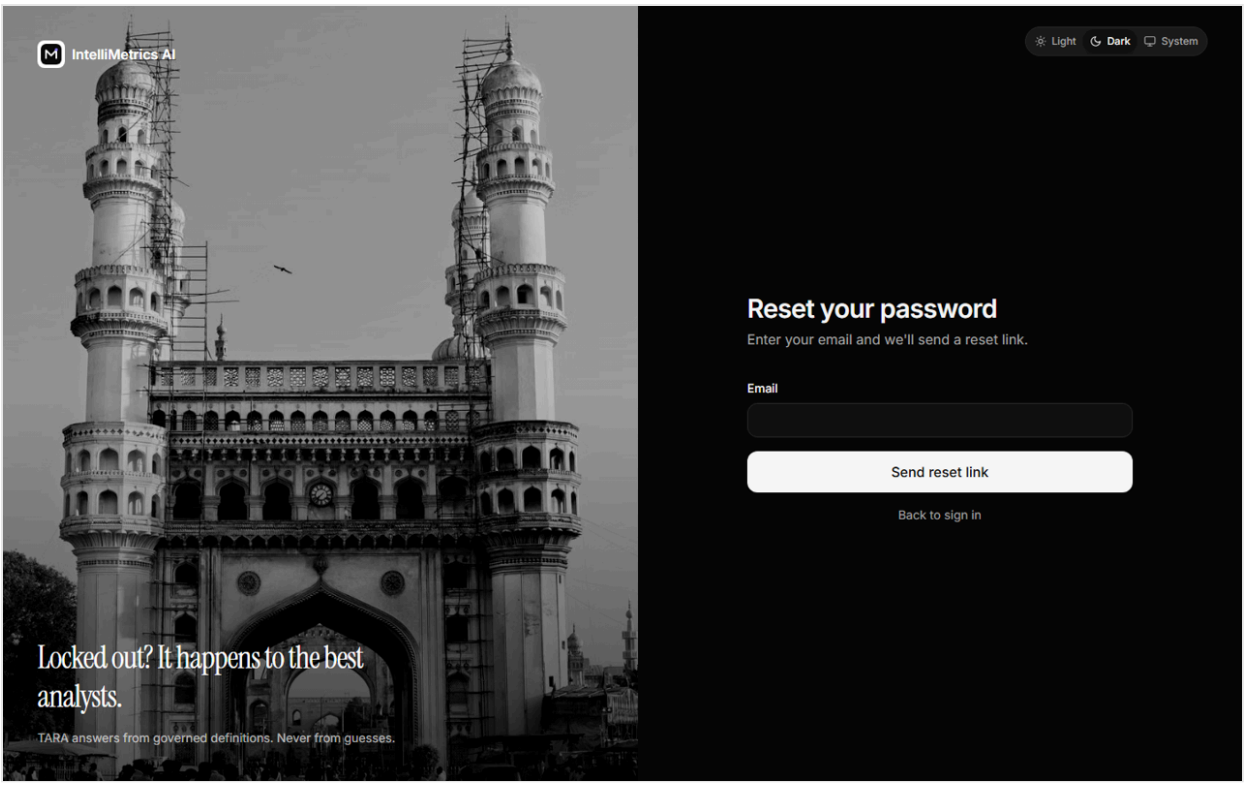
Register, light mode



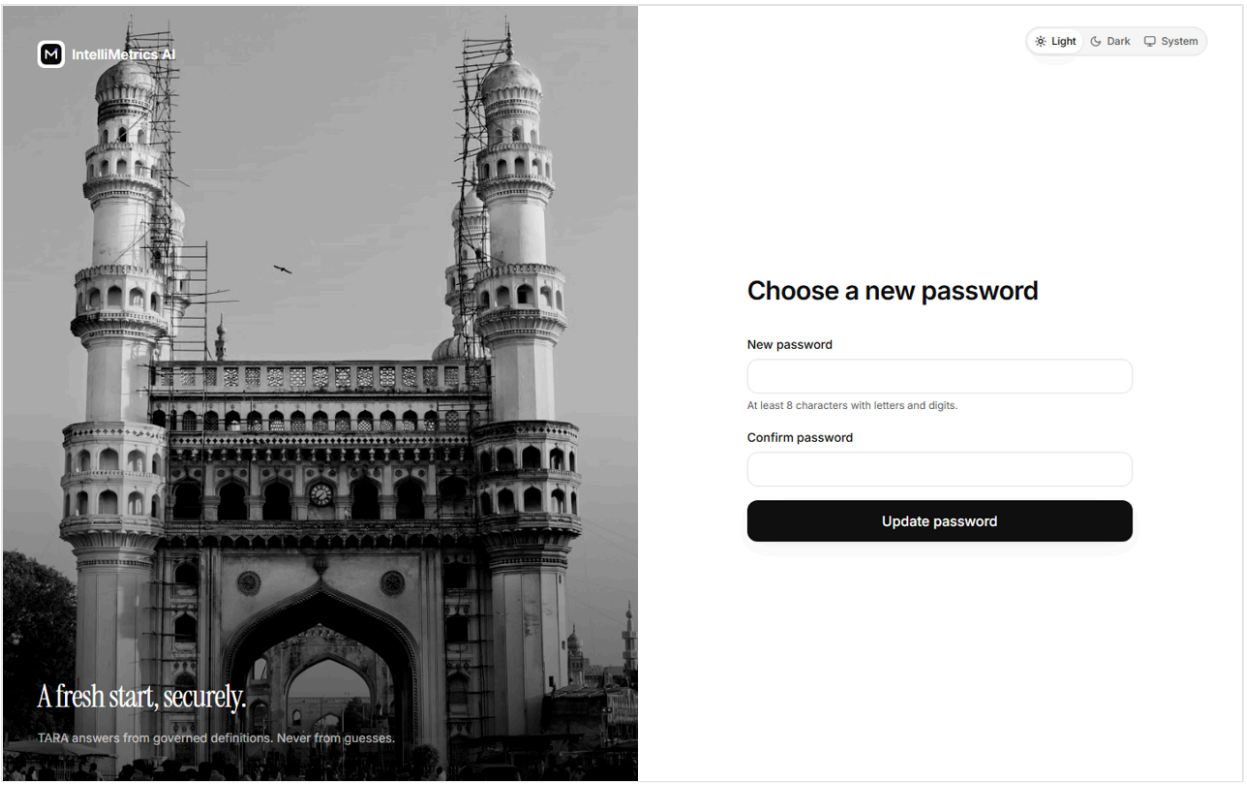
Register, dark mode



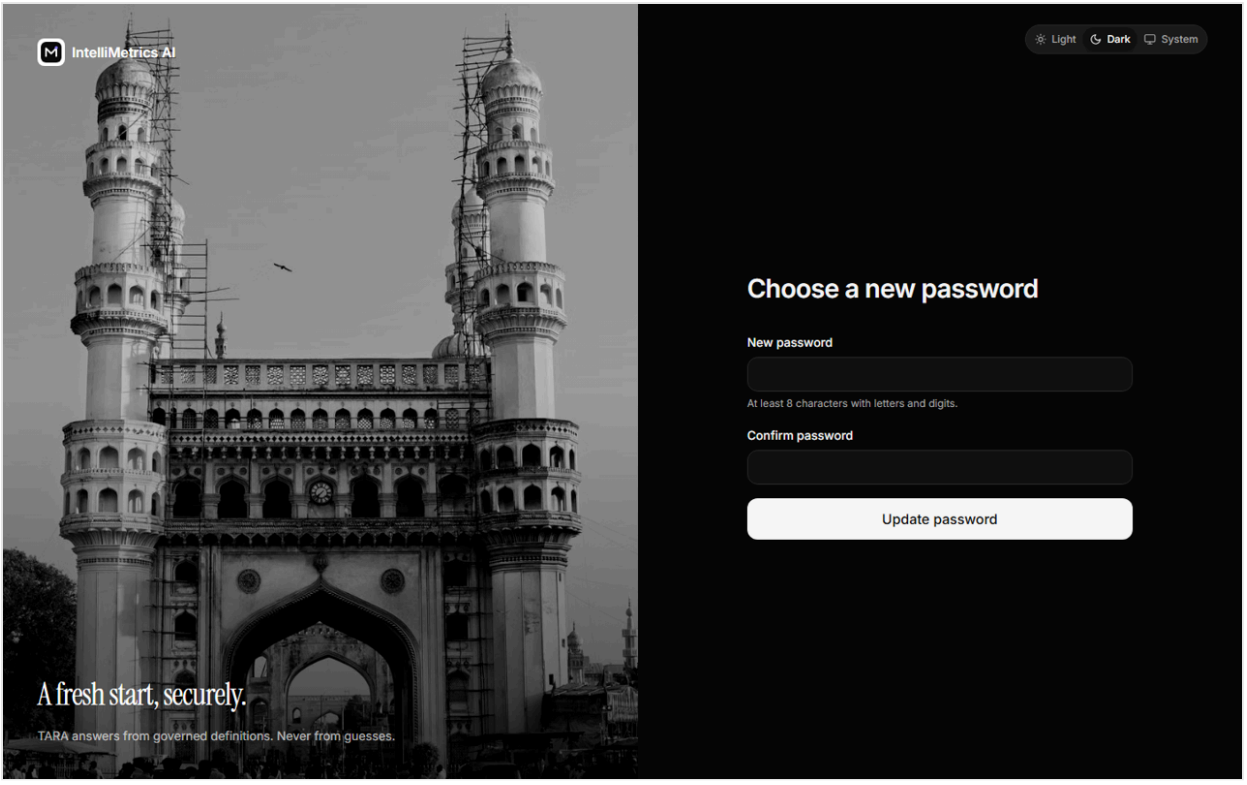
Forgot password, light mode



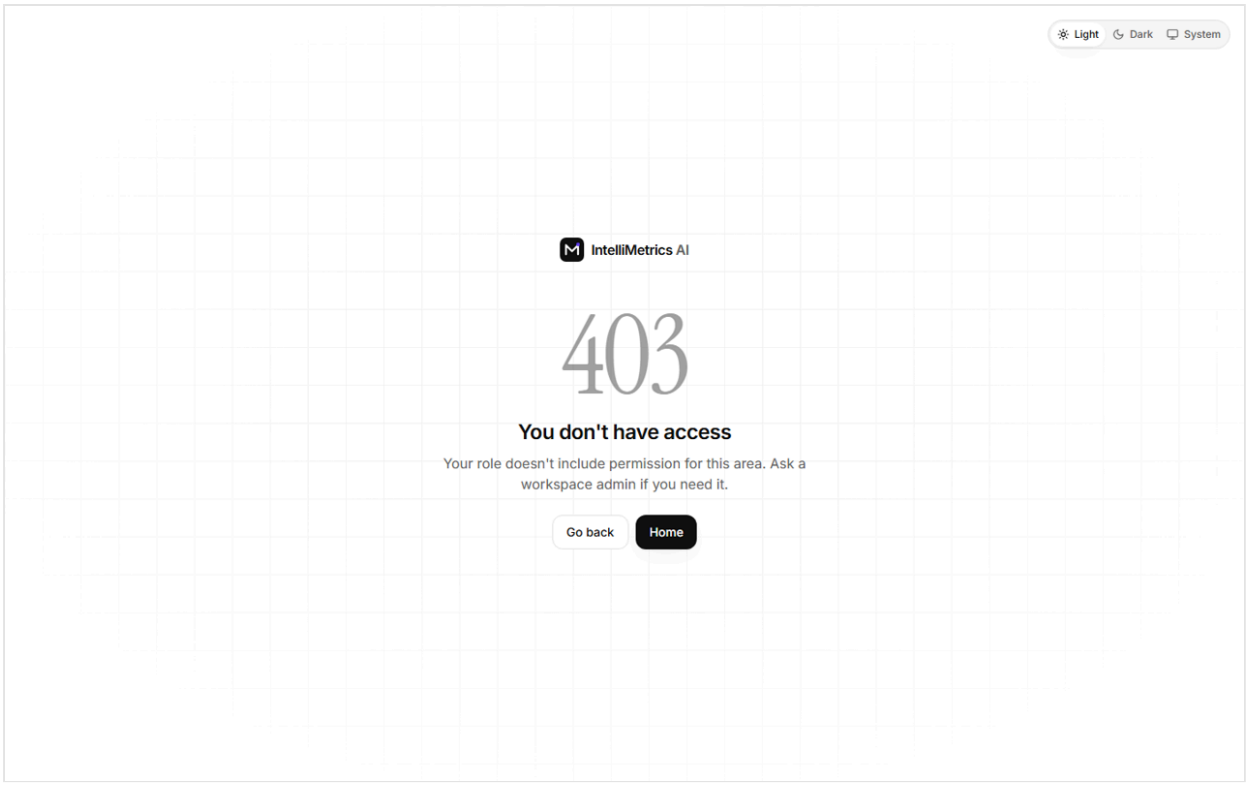
Forgot password, dark mode



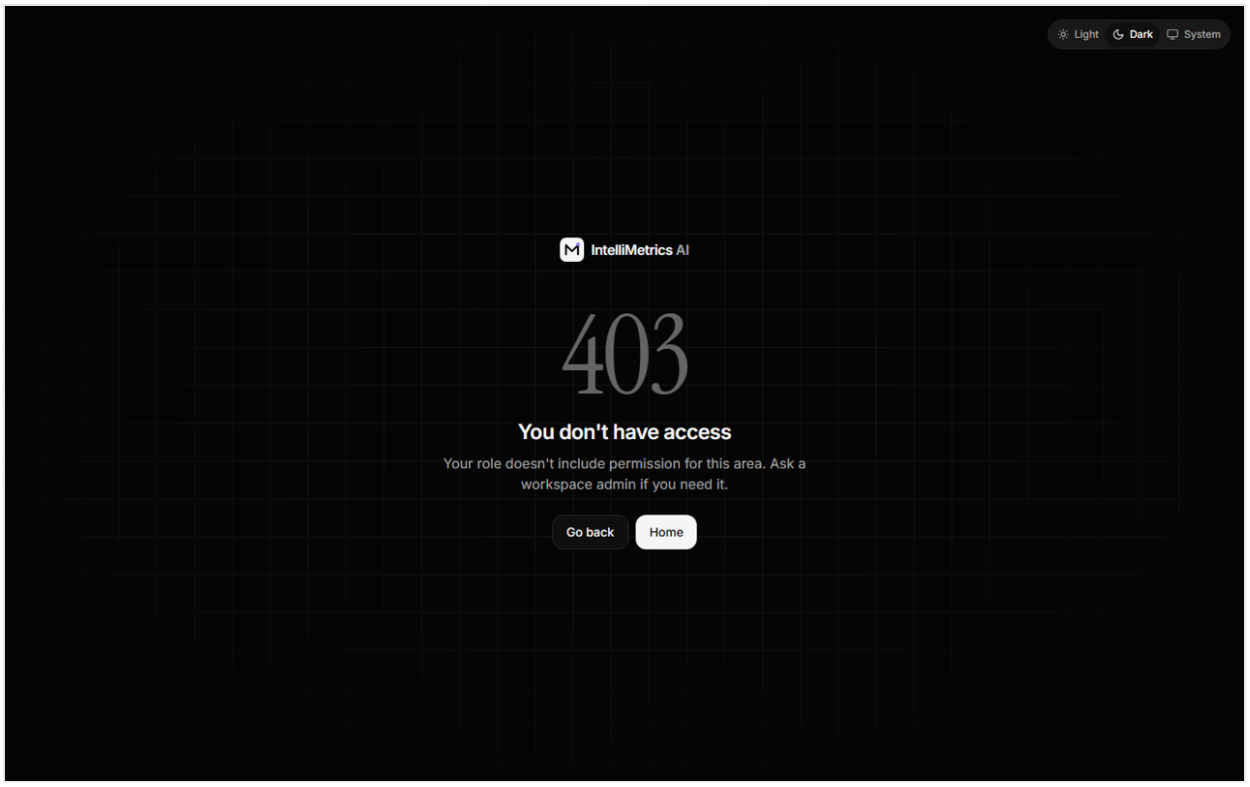
Reset password, light mode



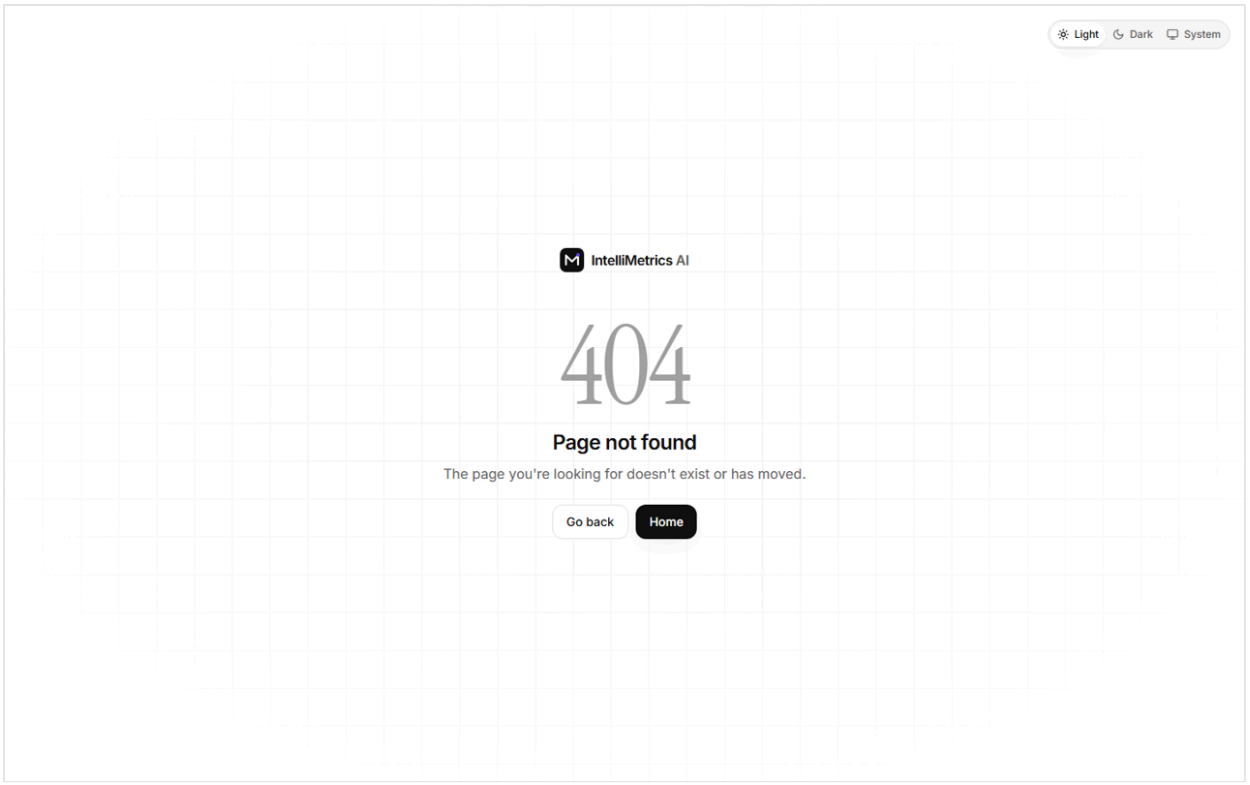
Reset password, dark mode



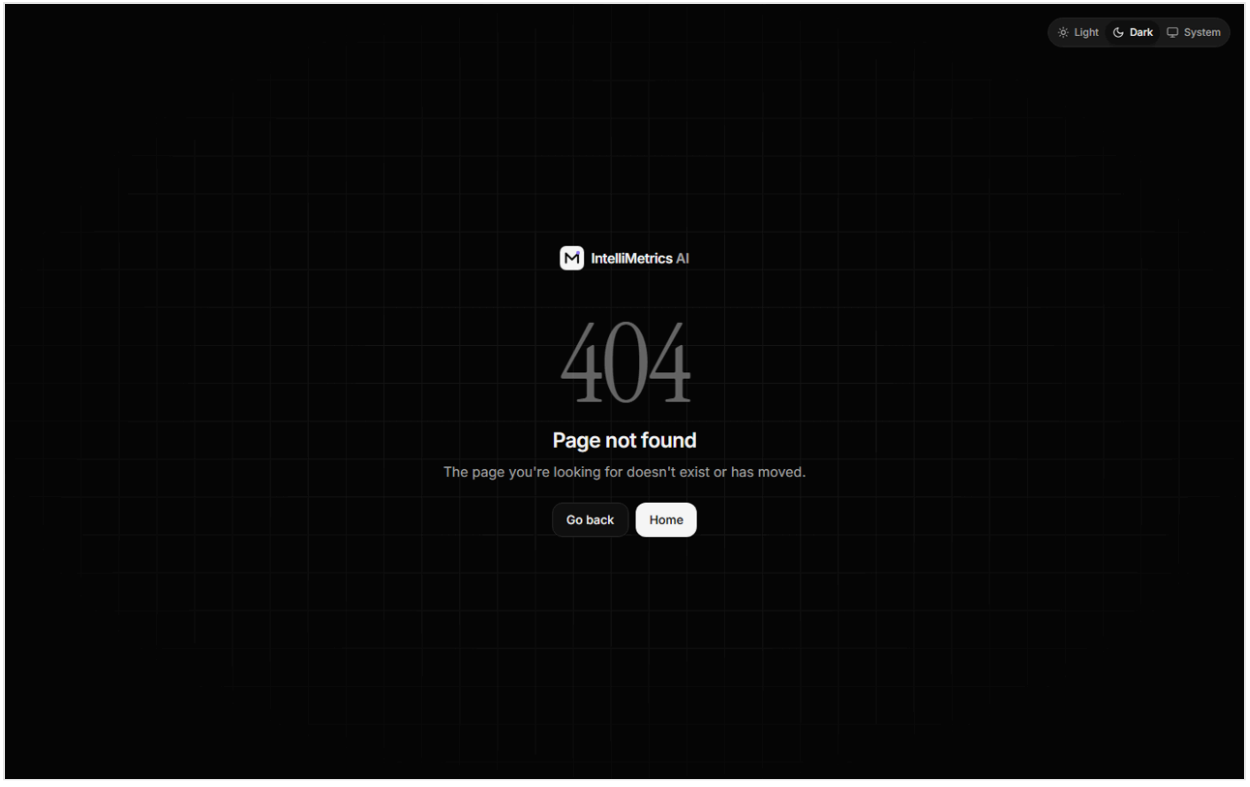
403 Forbidden, light mode



403 Forbidden, dark mode



404 Not found, light mode

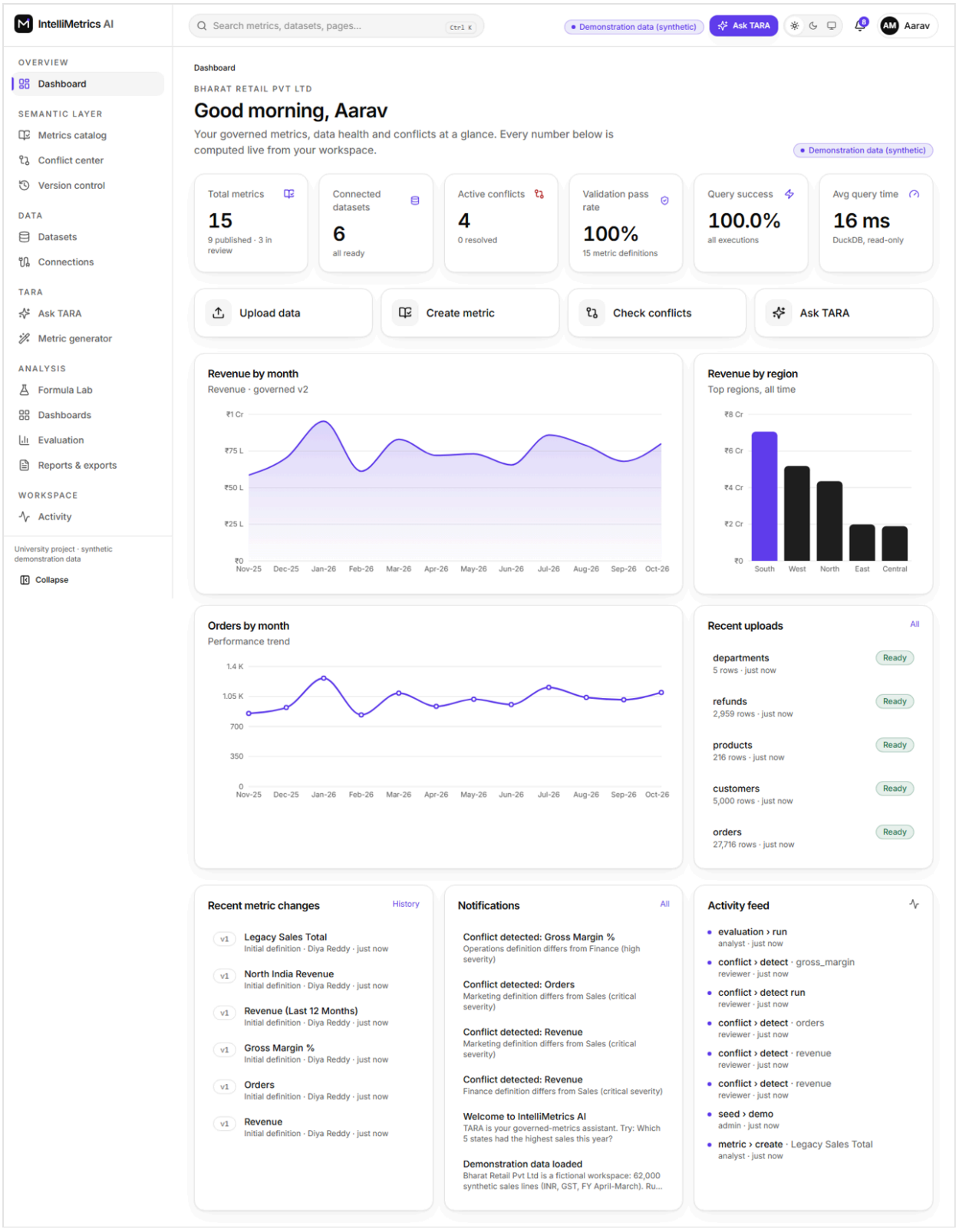


404 Not found, dark mode

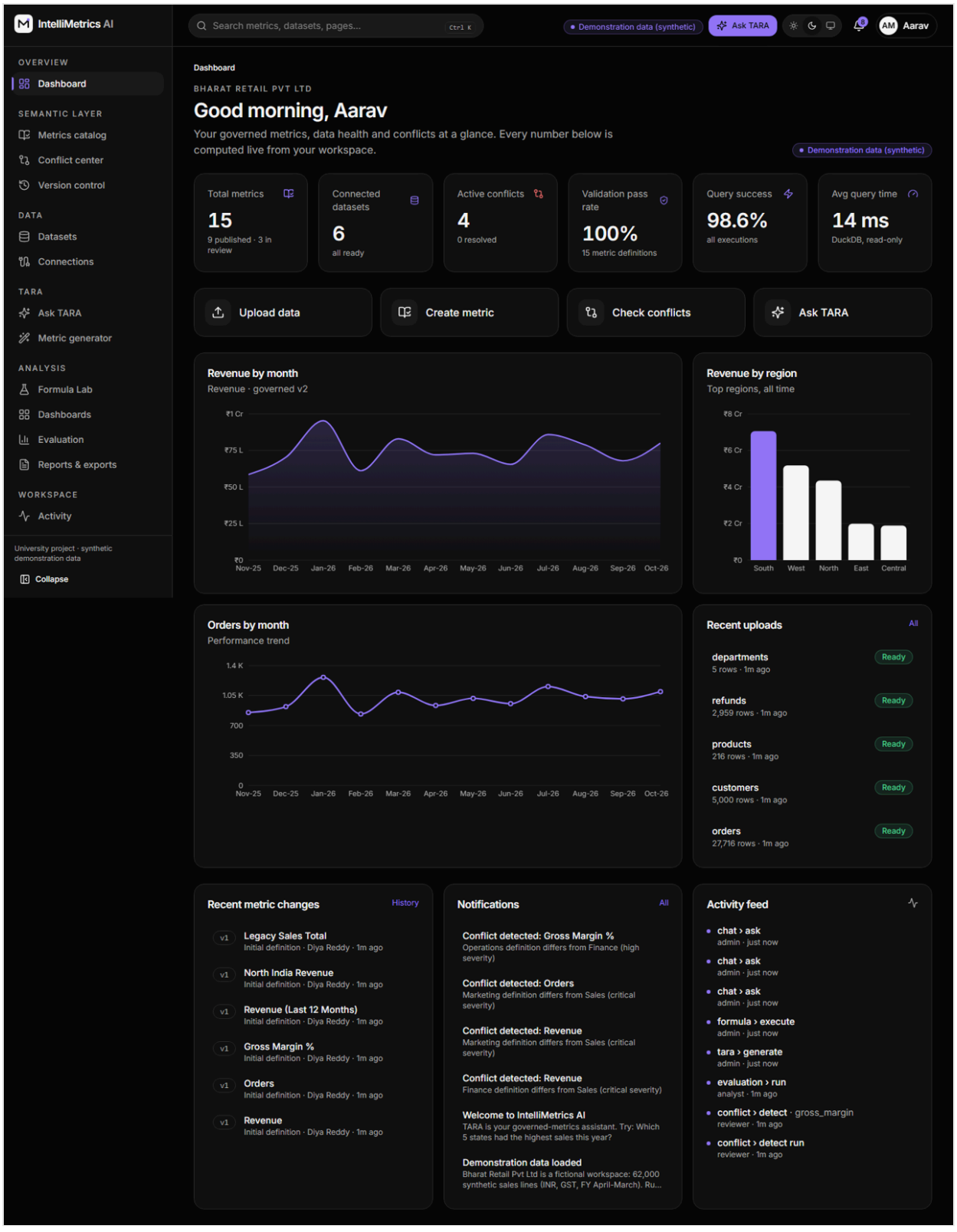
11.2 1. Main dashboard

Real-time KPIs from the API: total metrics, connected datasets, active conflicts, validation pass rate (every live definition re-validated), query success, average query time; revenue by month, revenue by region and orders by month charts drawn from the *governed* metrics; recent uploads with progress, recent metric changes,

notifications, activity feed and quick actions (upload data, create metric, check conflicts, ask TARA).



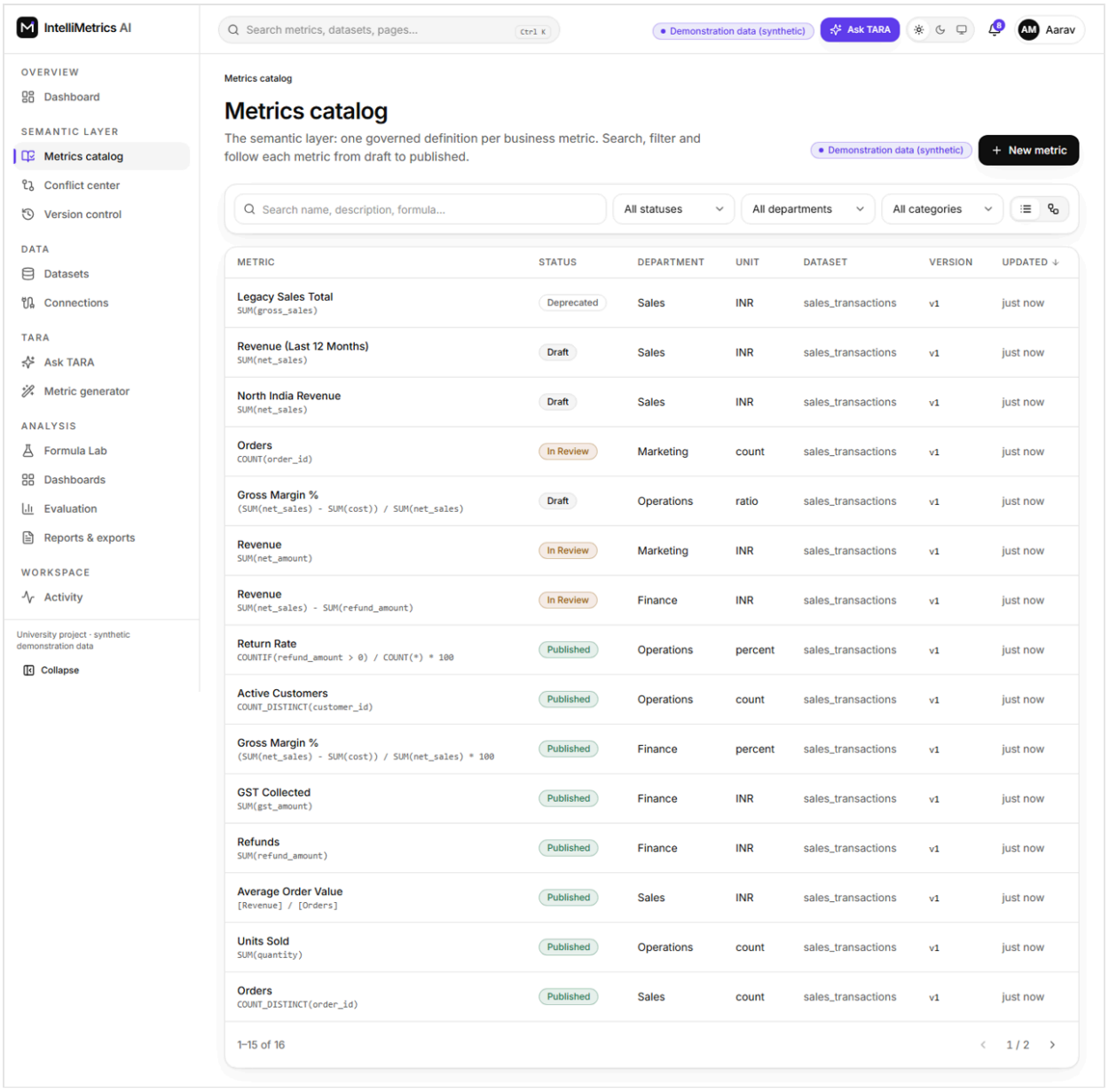
Main dashboard, light mode



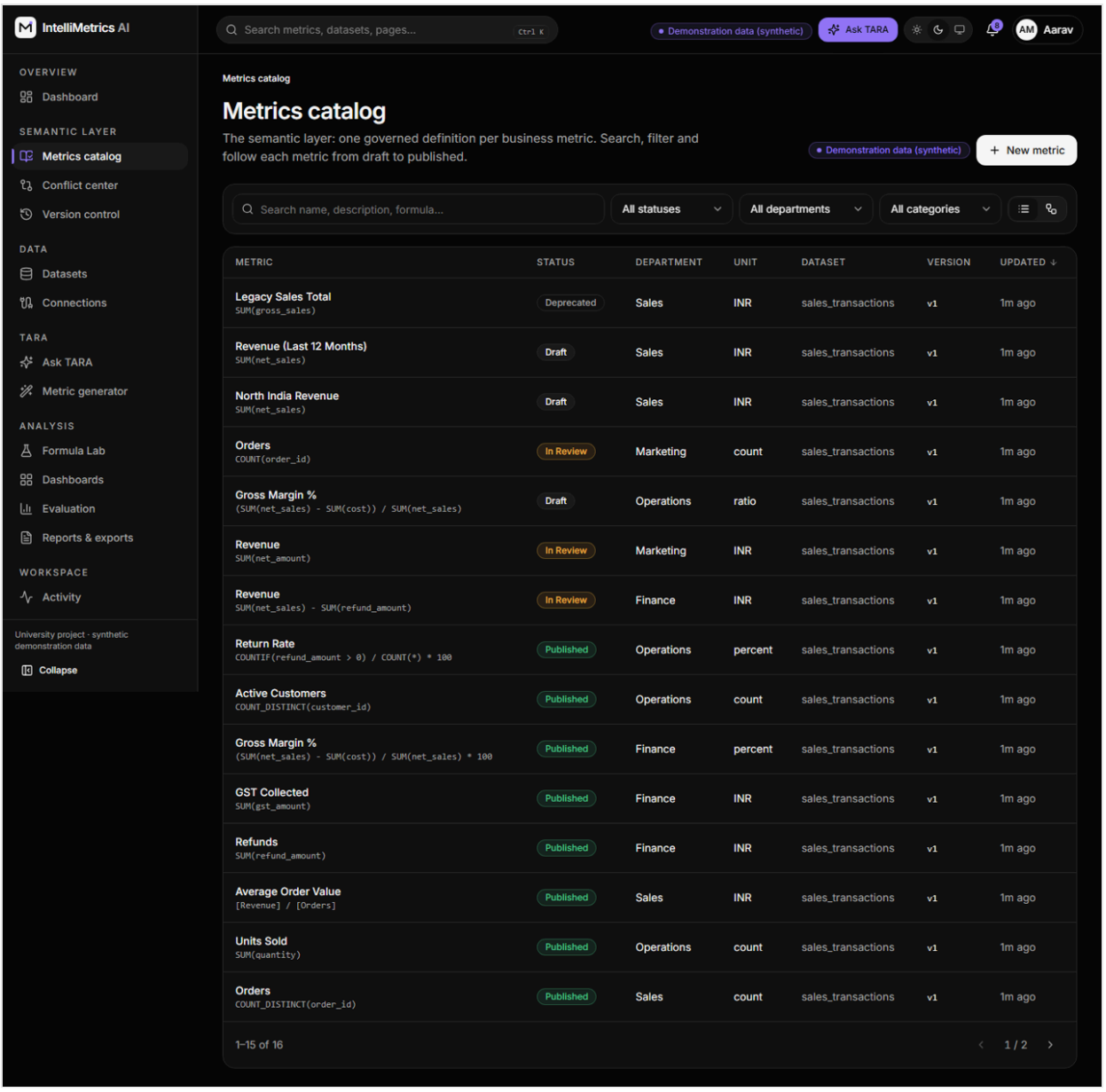
Main dashboard, dark mode

11.3 2. Semantic layer: metrics catalog and metric detail

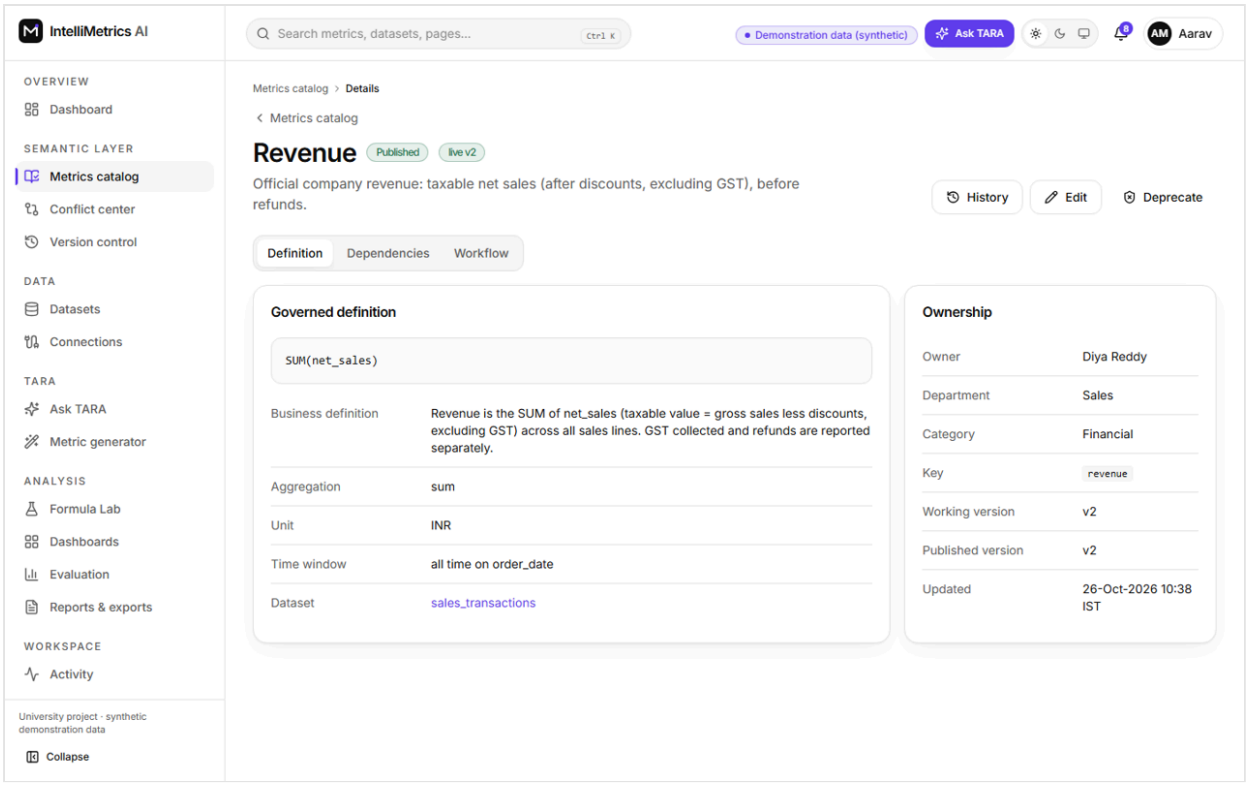
Search, filters (status, department, category), sort, pagination, list and dependency-graph views, create/edit with live formula validation, the approval workflow (submit, approve, reject with comment, publish, deprecate), dependencies and dependents, other definitions of the same key, and the workflow timeline.



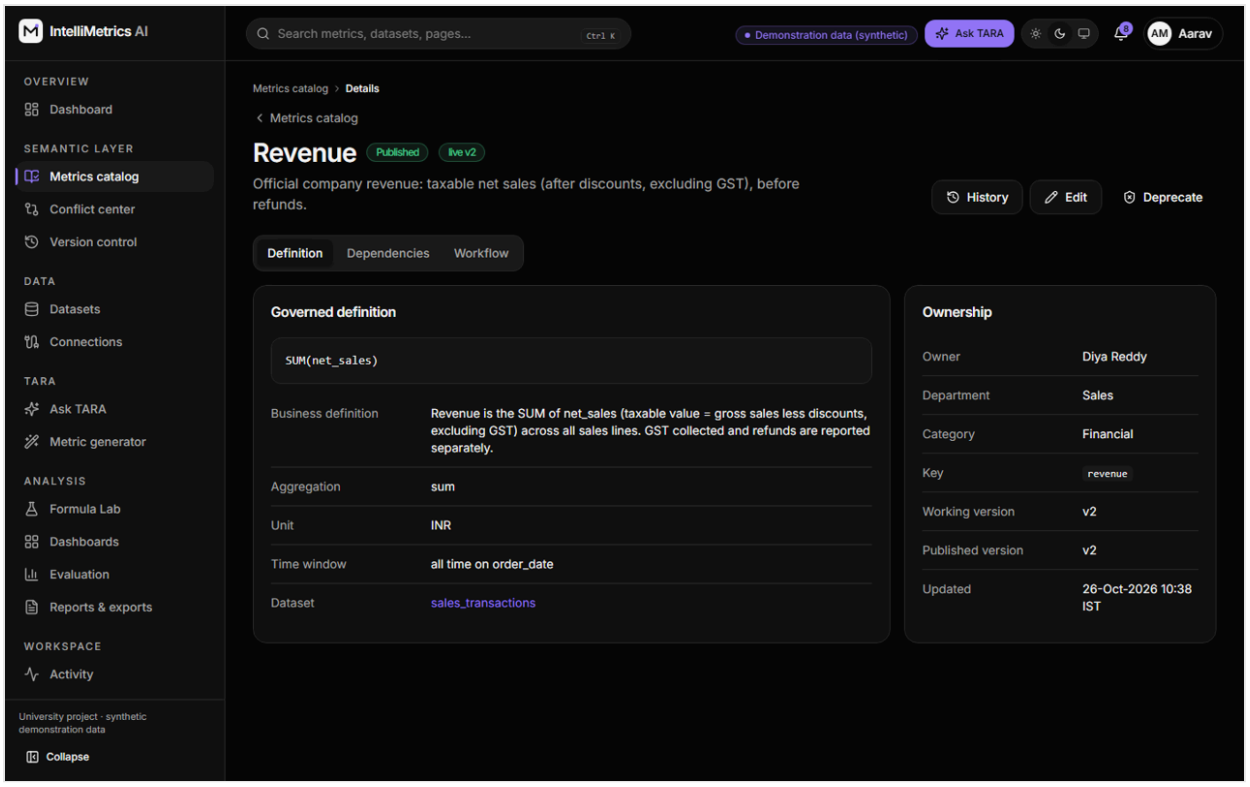
Metrics catalog, light mode



Metrics catalog, dark mode



Metric detail (Revenue), light mode

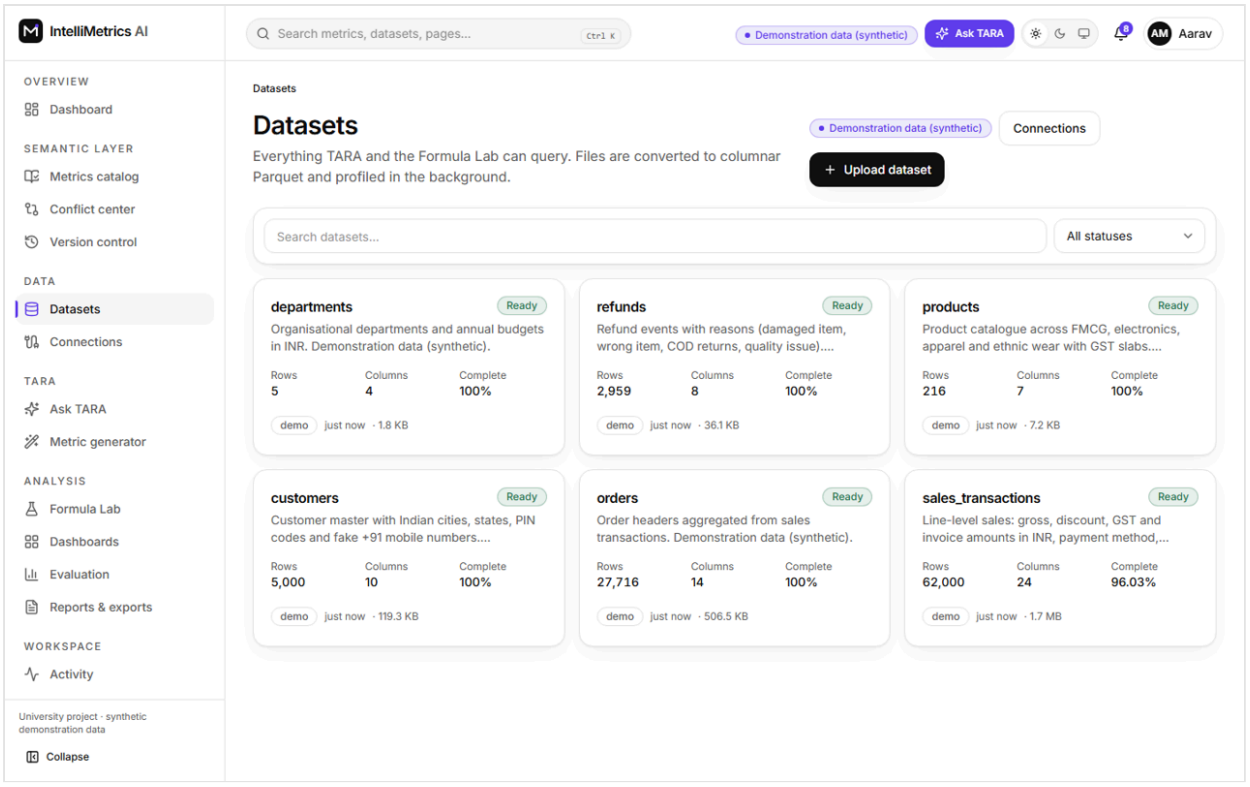


Metric detail (Revenue), dark mode

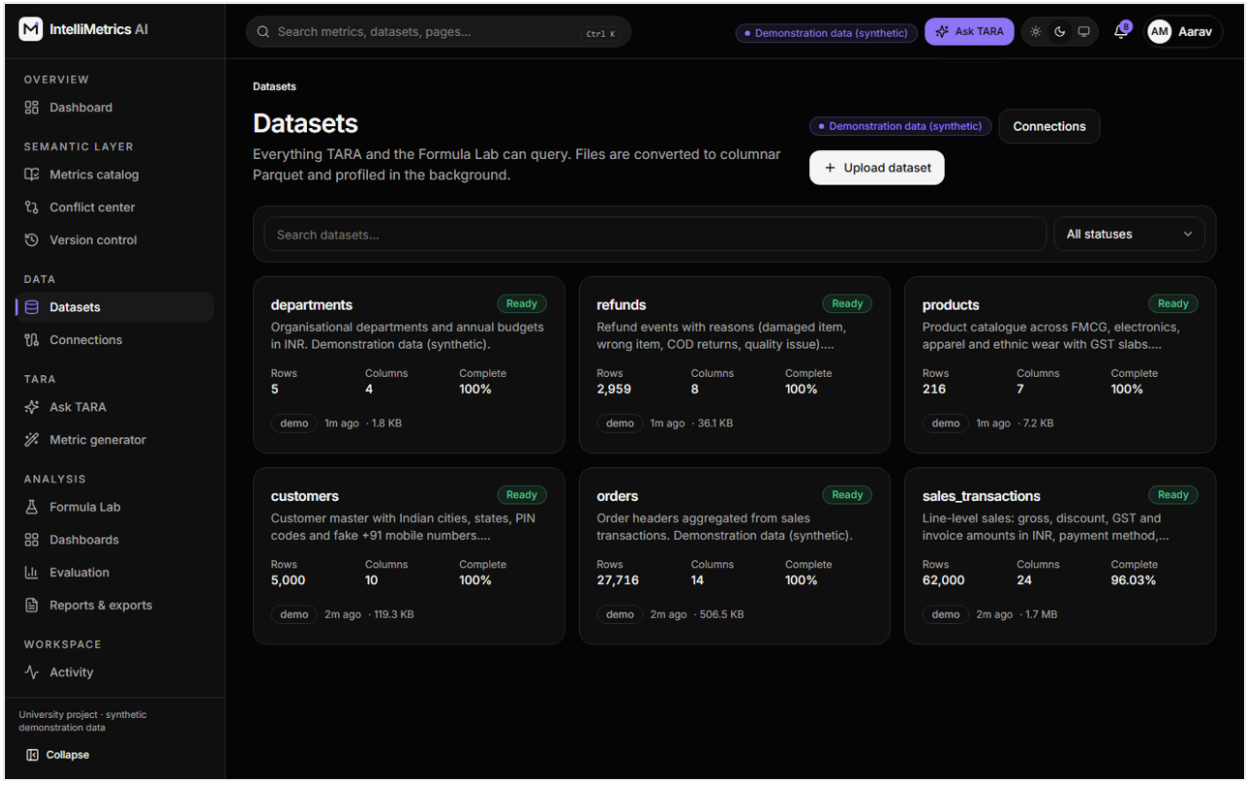
11.4 3. Datasets: catalog, upload, connections, detail

Drag-and-drop upload with background processing and a progress bar; PostgreSQL/MySQL connections with encrypted credentials and read-only imports; dataset detail with sortable preview, schema, column profiling (nulls, distincts, min/max/mean, top values, duplicate rows) and processing history with error

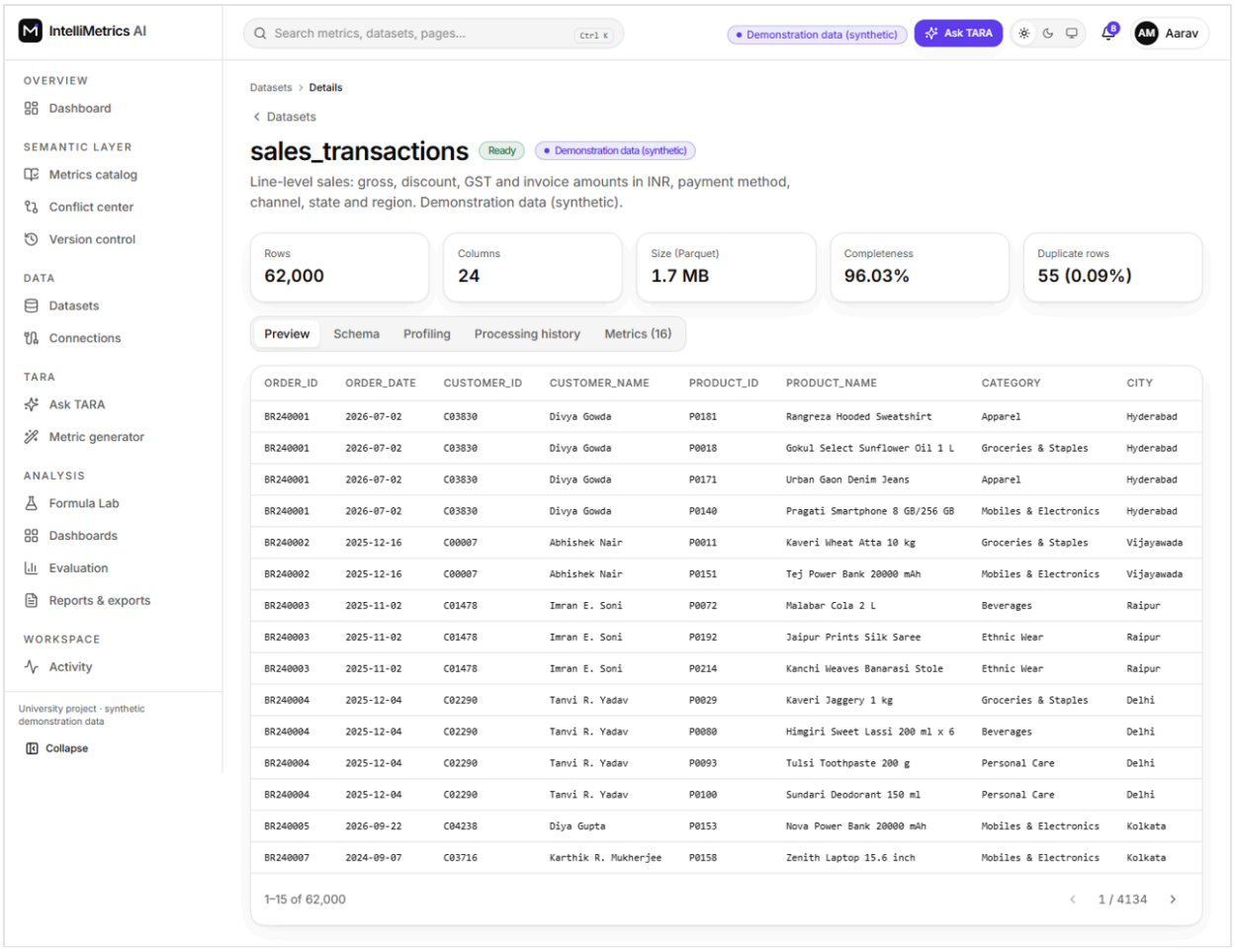
reports.



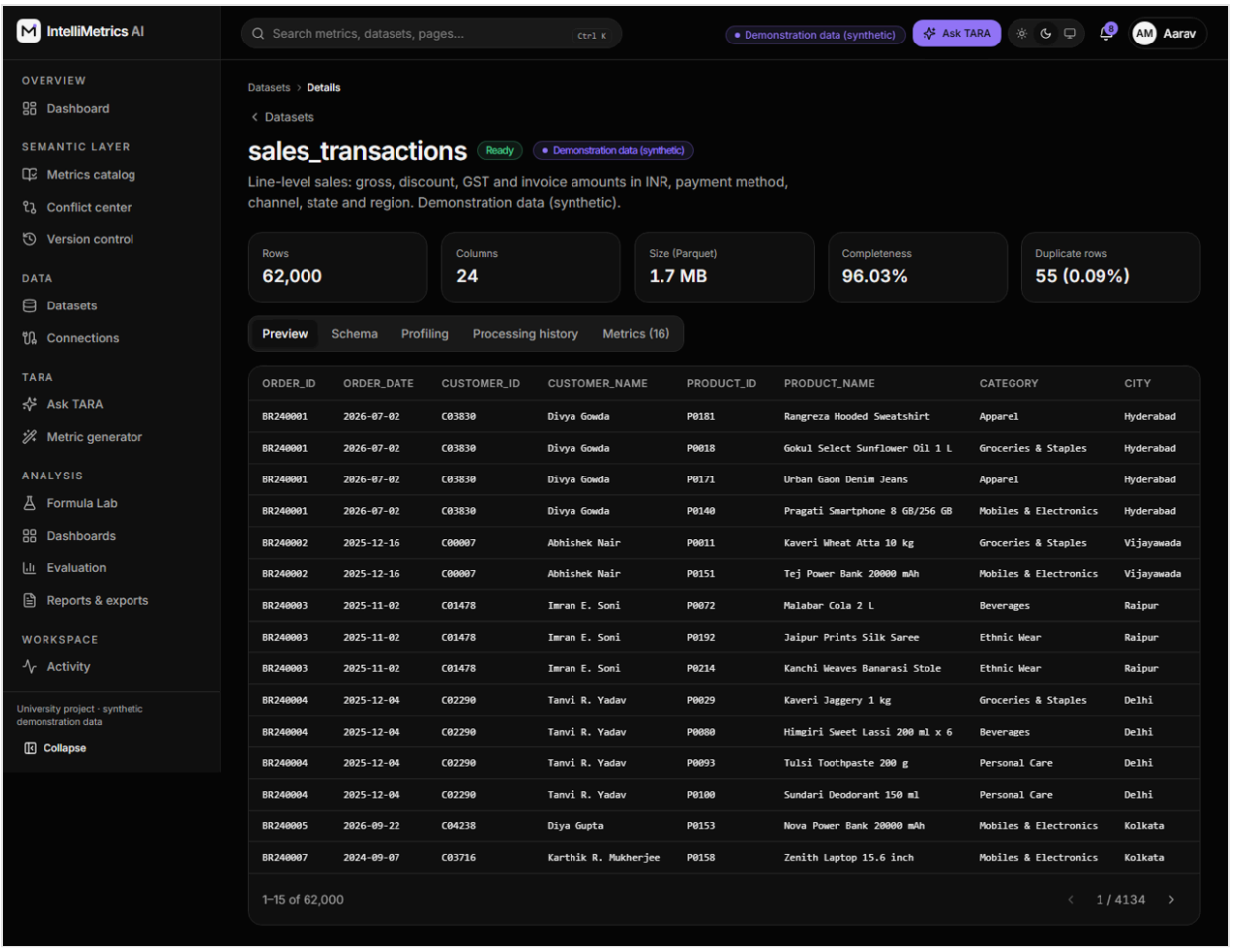
Datasets catalog, light mode



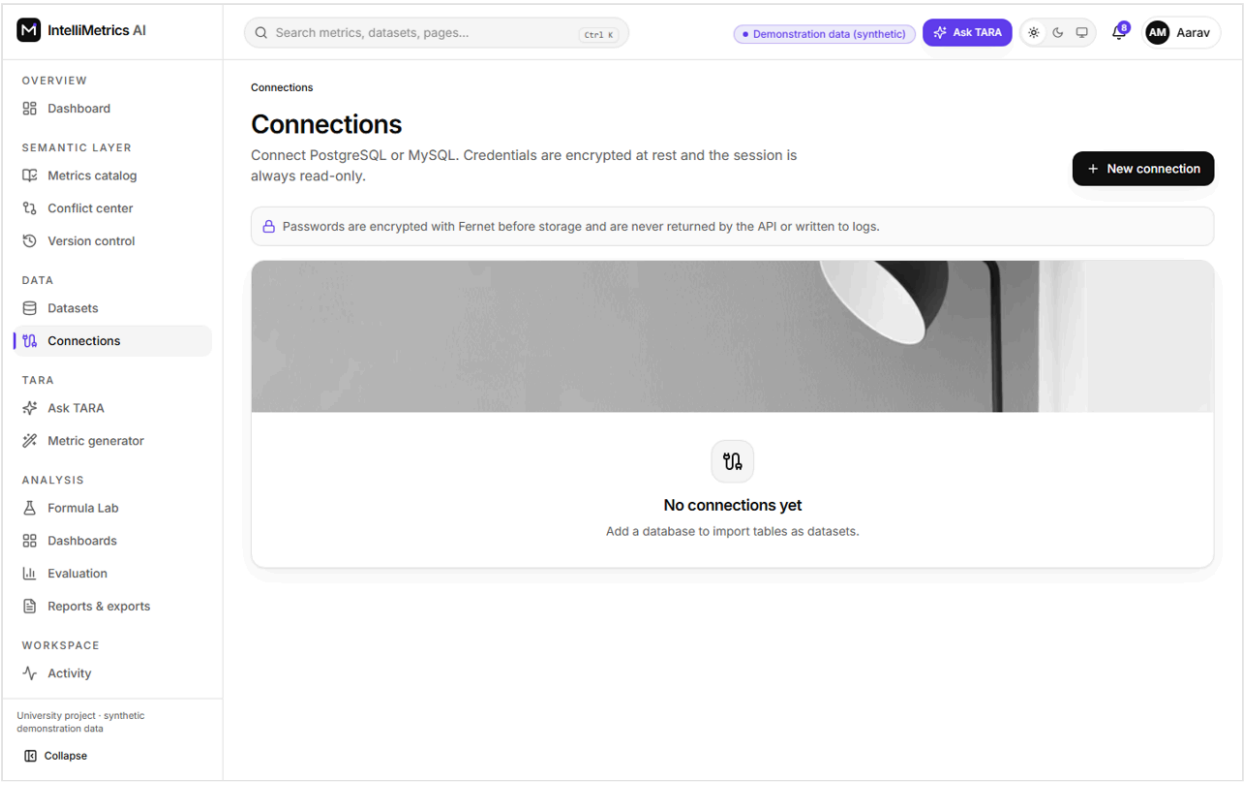
Datasets catalog, dark mode



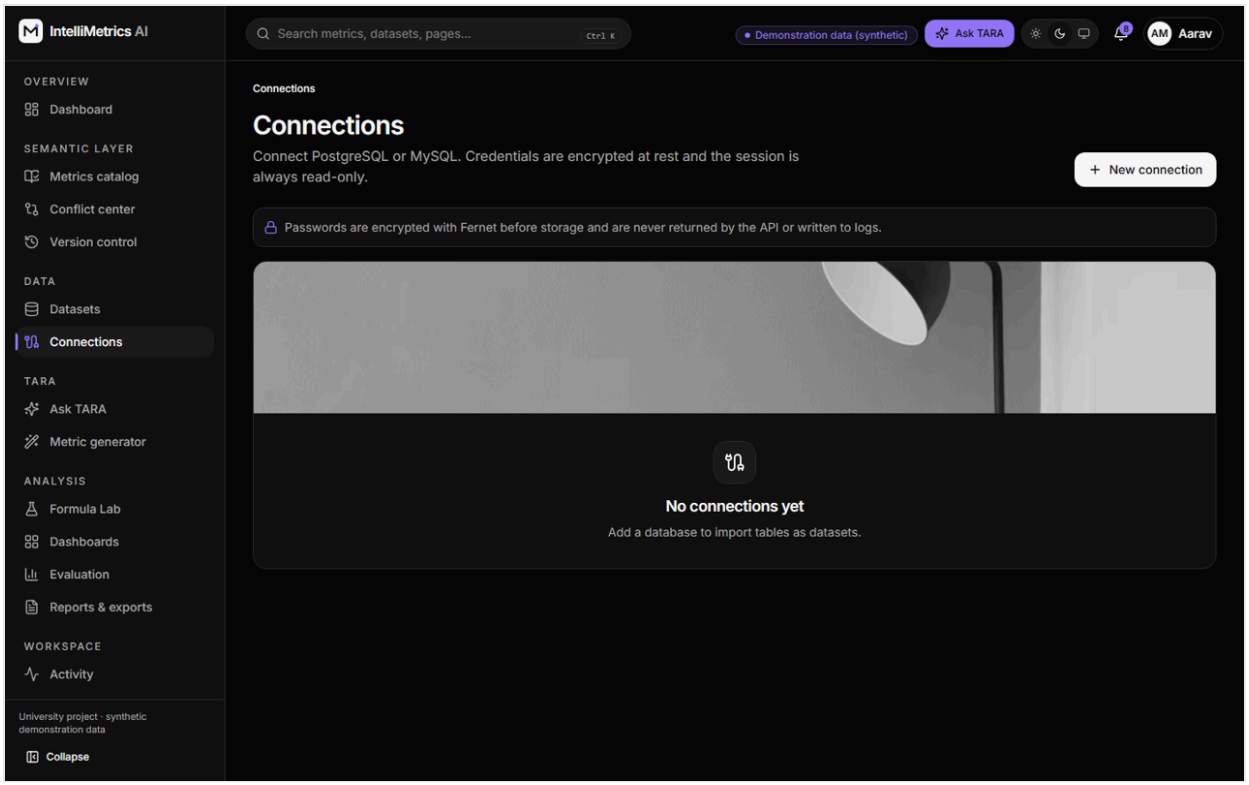
Dataset detail (preview), light mode



Dataset detail (preview), dark mode



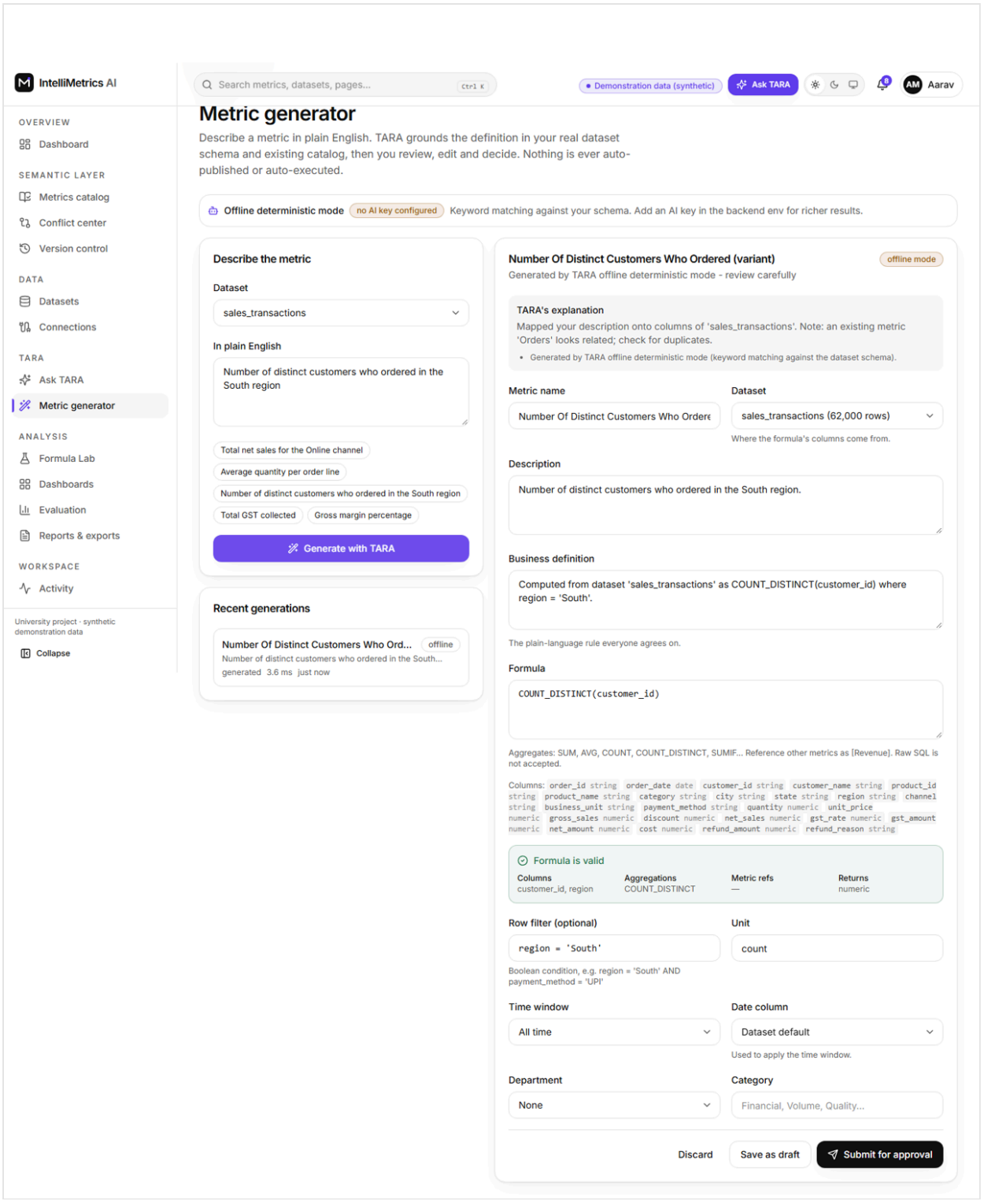
Database connections, light mode



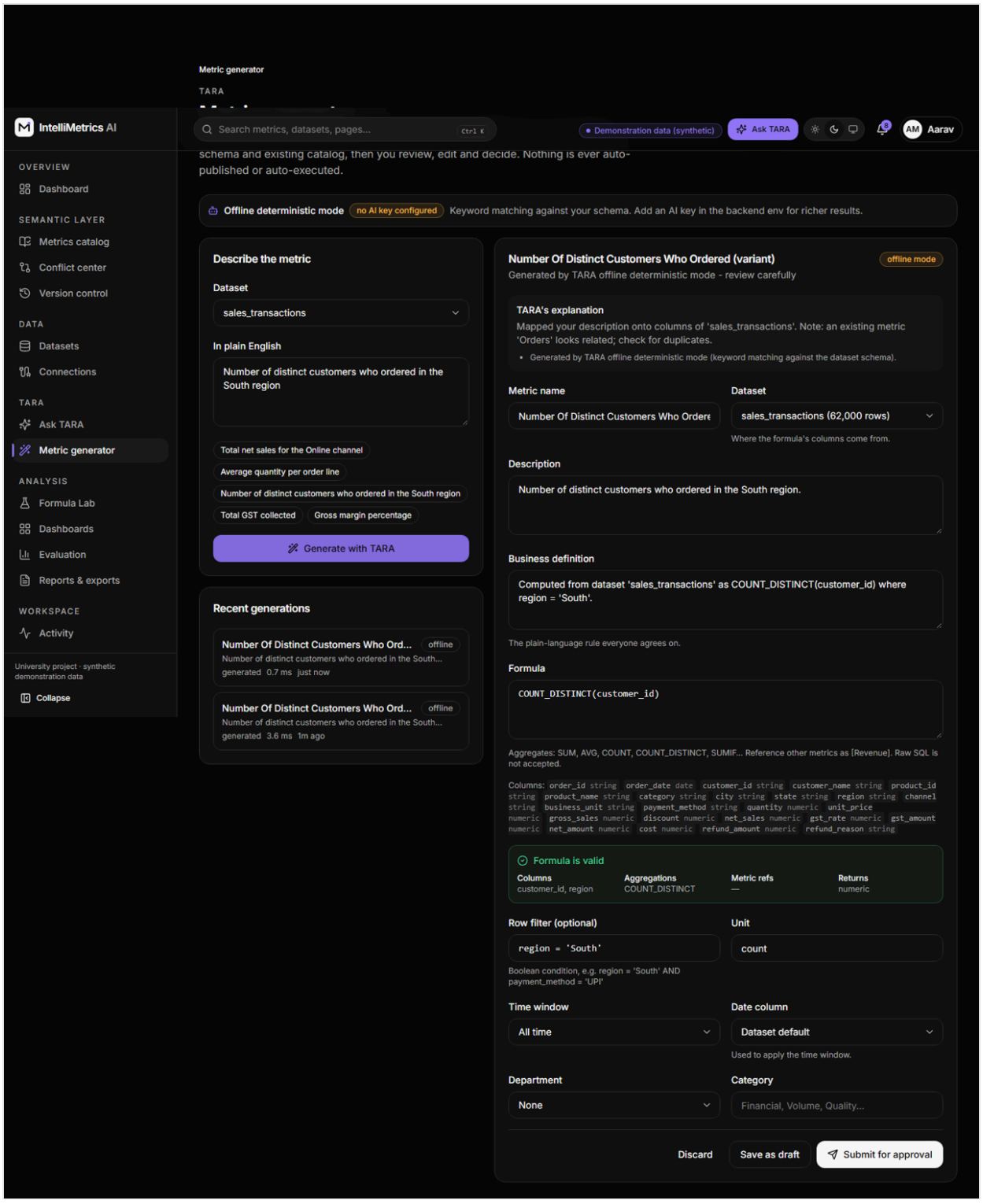
Database connections, dark mode

11.5 4. TARA metric generator

The user describes a metric in English and chooses a dataset. TARA returns an editable, validated suggestion with explanation and assumptions, labelled as AI or offline deterministic. The user may *Save as draft* or *Submit for approval*; nothing is auto-published or auto-executed. Failures, timeouts and malformed output degrade gracefully to the labelled offline mode.



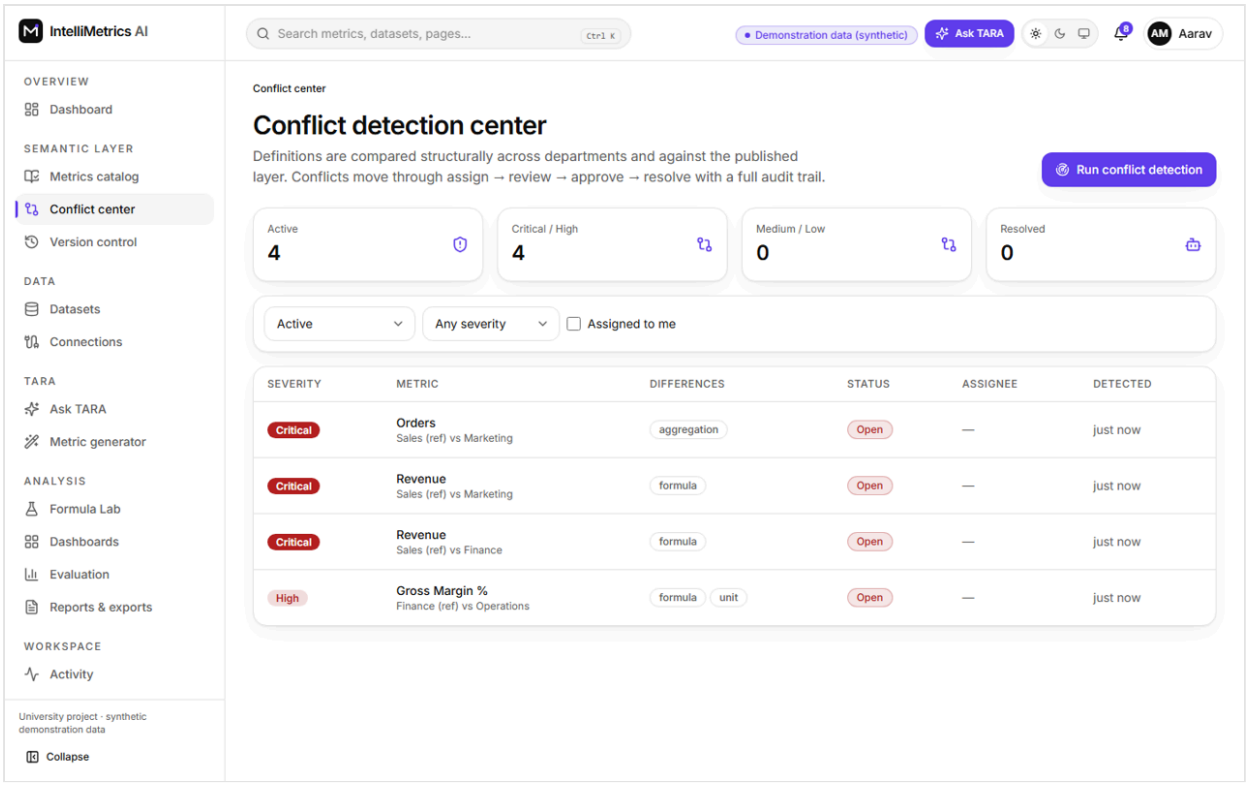
TARA metric generator (offline deterministic mode, editable suggestion), light mode



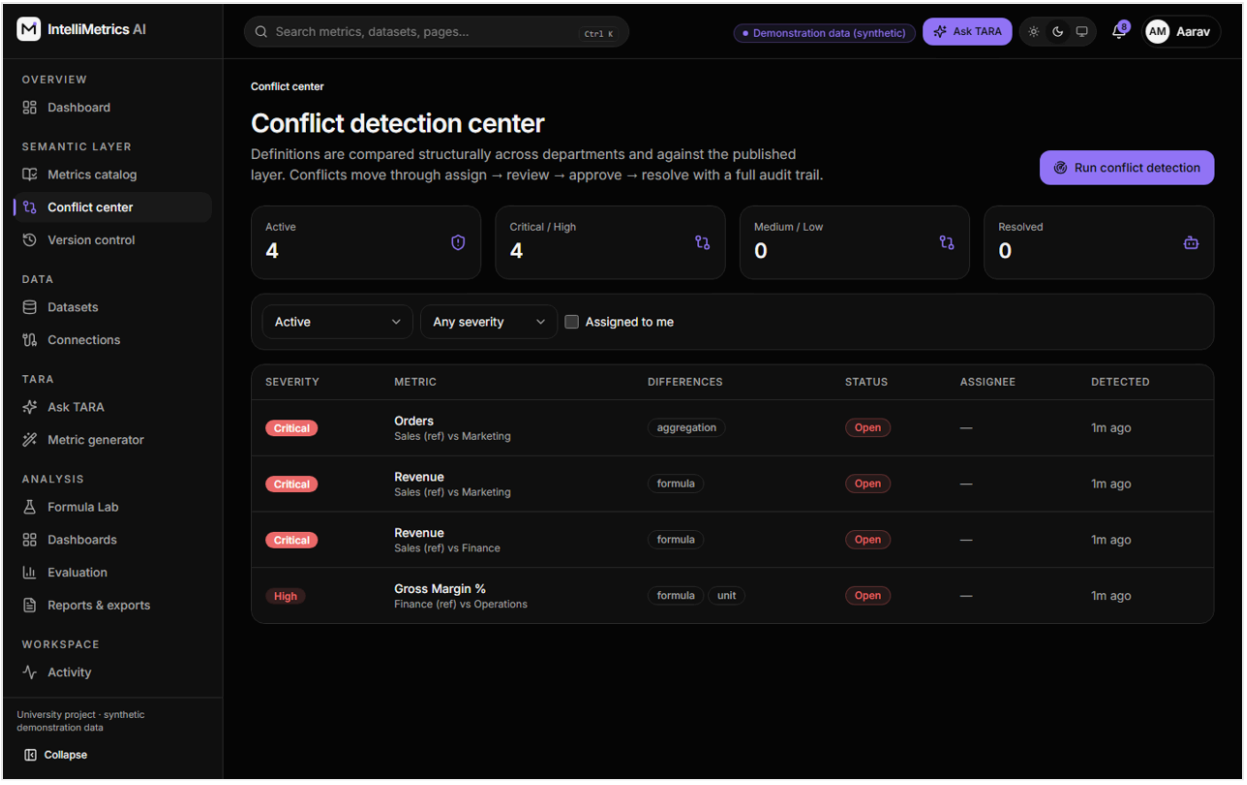
TARA metric generator (offline deterministic mode, editable suggestion), dark mode

11.6 5. Conflict detection center and conflict detail

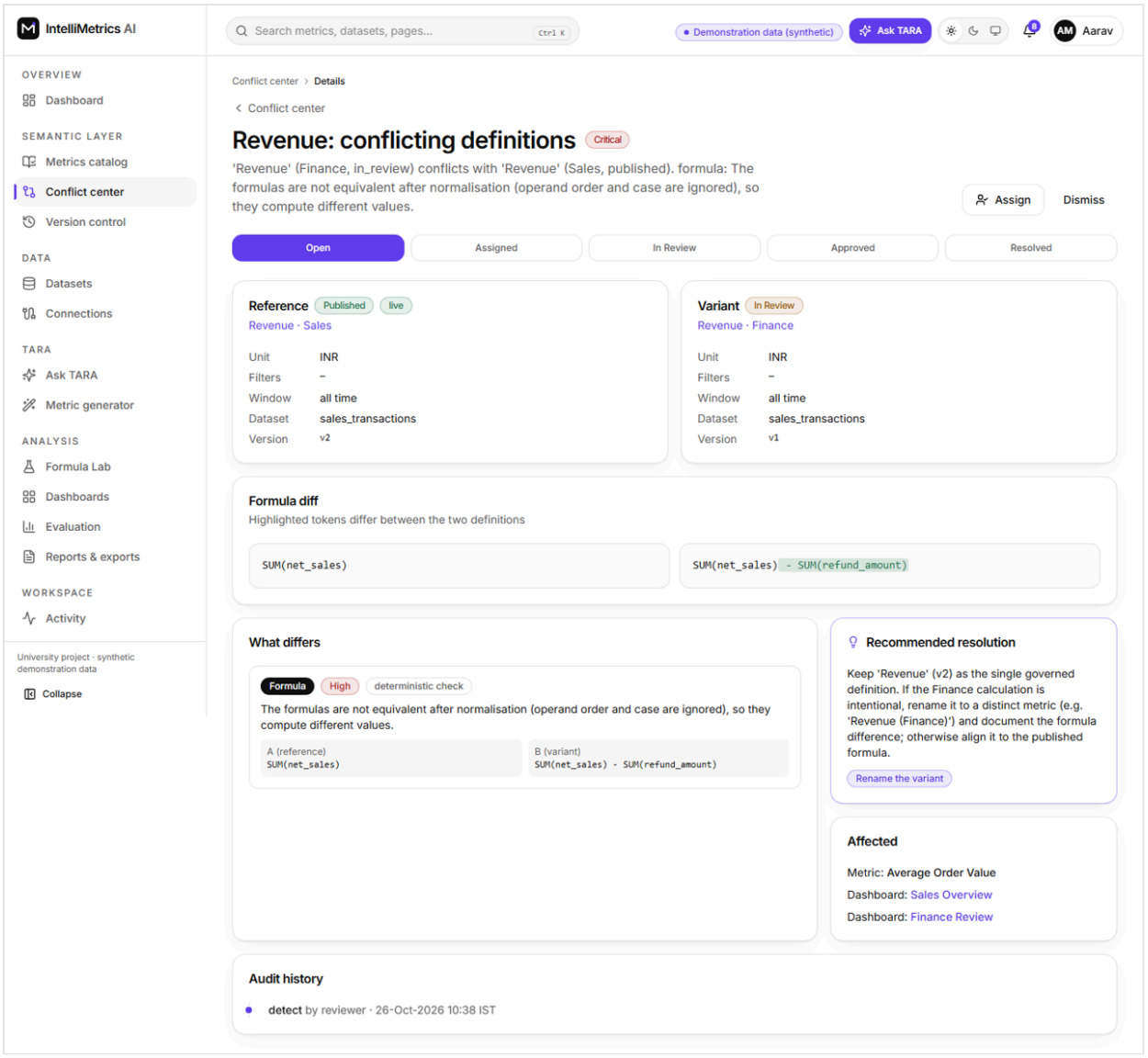
A prioritised list with severity, type, status and assignee; detail pages show reference and variant side by side with a token-level formula diff, each difference explained (deterministic check or TARA suggestion), the recommended resolution, affected objects, the strict workflow (assign, review, approve, resolve) and the audit history.



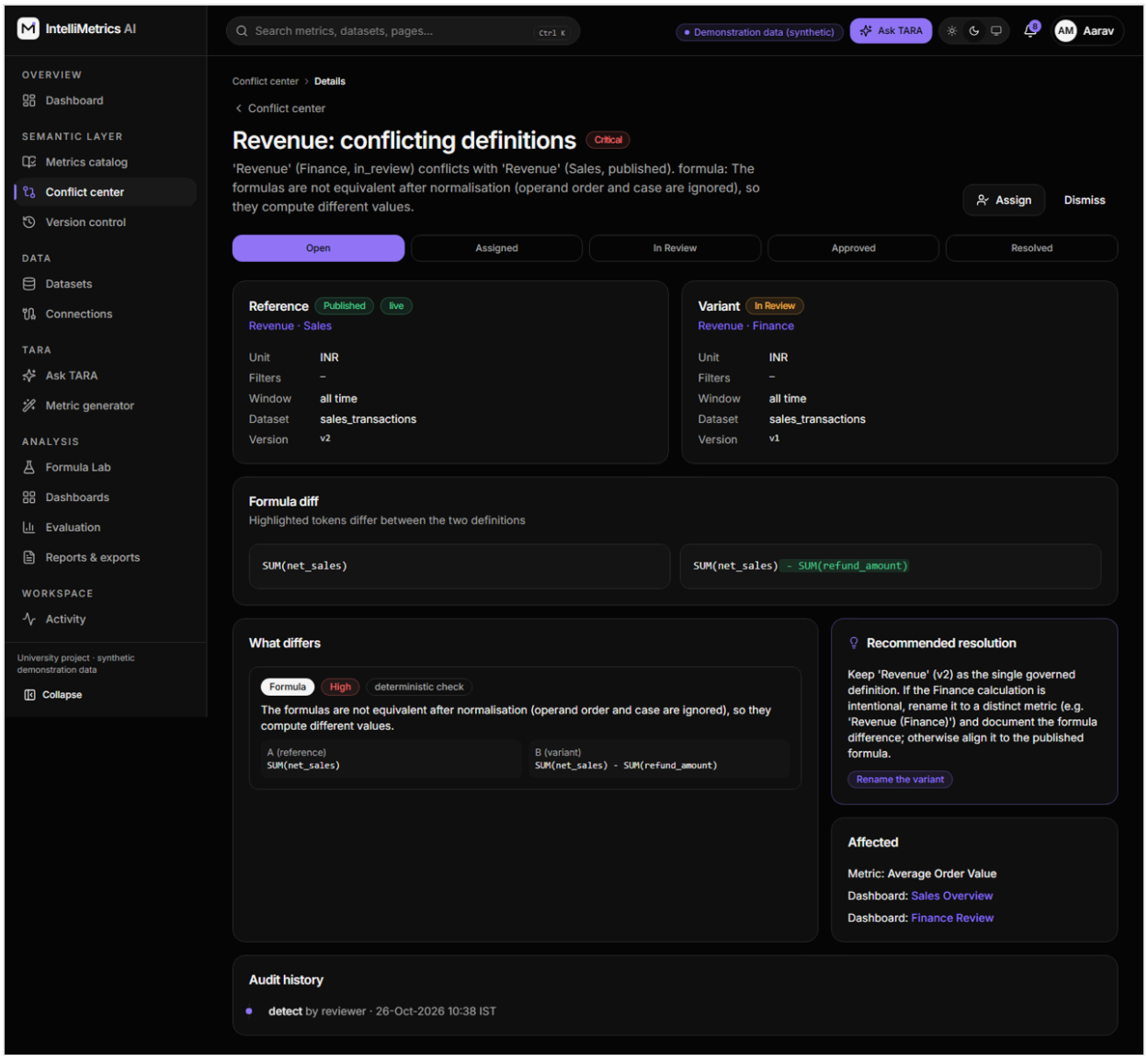
Conflict detection center, light mode



Conflict detection center, dark mode



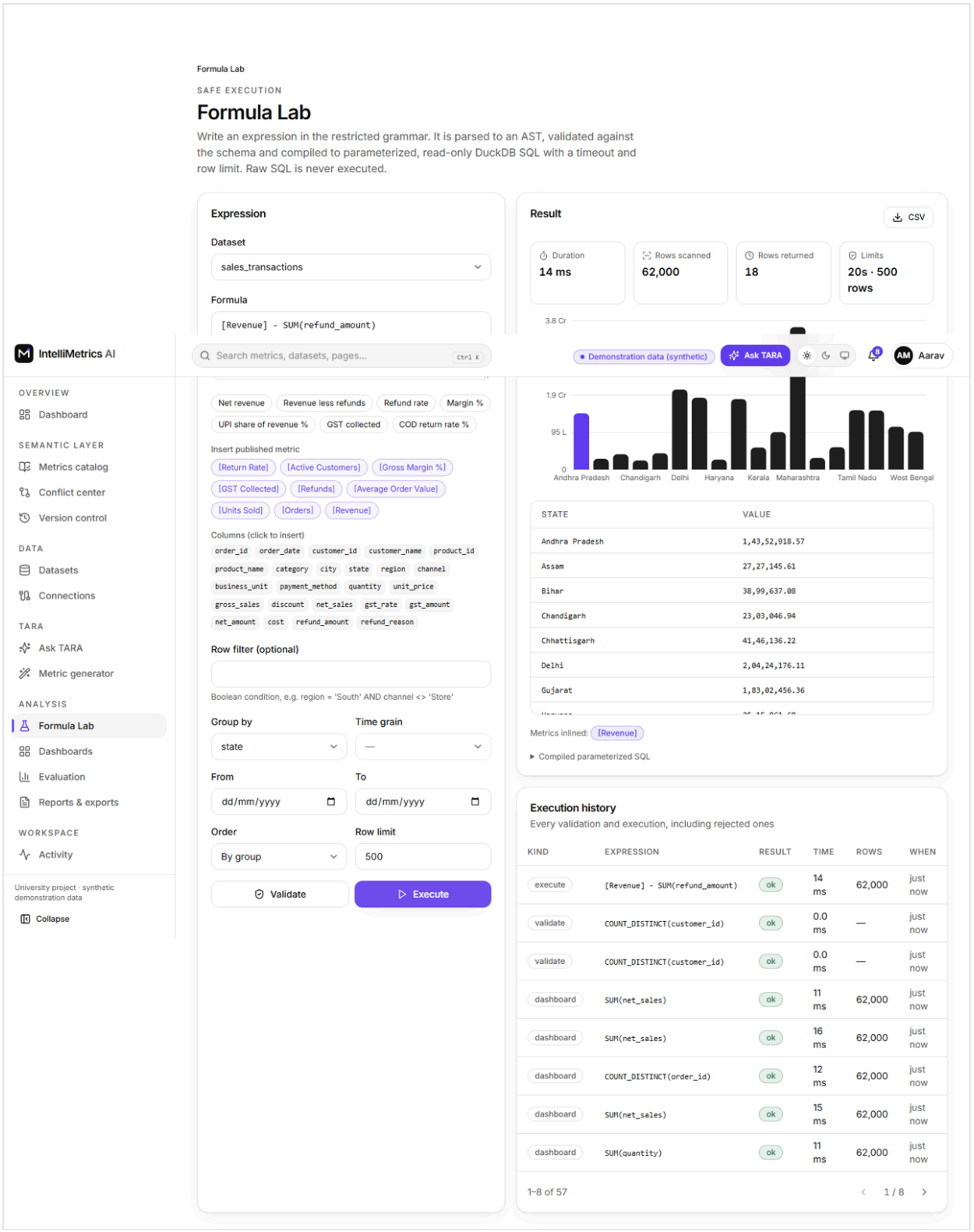
Conflict detail (side-by-side), light mode



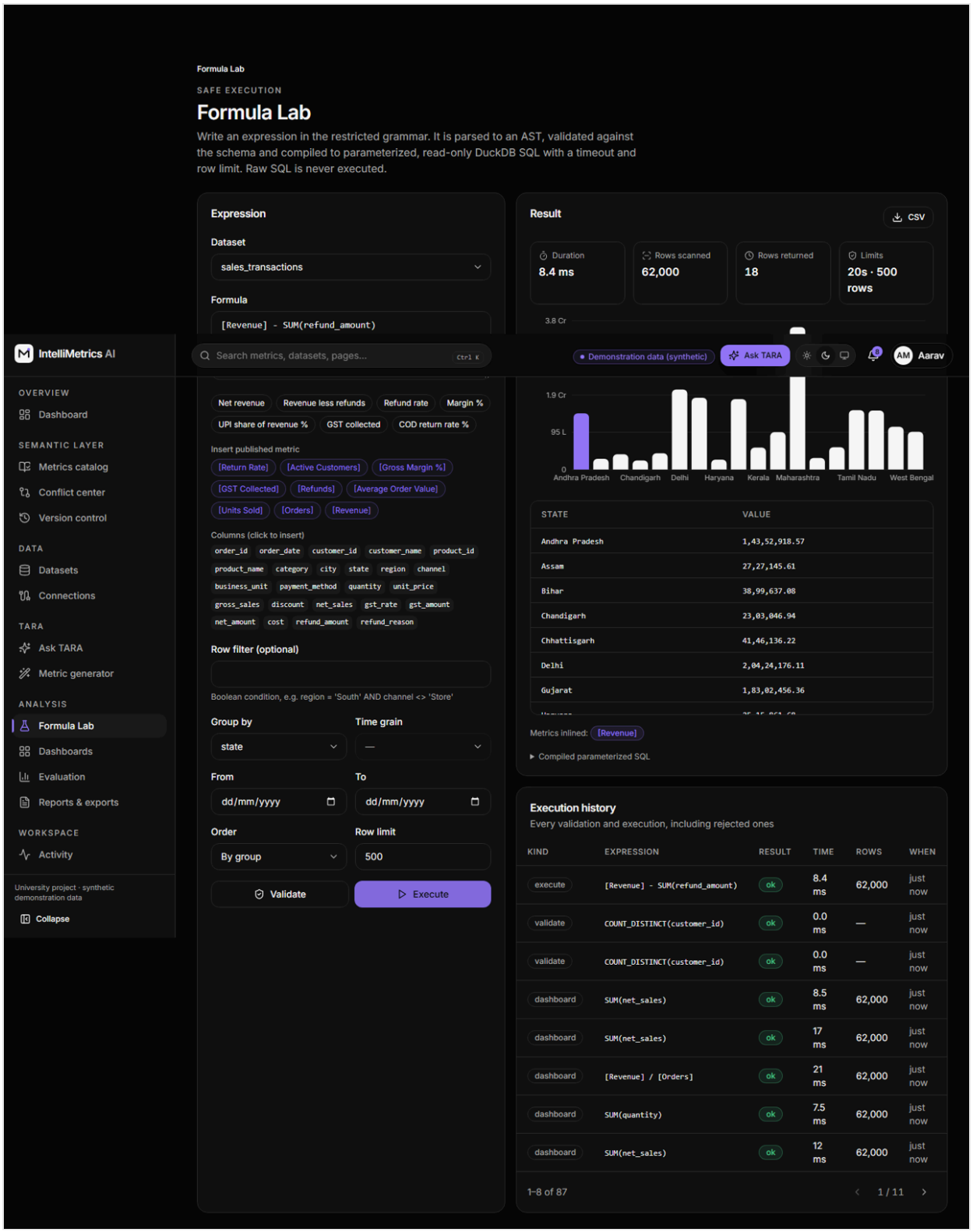
Conflict detail (side-by-side), dark mode

11.7 6. Formula Lab

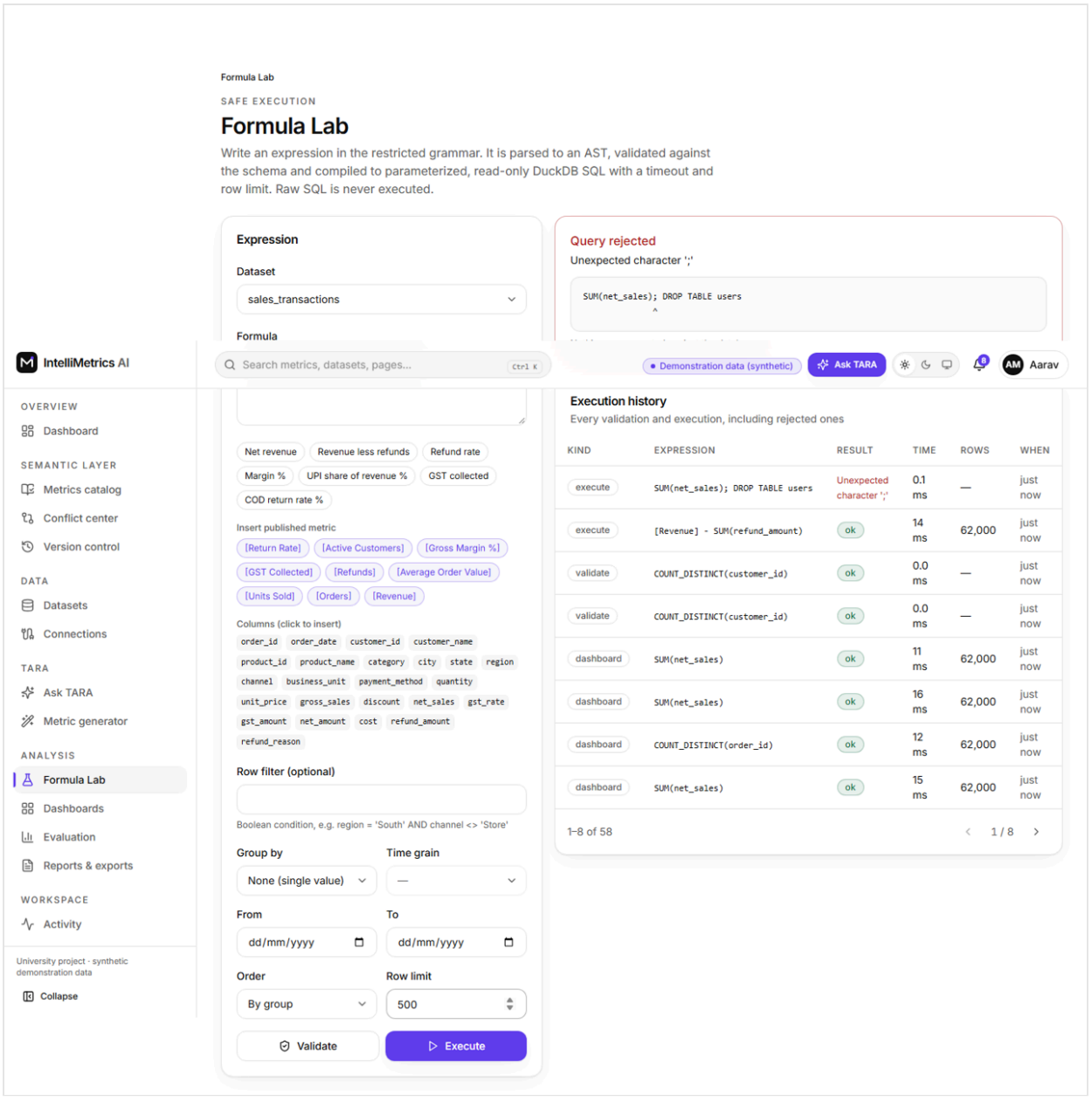
Pick a dataset, write an expression, validate (syntax, columns, types, aggregation compatibility, permissions) and execute read-only. Results show duration, rows scanned, rows returned, limits and the parameterised SQL; rejected inputs show the error position and state that nothing was executed. Execution history records every attempt.



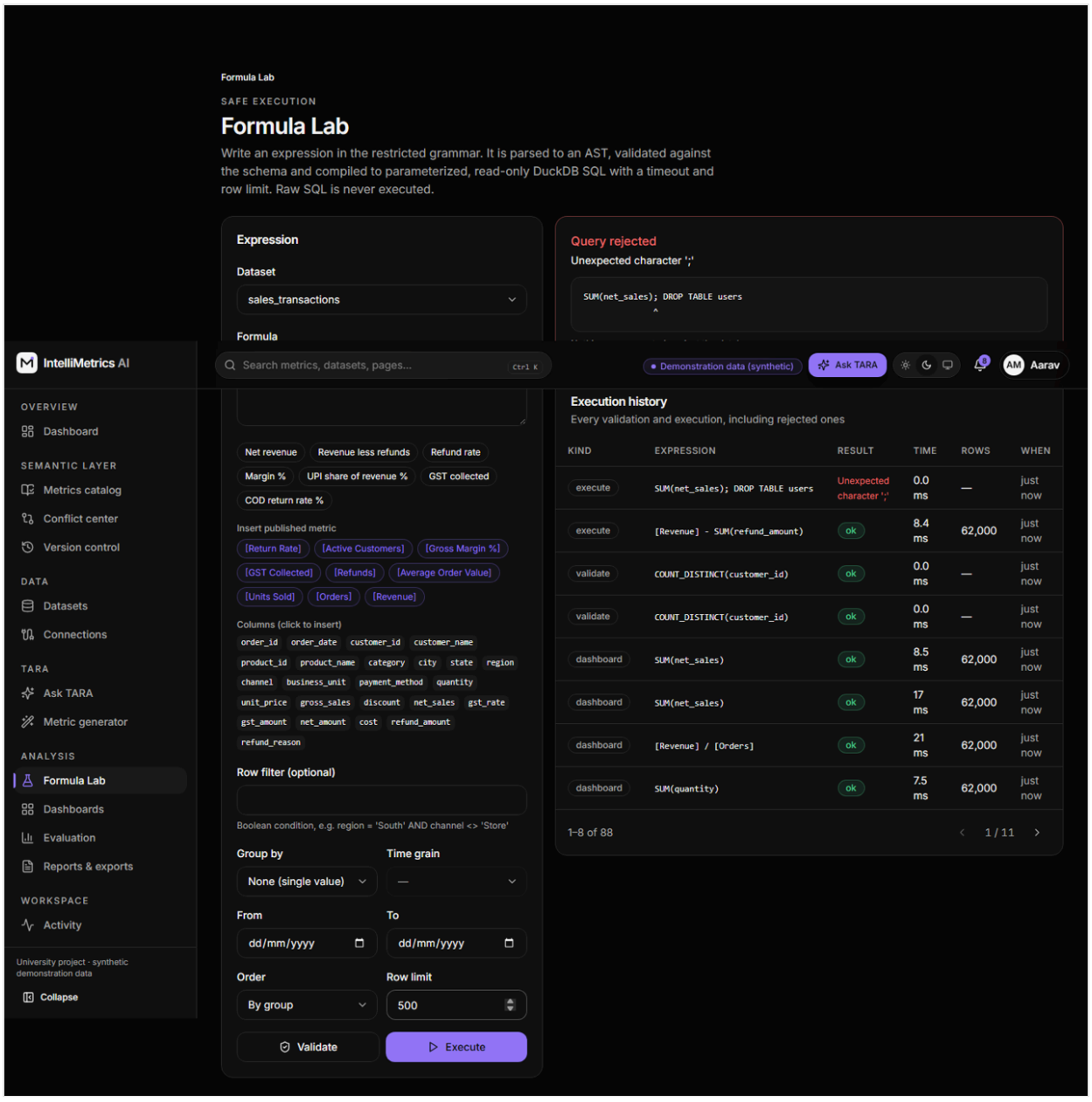
Formula Lab: validate and execute (Revenue less refunds by state), light mode



Formula Lab: validate and execute (Revenue less refunds by state), dark mode



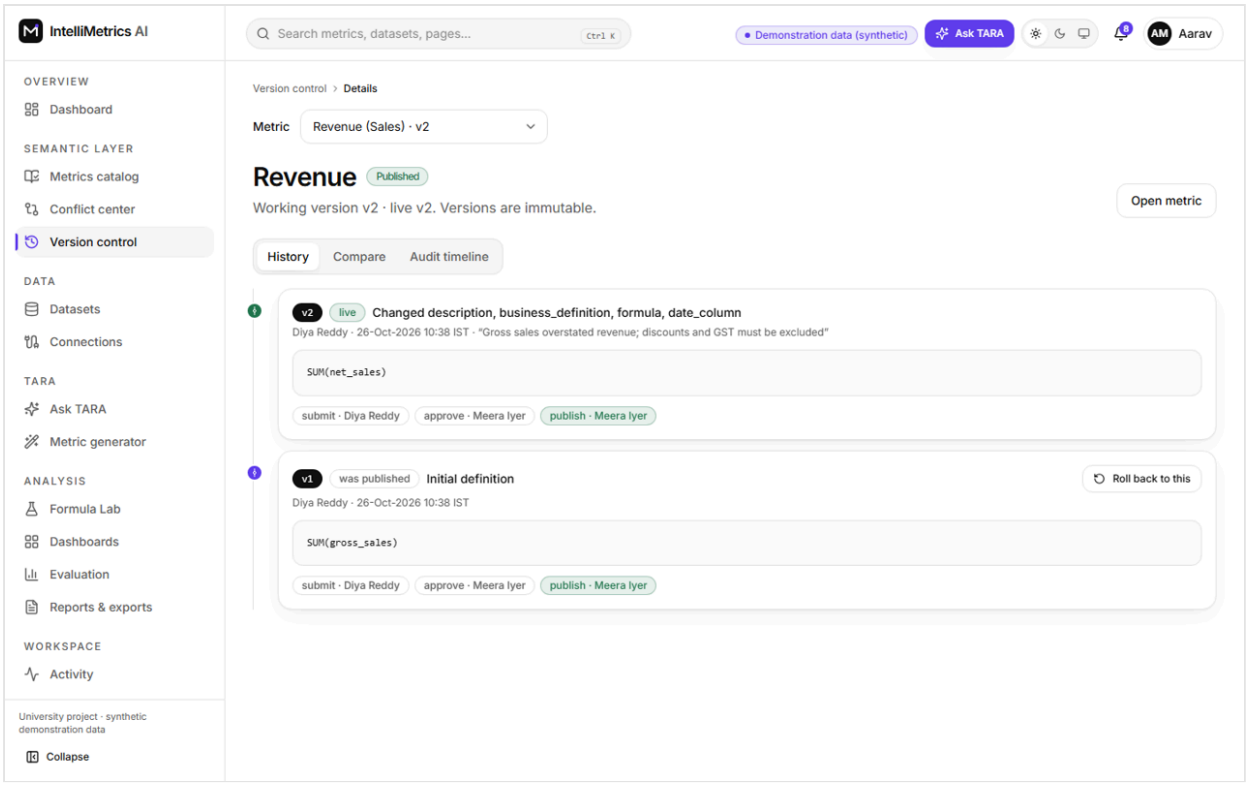
Formula Lab rejecting an injection attempt, light mode



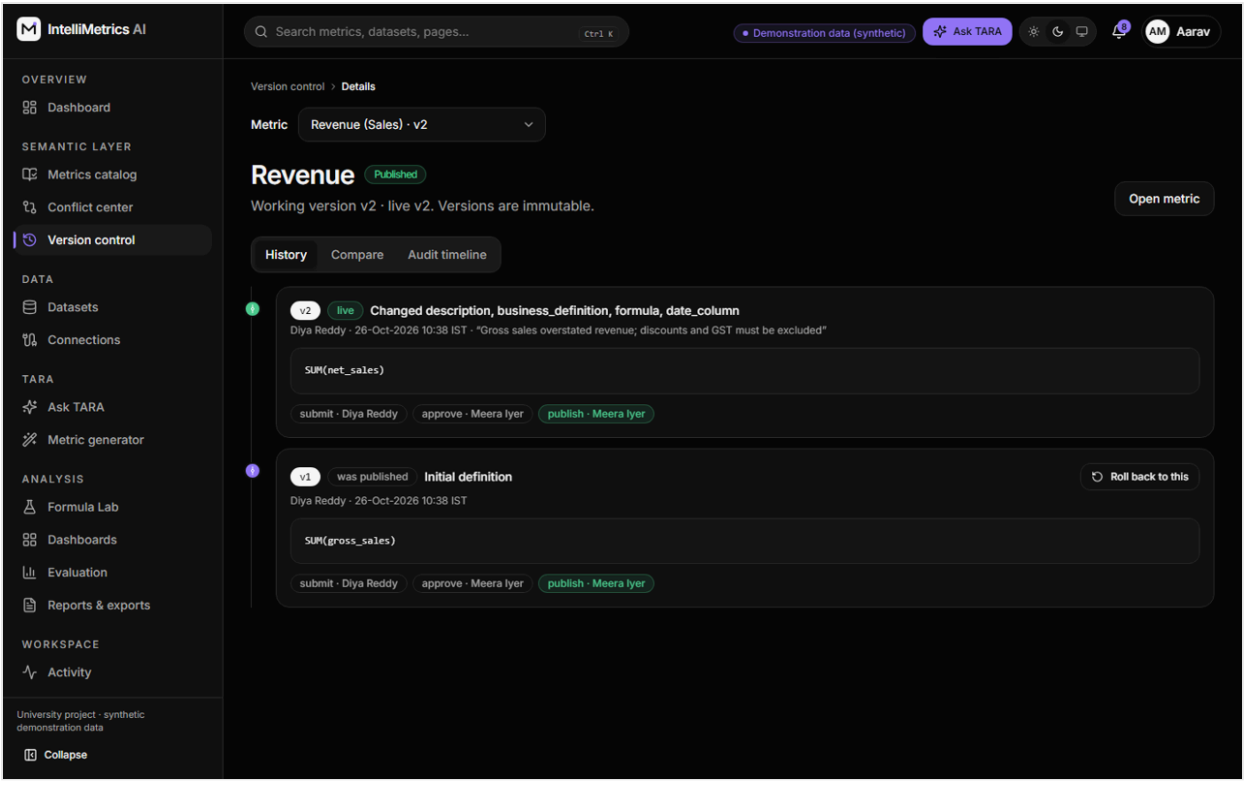
Formula Lab rejecting an injection attempt, dark mode

11.8 7. Version control

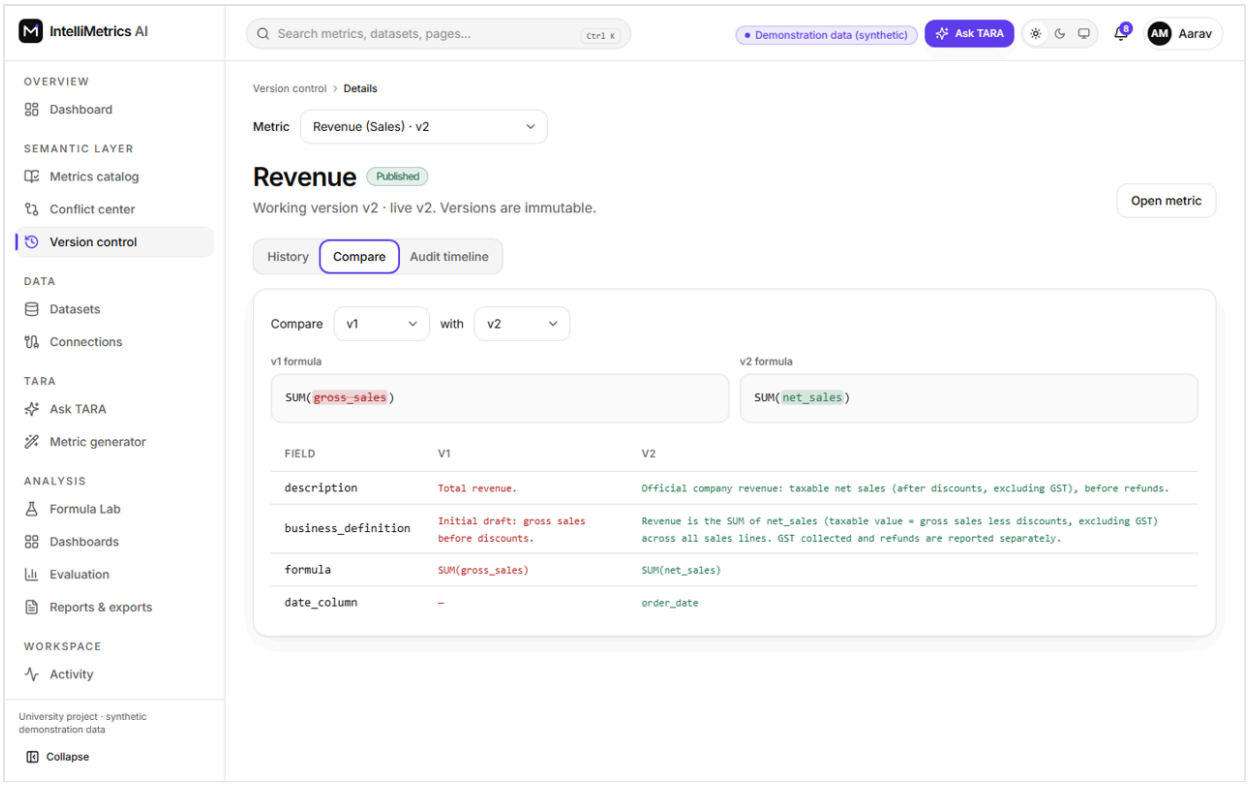
History with authors, reasons, approvals and publication markers; side-by-side compare; rollback as a new version; audit timeline.



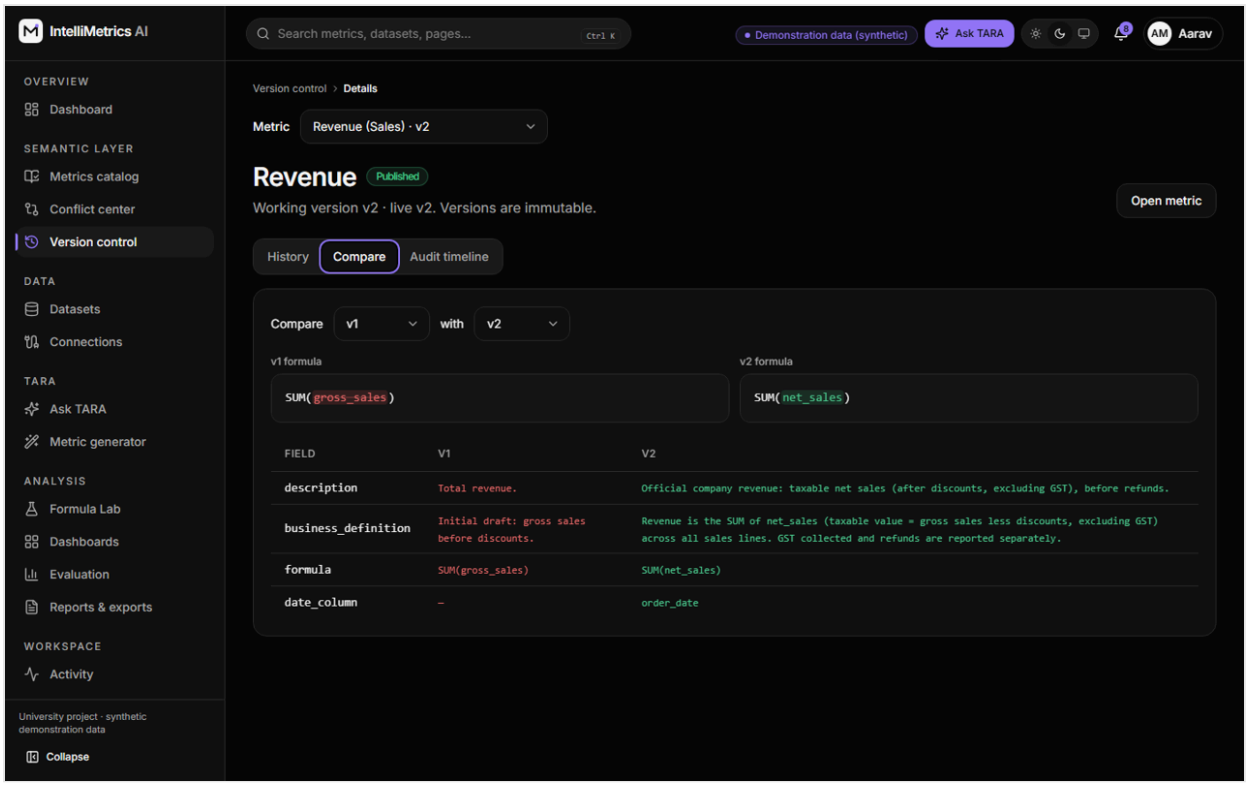
Version control (Revenue history), light mode



Version control (Revenue history), dark mode



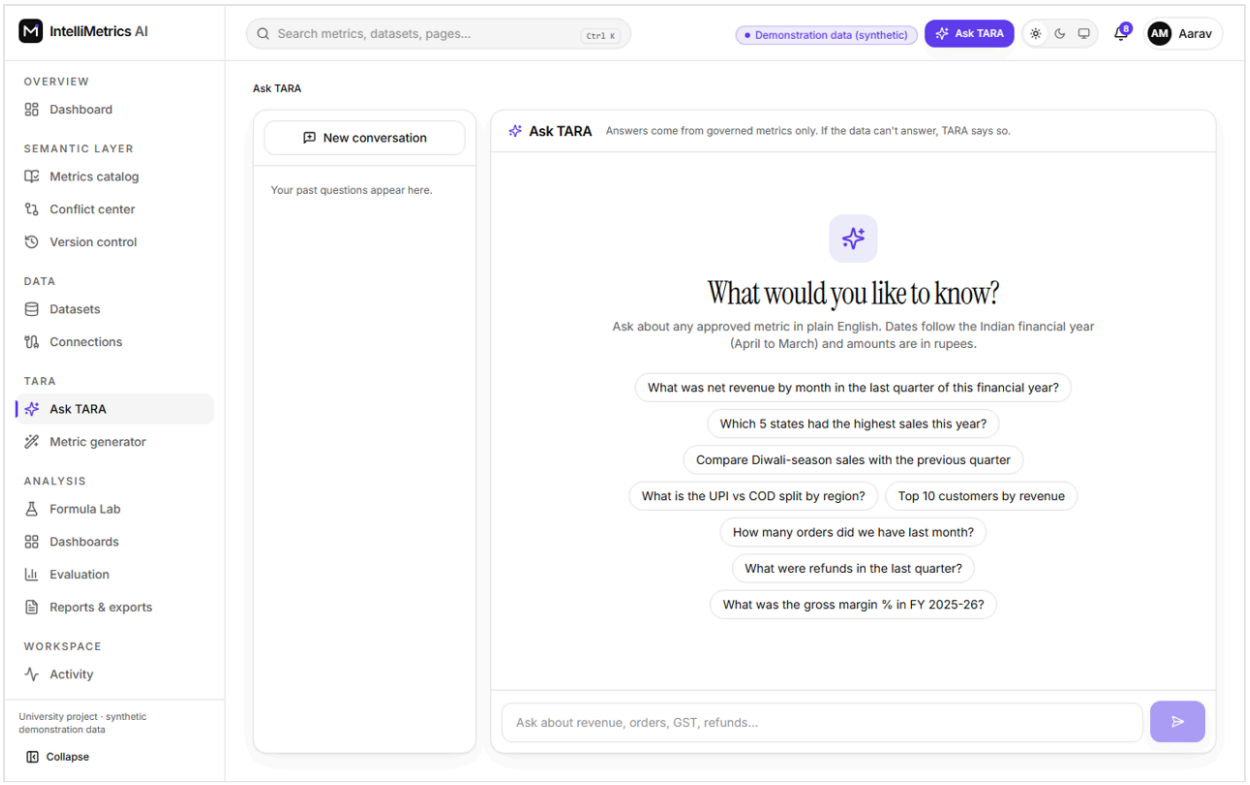
Version compare (diff view), light mode



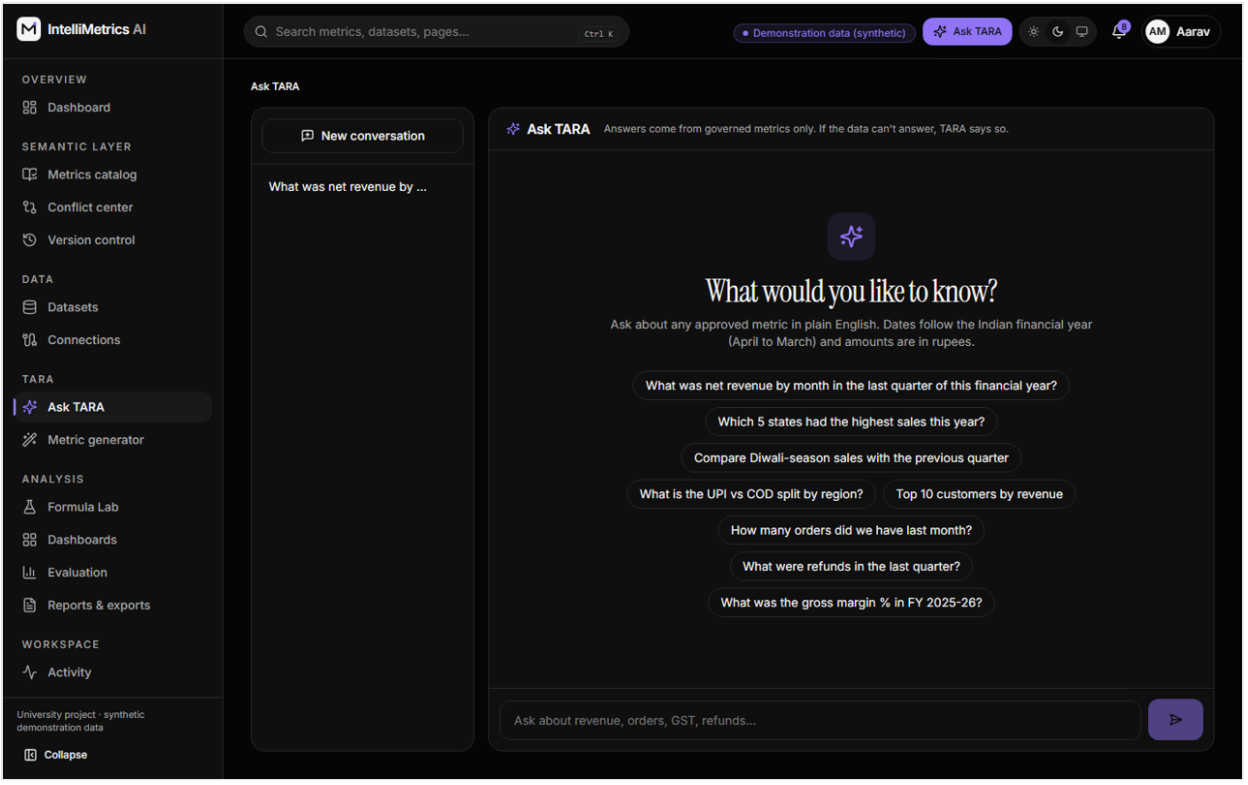
Version compare (diff view), dark mode

11.9 8. Ask TARA

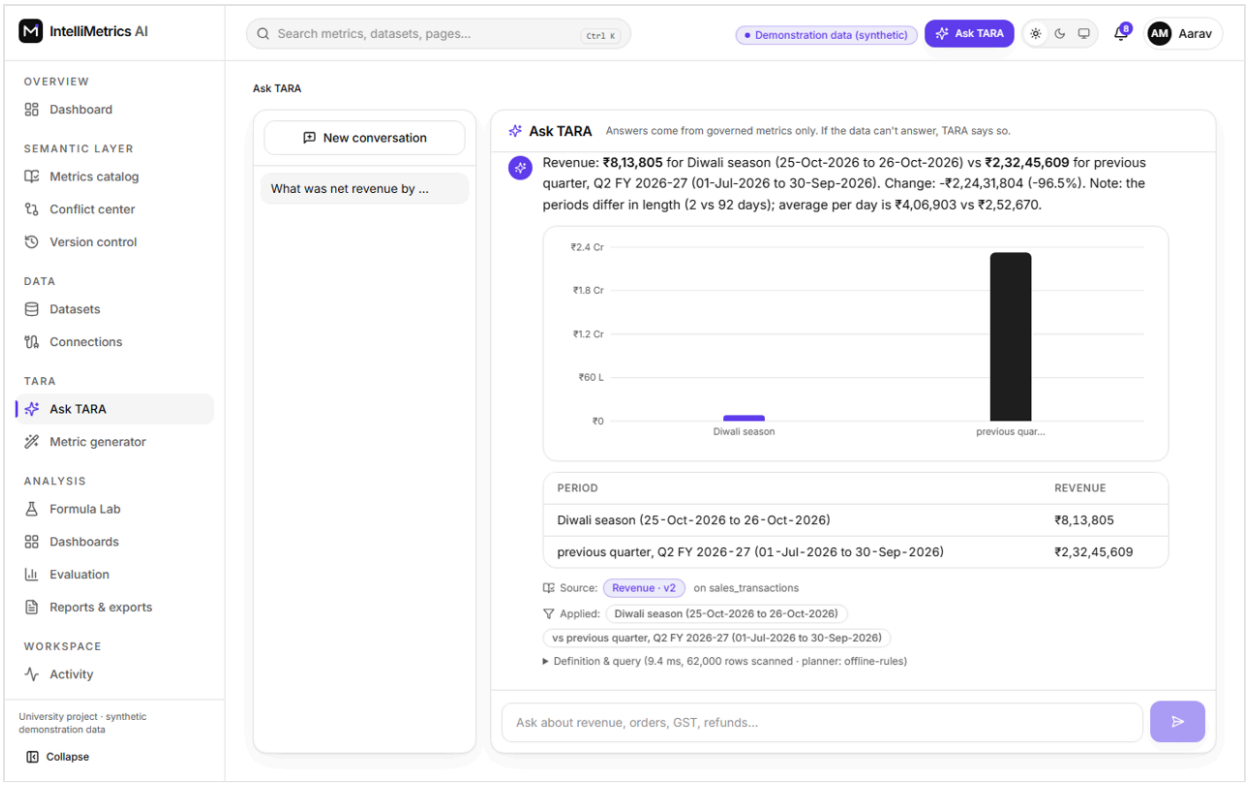
Conversation history, Indian suggested questions, streamed answers with KPI, table, chart, source metric and version, applied time range and filters, and the compiled query on demand.



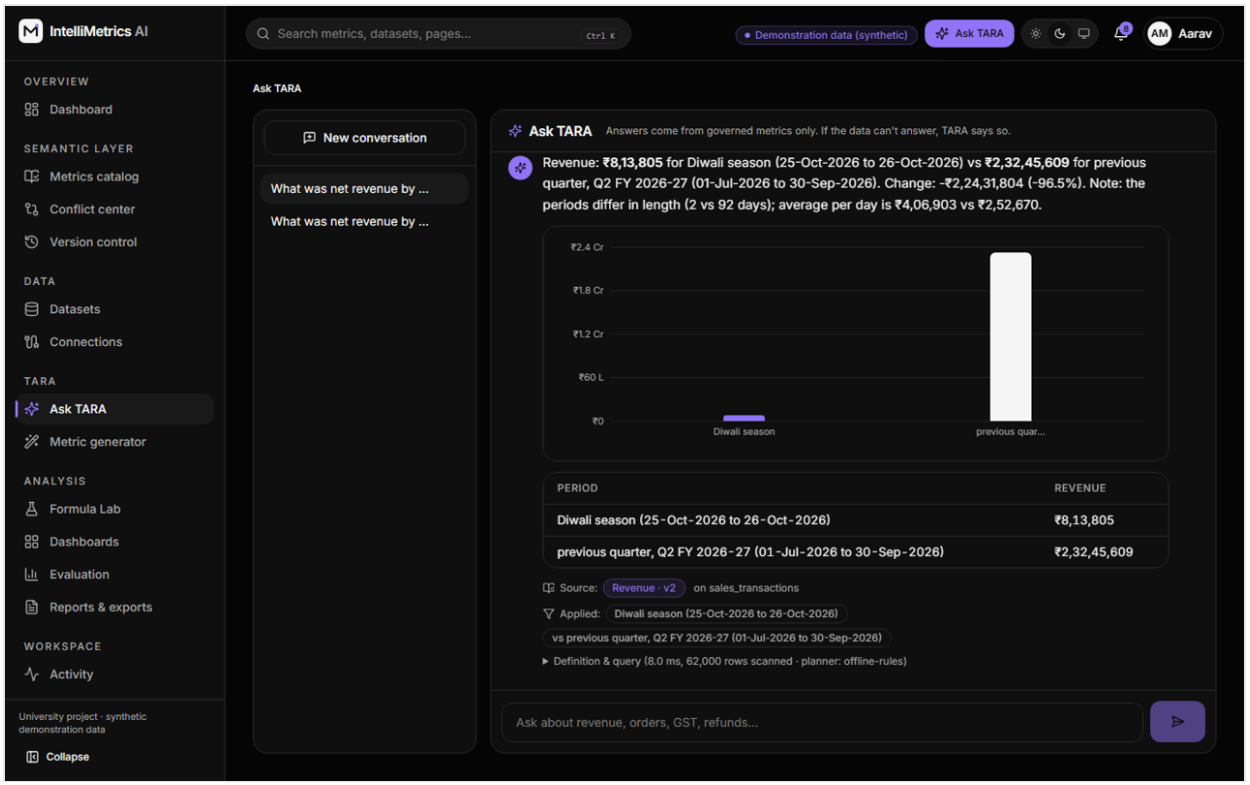
Ask TARA: suggested Indian questions, light mode



Ask TARA: suggested Indian questions, dark mode



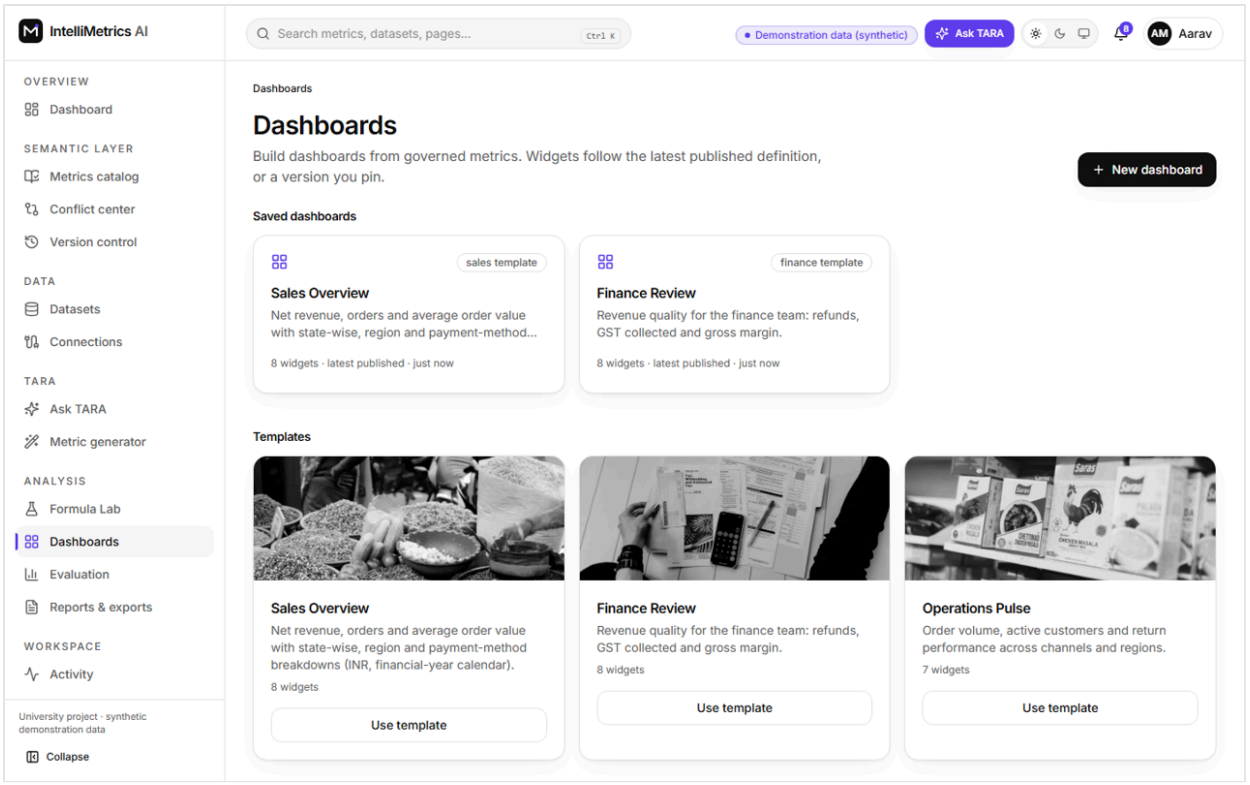
Ask TARA: answers with chart, table, sources and applied filters, light mode



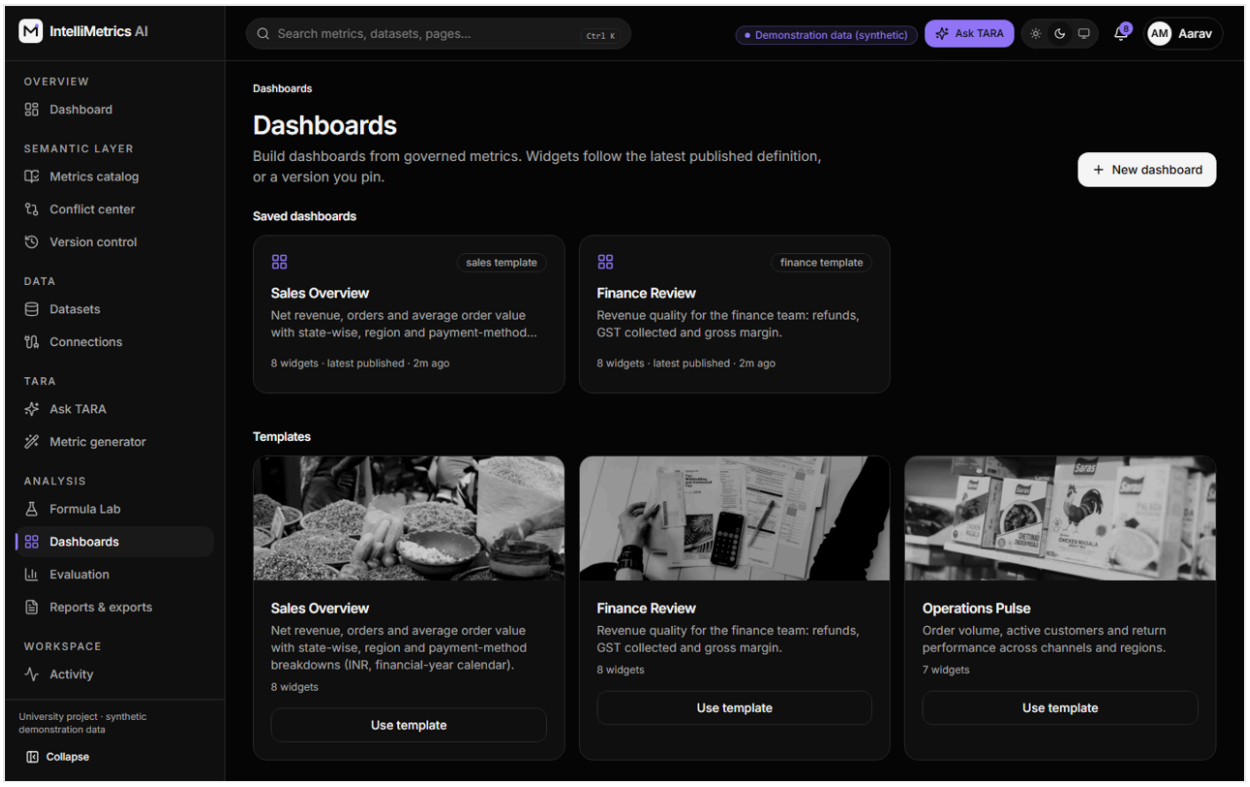
Ask TARA: answers with chart, table, sources and applied filters, dark mode

11.10 9. Dashboards: list, templates and builder

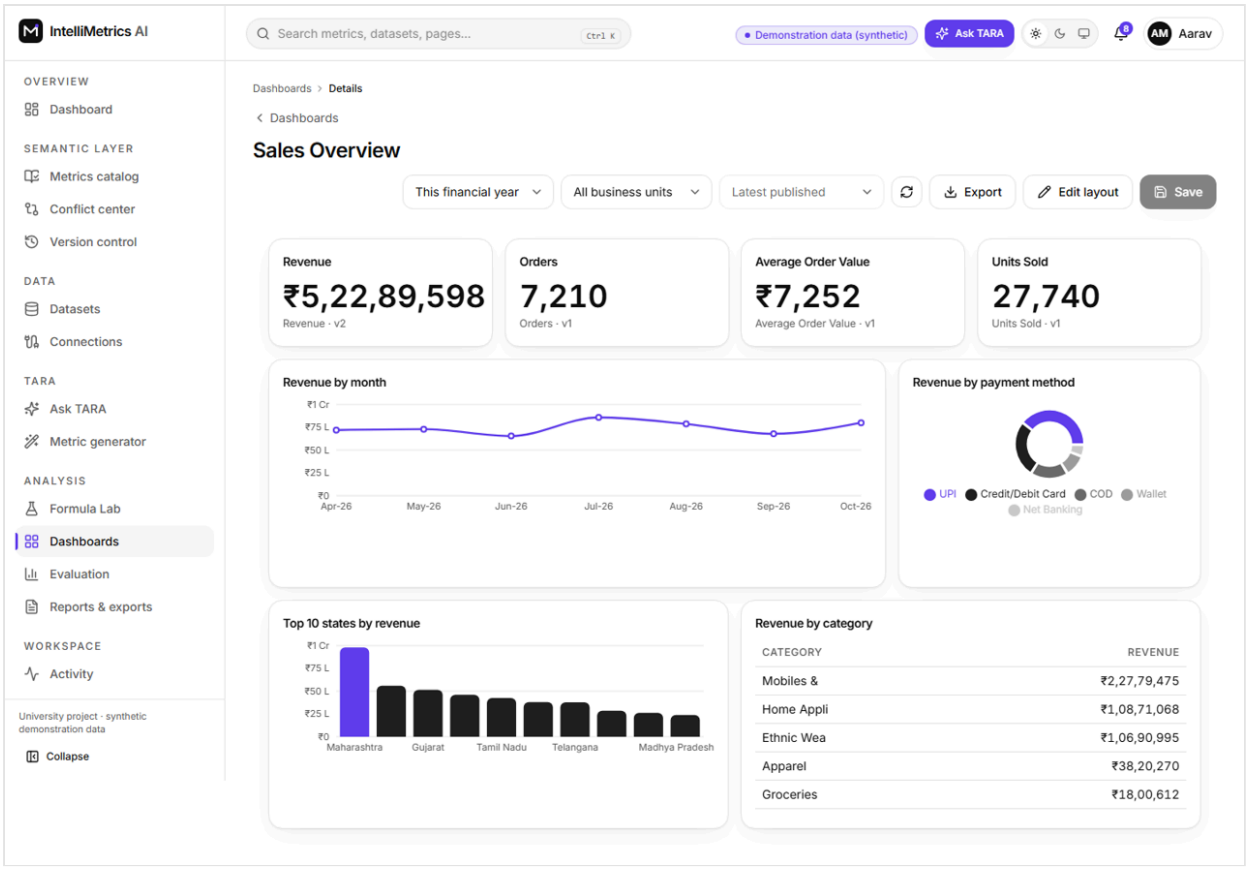
Saved dashboards and Sales, Finance and Operations templates; the builder offers KPI, line, bar, pie and table widgets, metric selection, time-range and business-unit filters, drag-to-rearrange and resize, refresh, a version-policy switch (latest published or pinned) and PNG/PDF/CSV export.



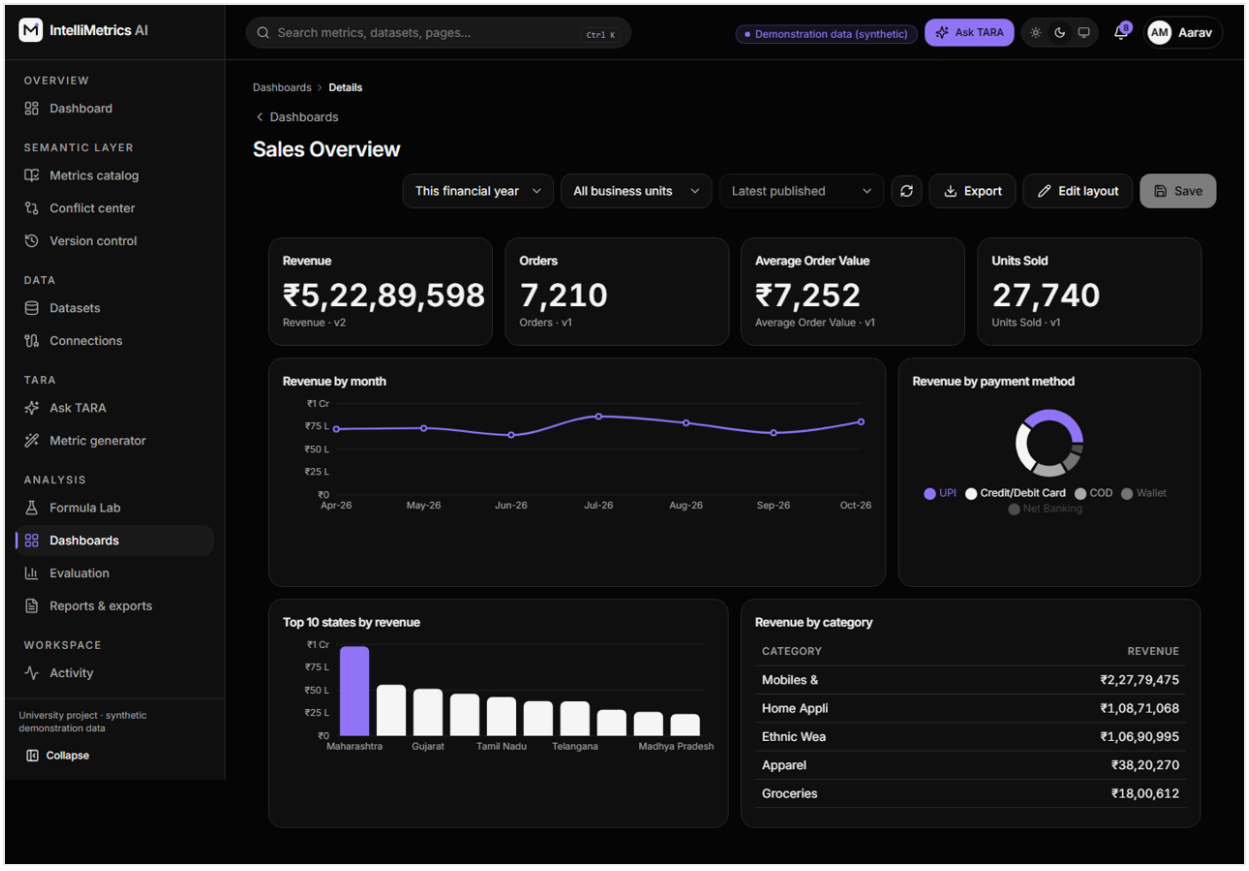
Saved dashboards and templates, light mode



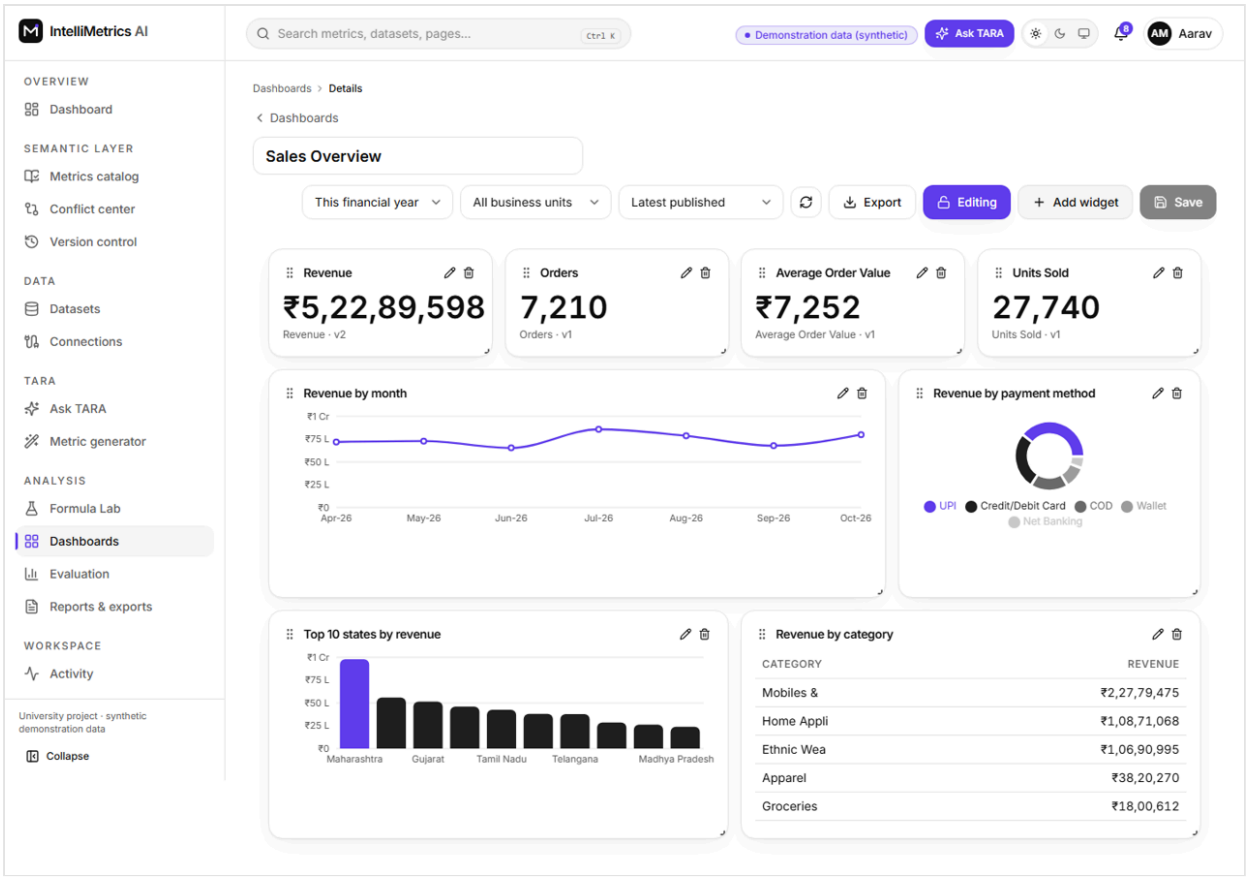
Saved dashboards and templates, dark mode



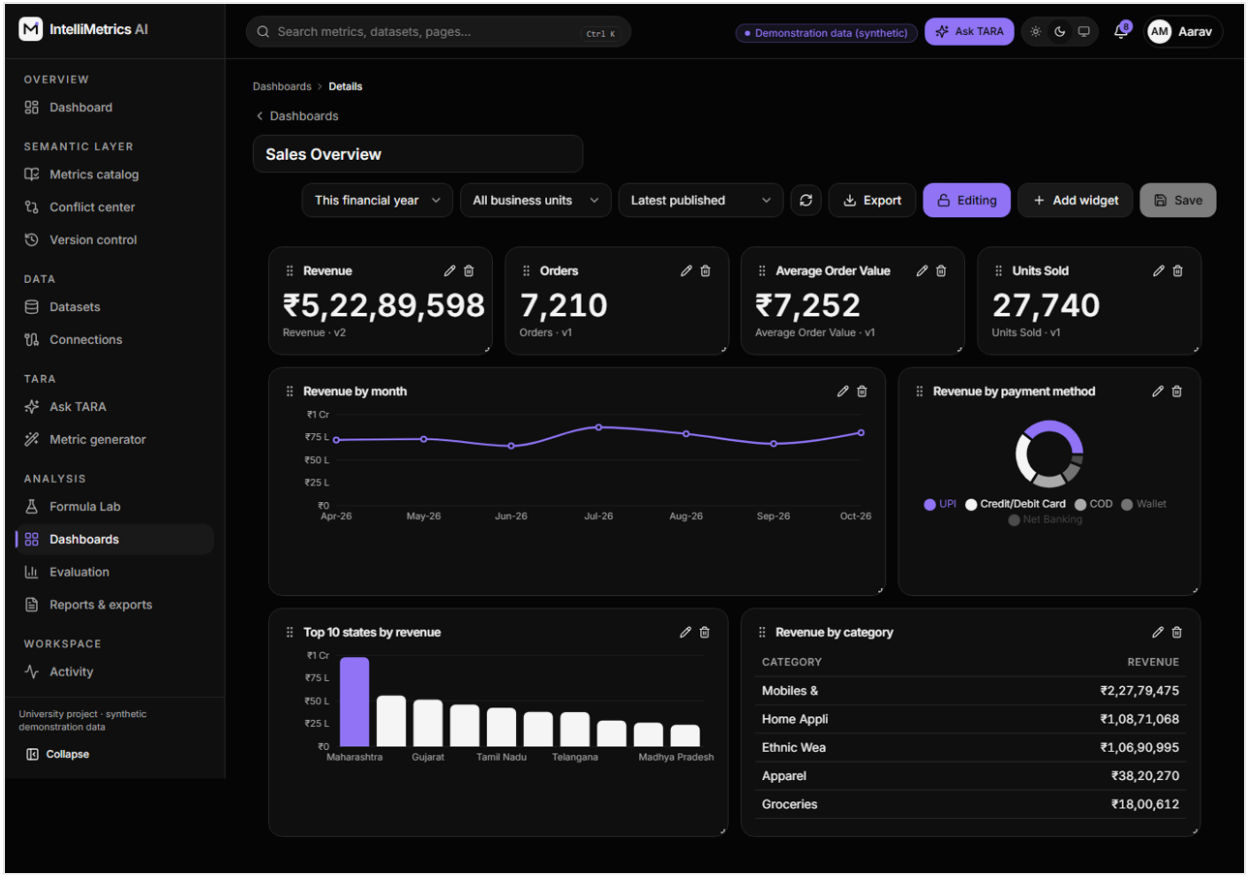
Dashboard builder (Sales Overview), light mode



Dashboard builder (Sales Overview), dark mode



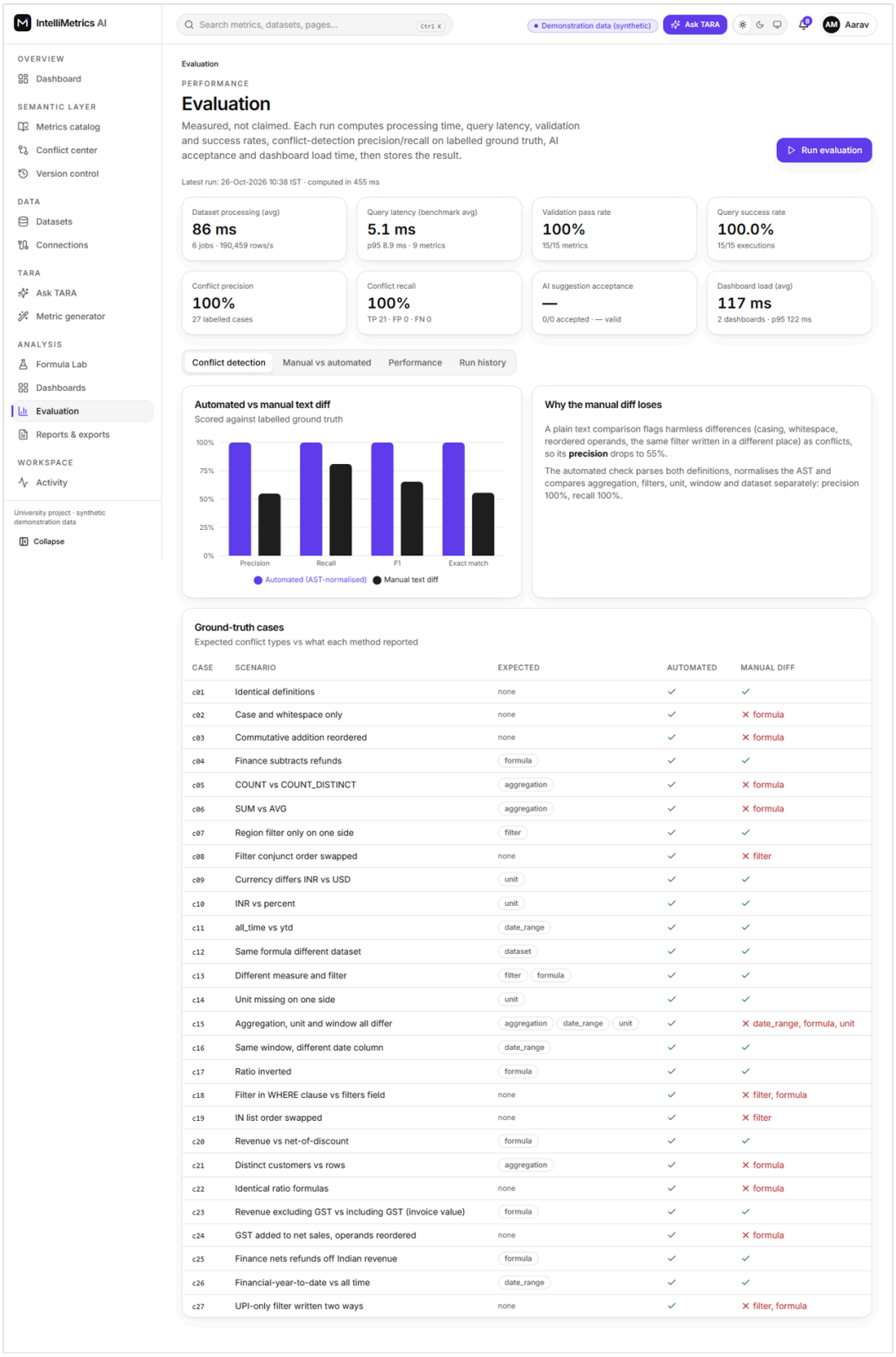
Dashboard builder in edit mode (drag handles), light mode



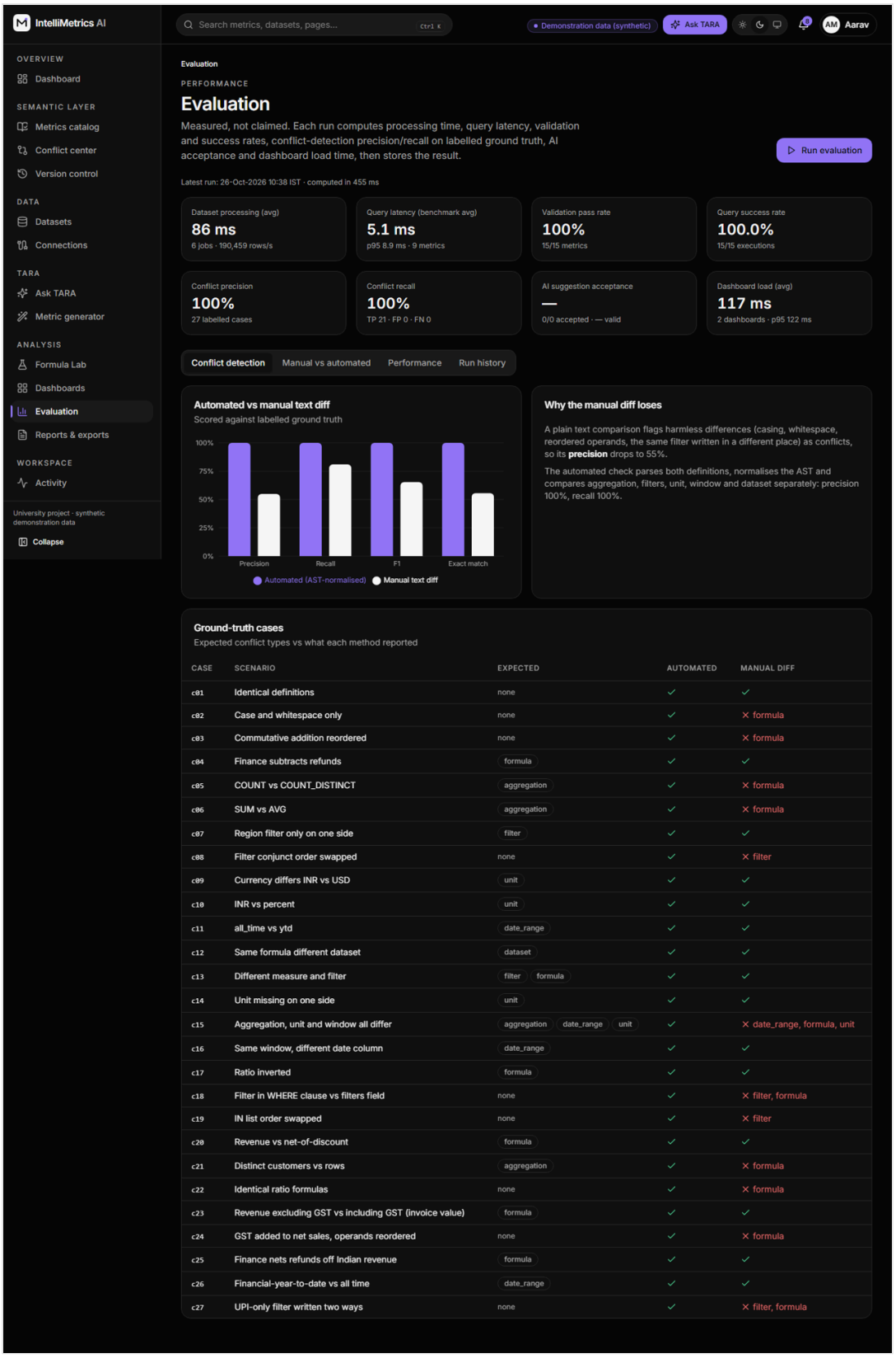
Dashboard builder in edit mode (drag handles), dark mode

11.11 10. Performance evaluation

“Run evaluation” really computes and stores results: processing time, query latency, validation and success rates, conflict precision and recall on labelled ground truth, AI acceptance, dashboard load, and two comparisons (manual versus AI-assisted creation; manual versus automated detection) with clearly marked assumptions.

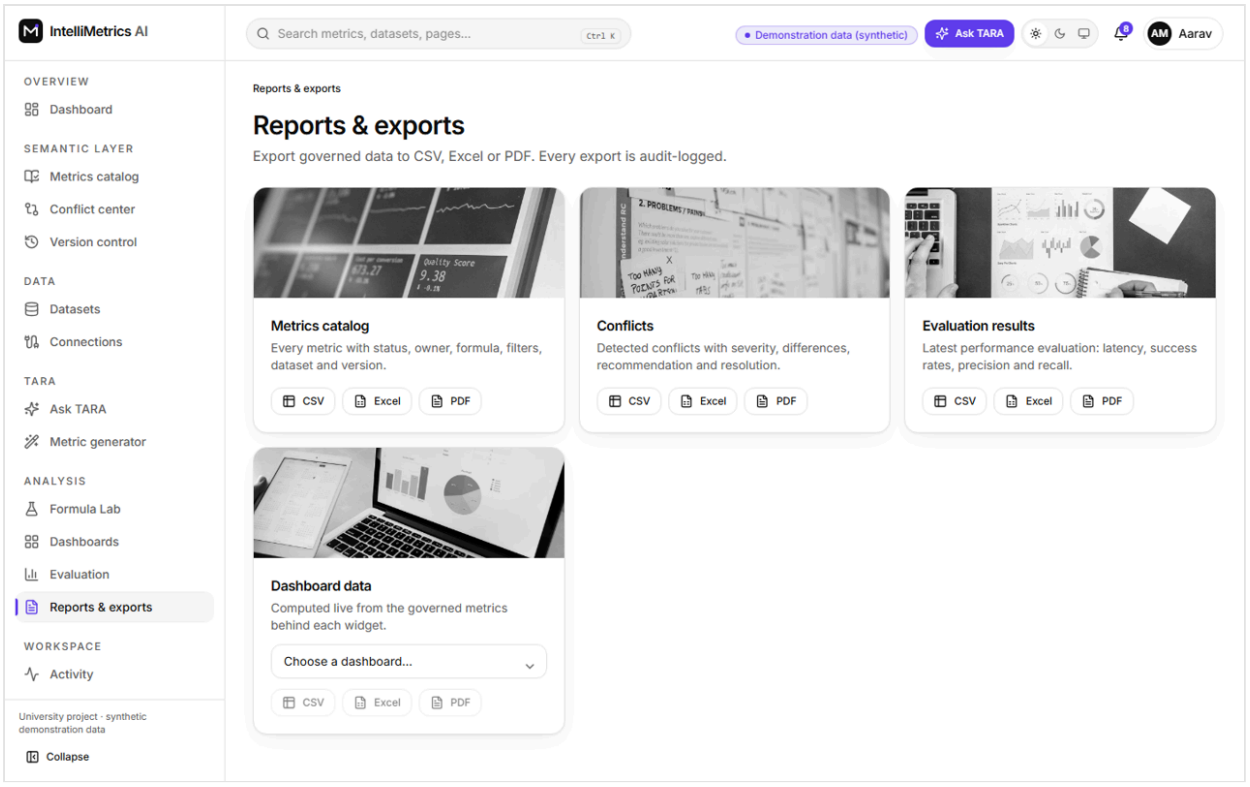


Performance evaluation, light mode

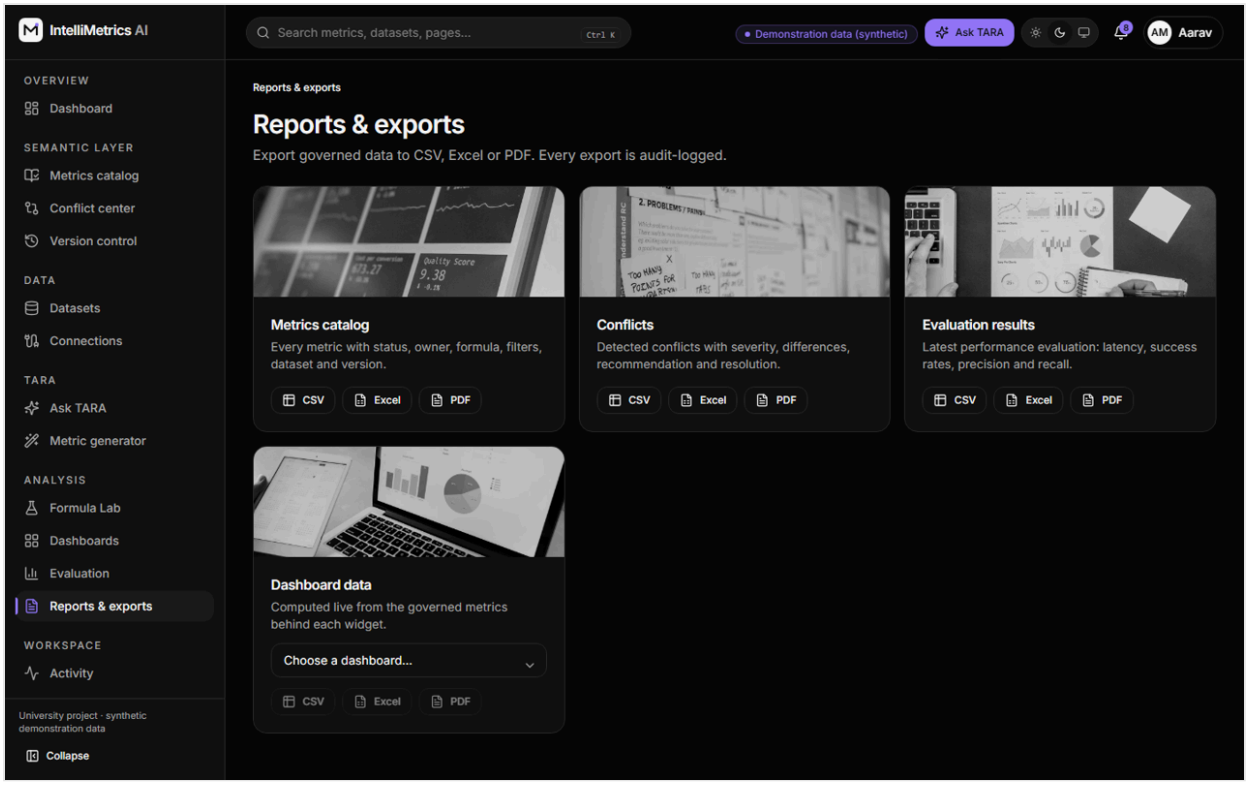


Performance evaluation, dark mode

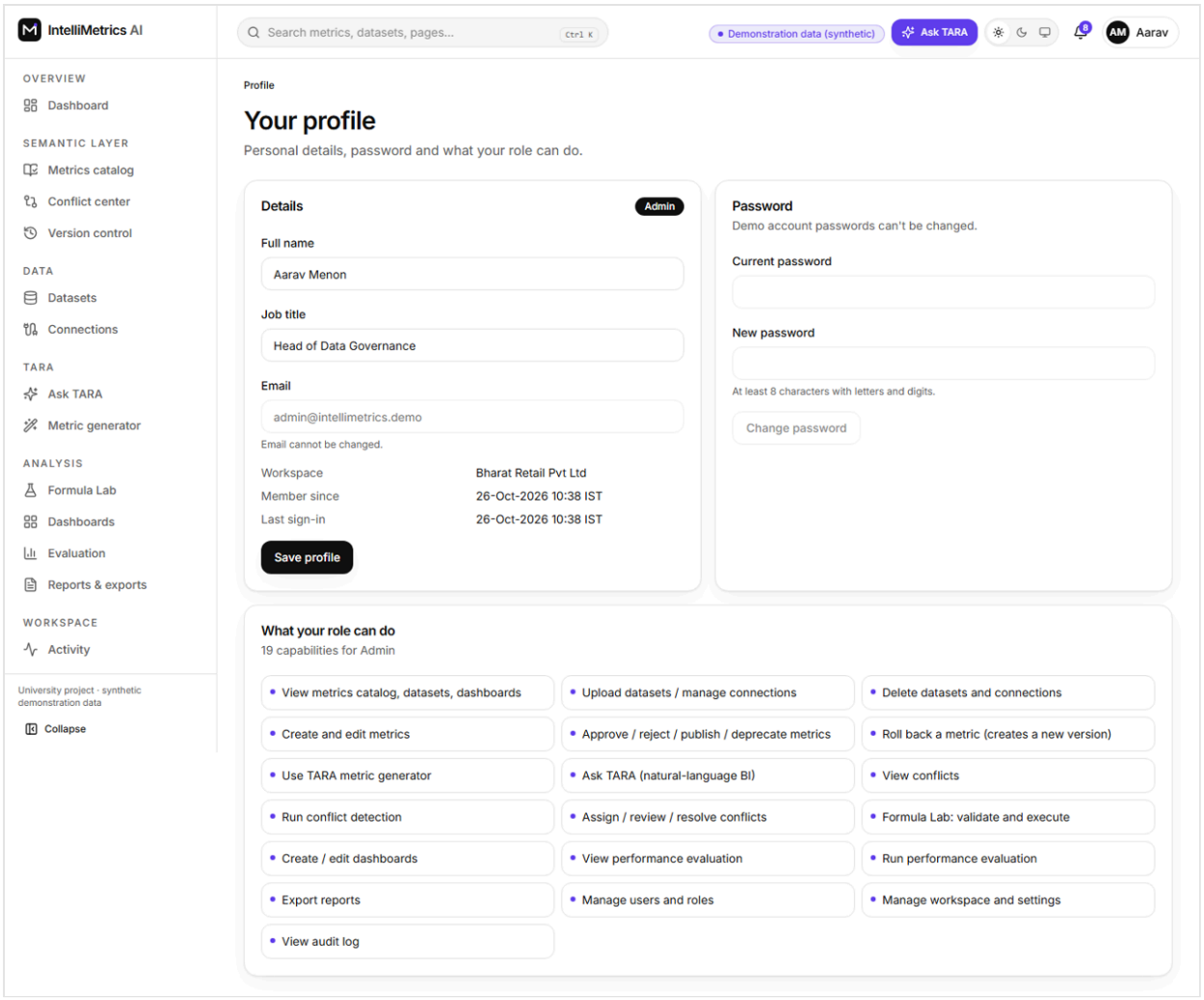
11.12 11. Reports and exports, profile, administration, settings, activity



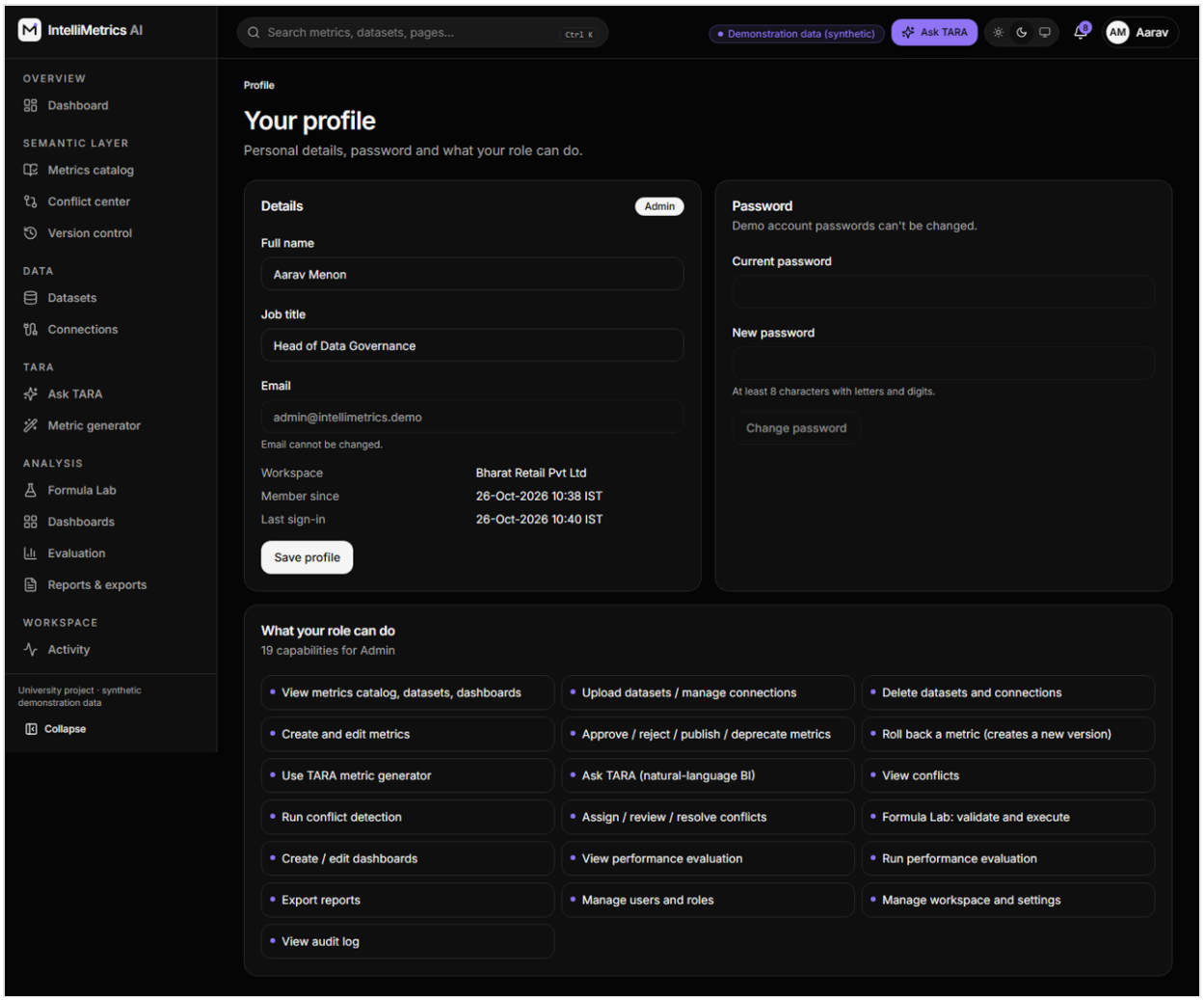
Reports and exports, light mode



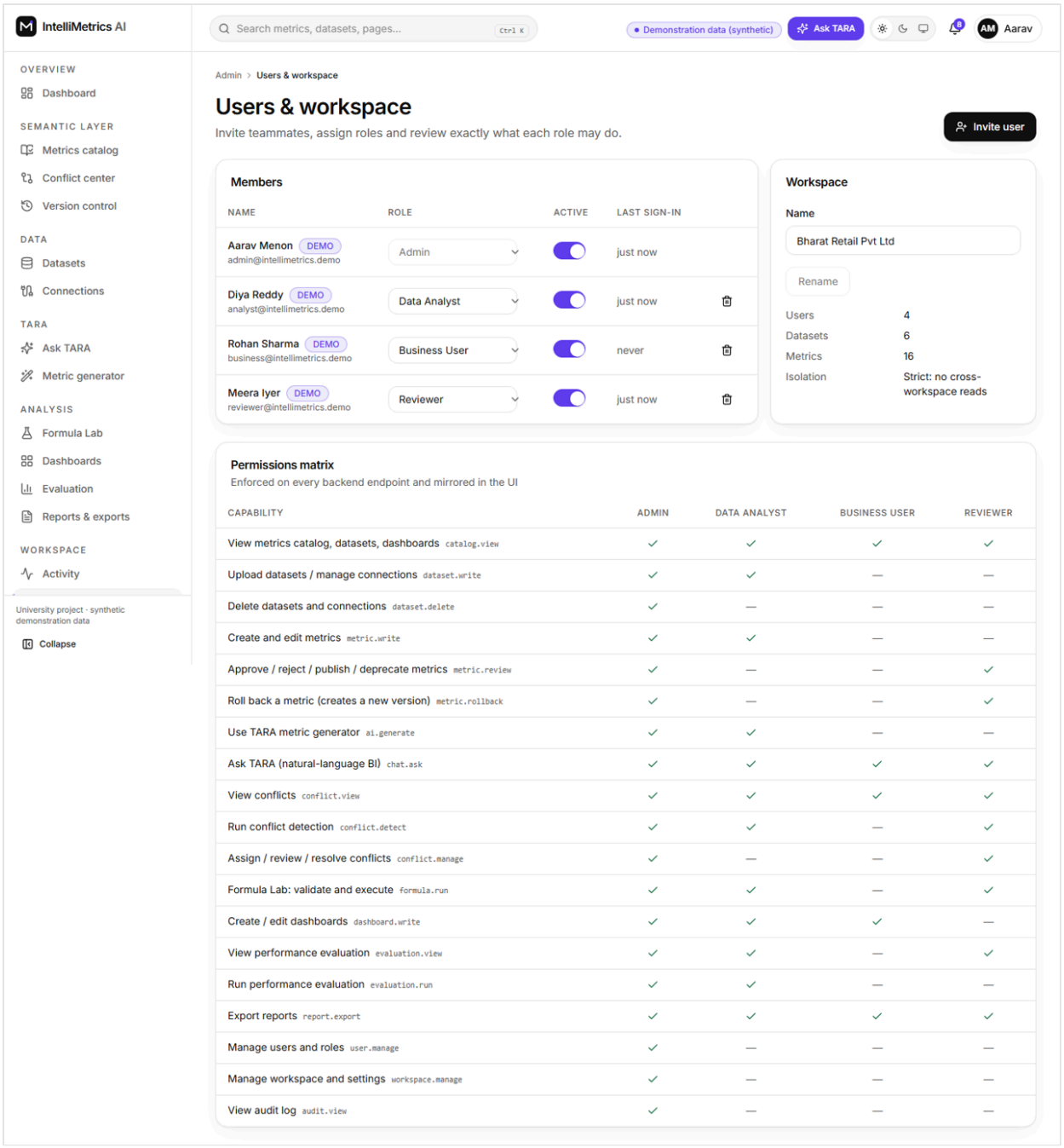
Reports and exports, dark mode



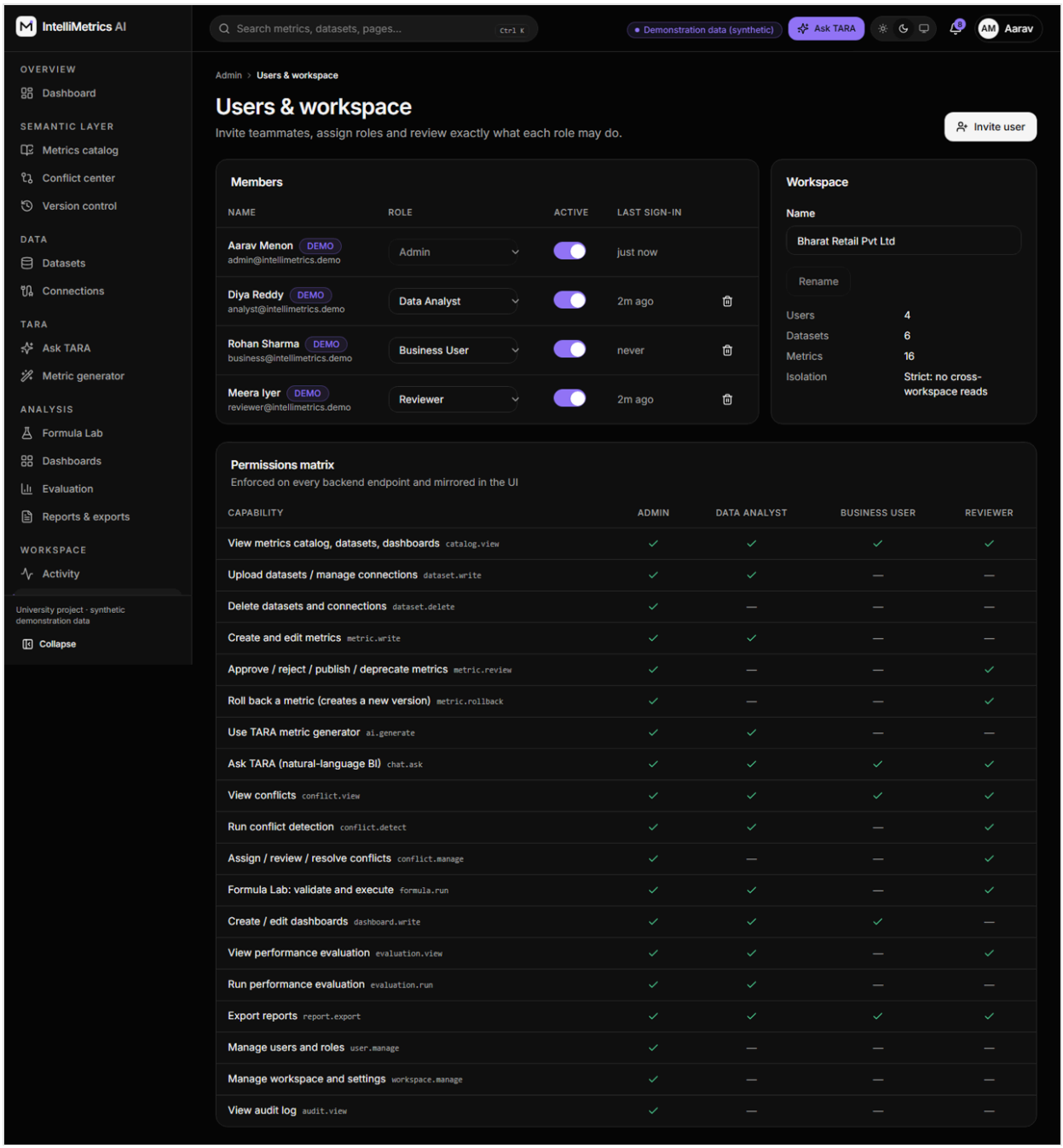
User profile, light mode



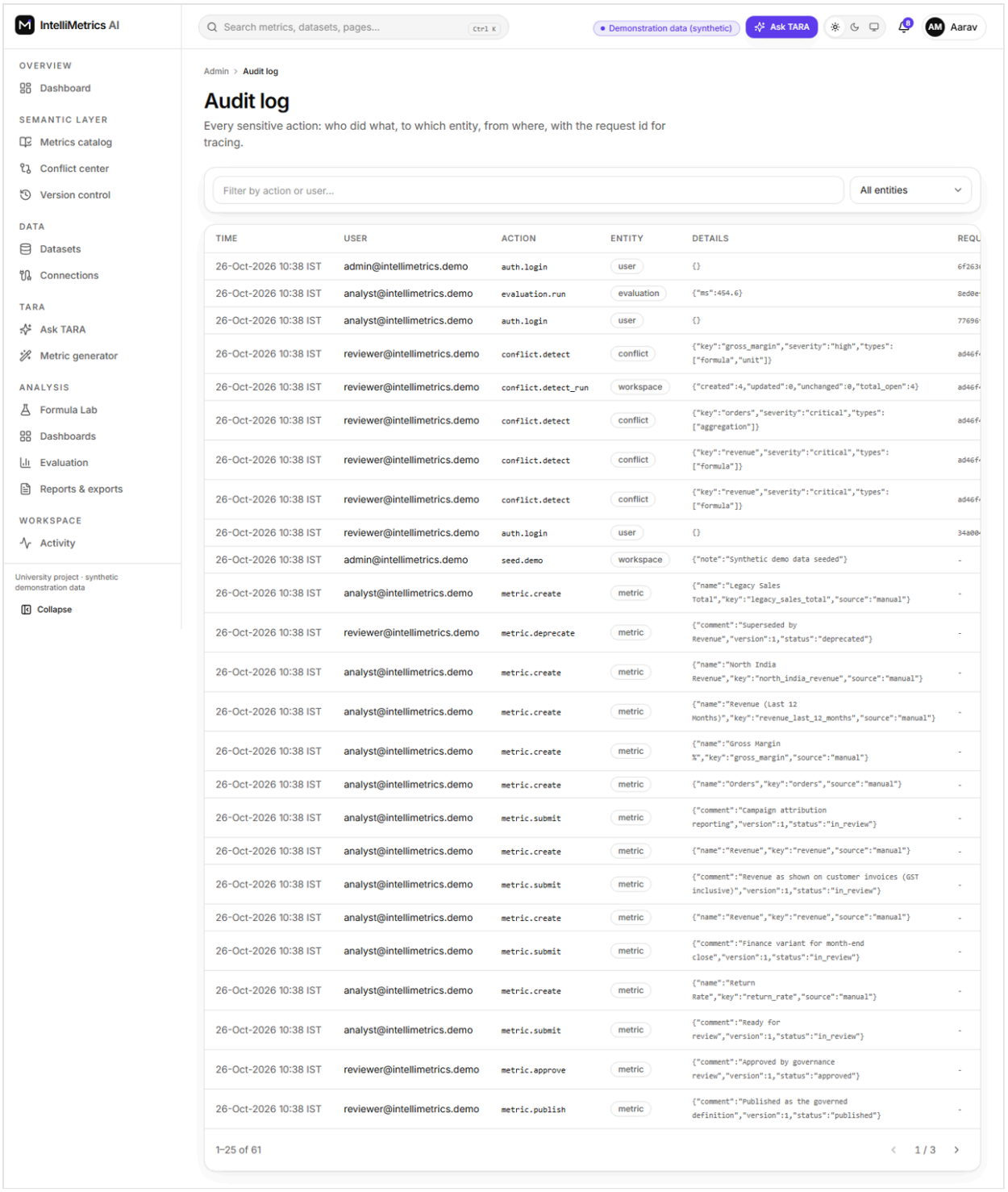
User profile, dark mode



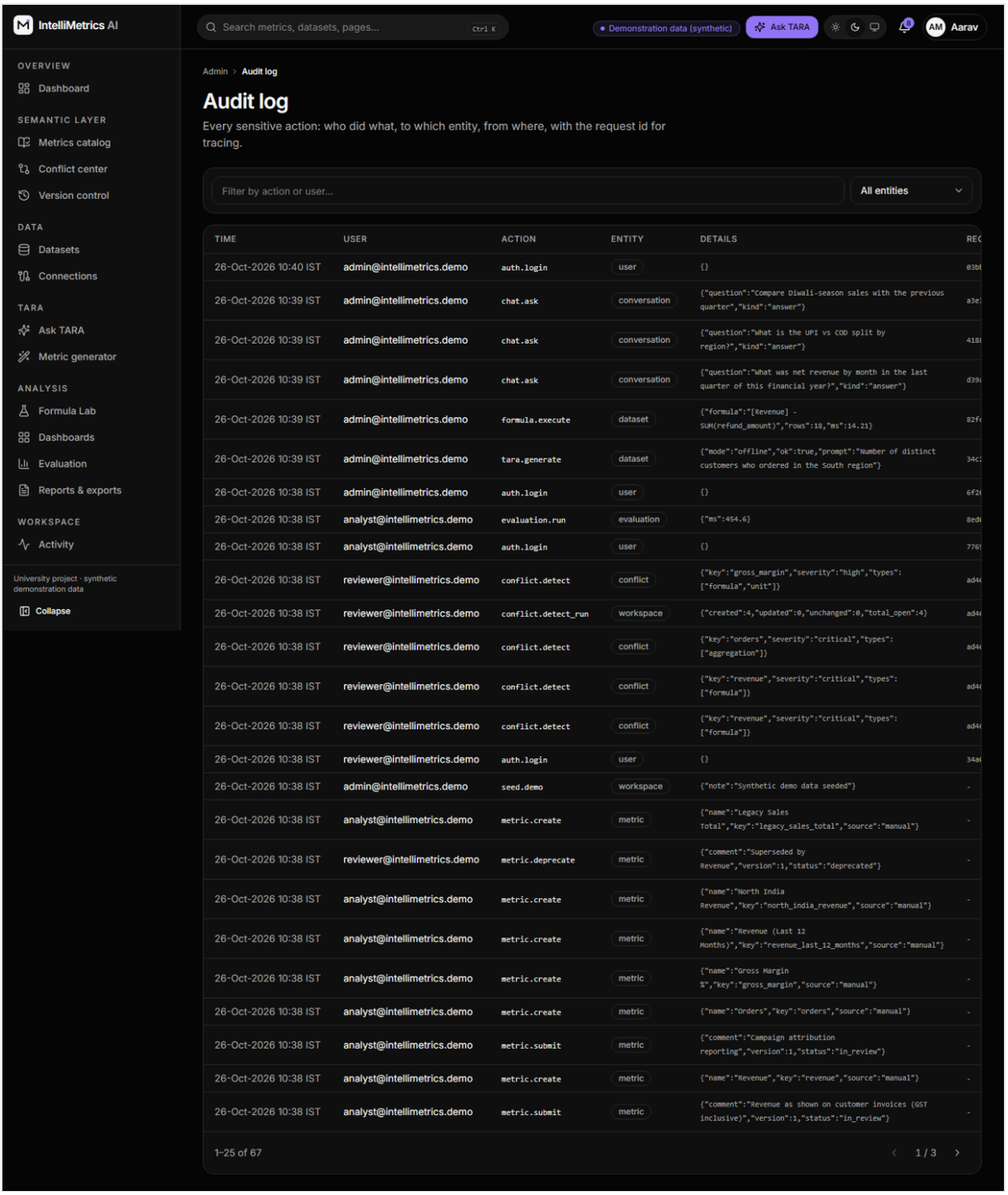
Workspace and user management, light mode



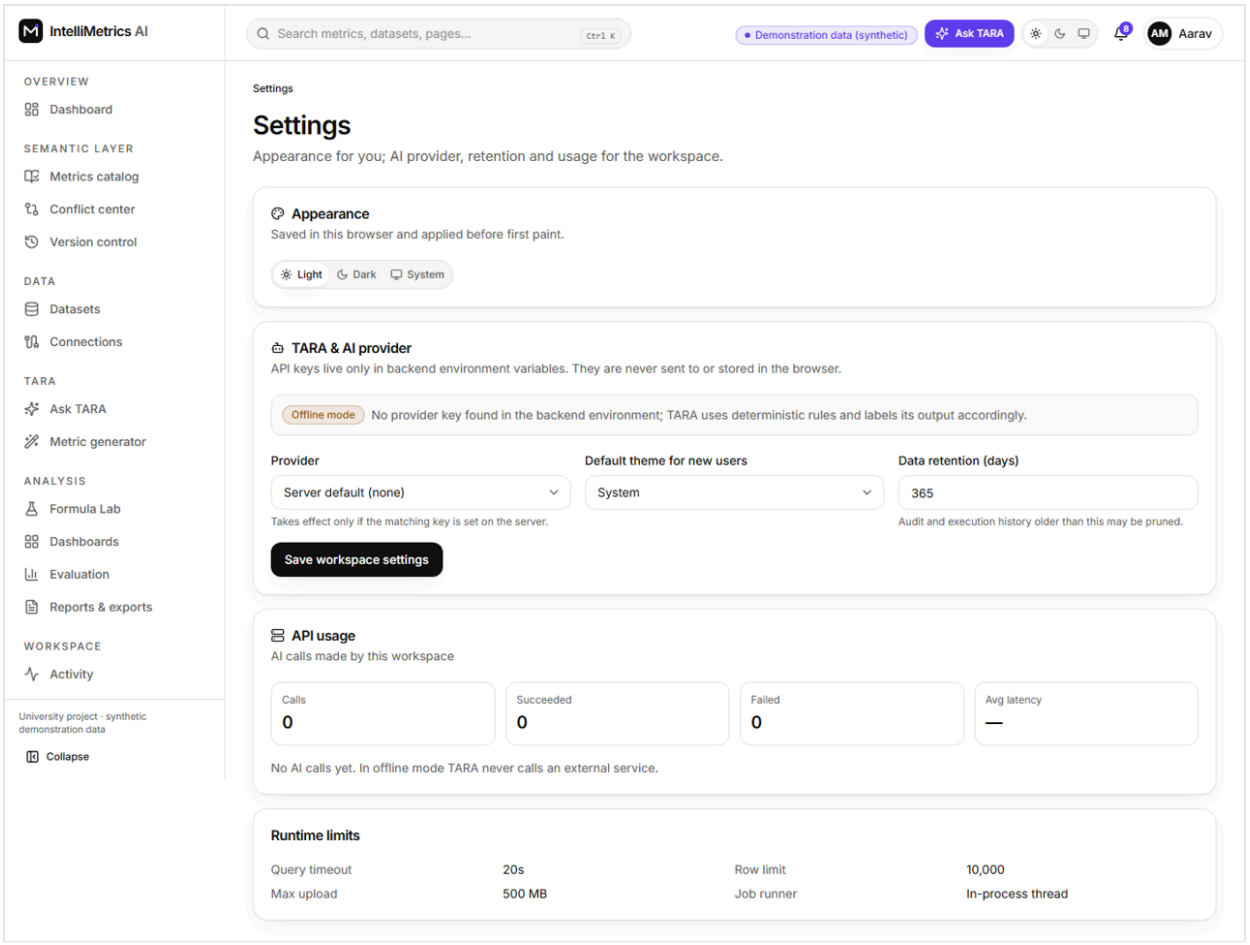
Workspace and user management, dark mode



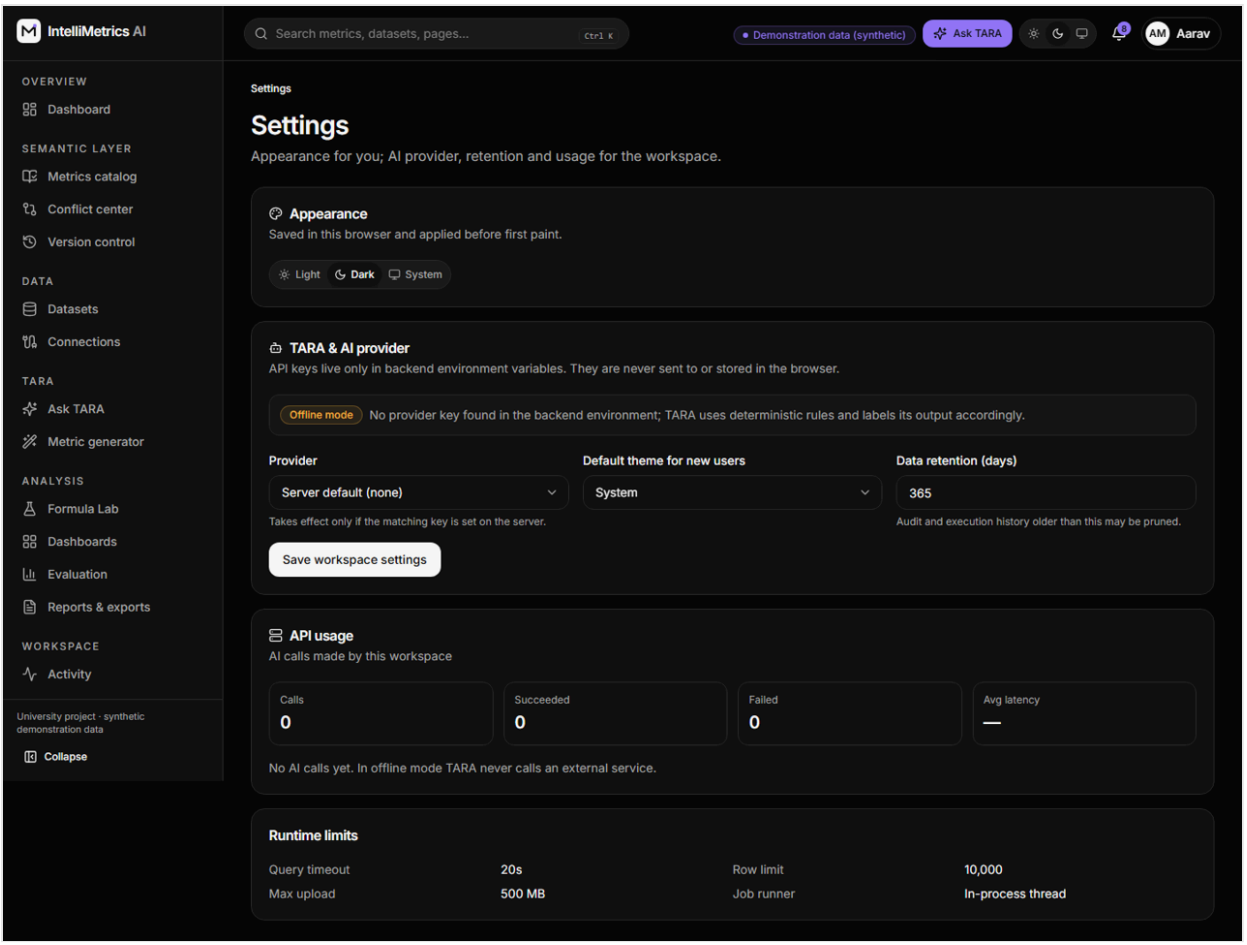
Audit log, light mode



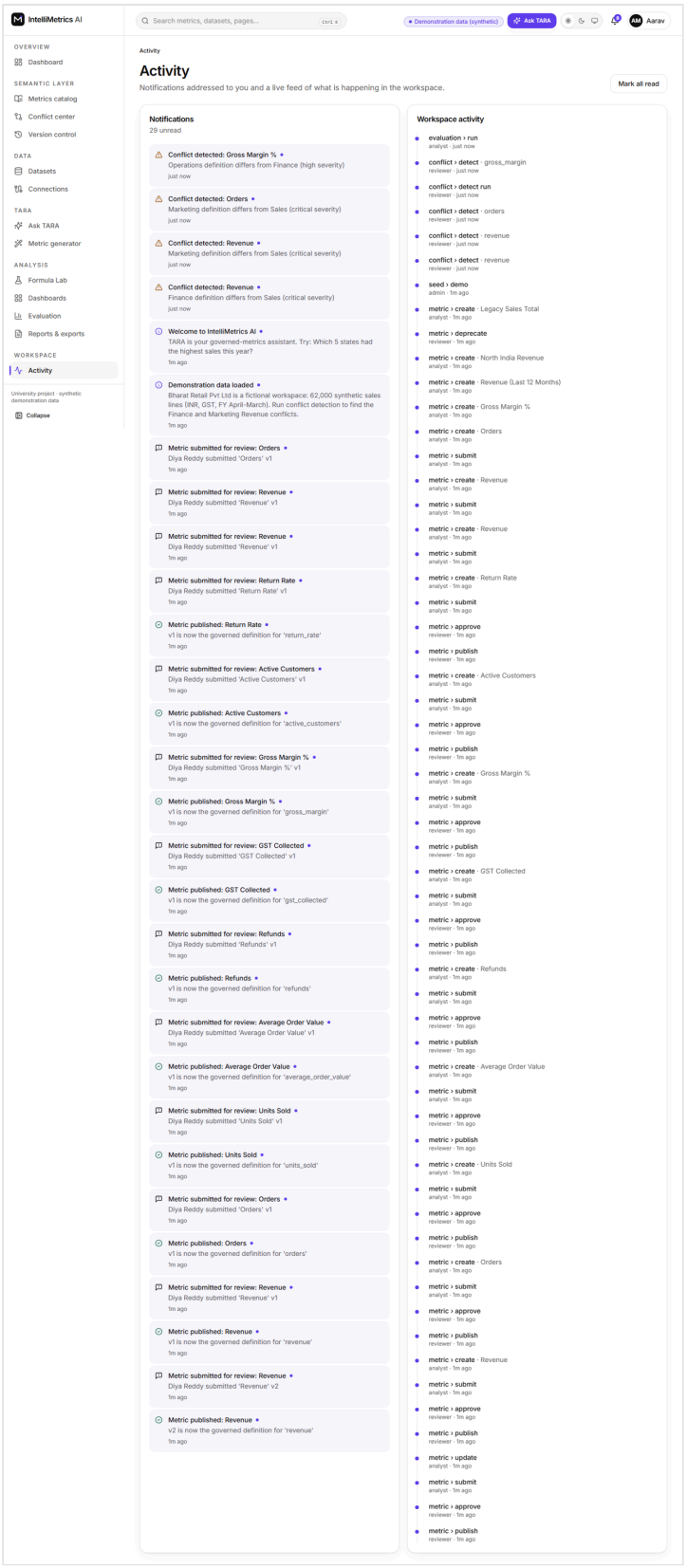
Audit log, dark mode



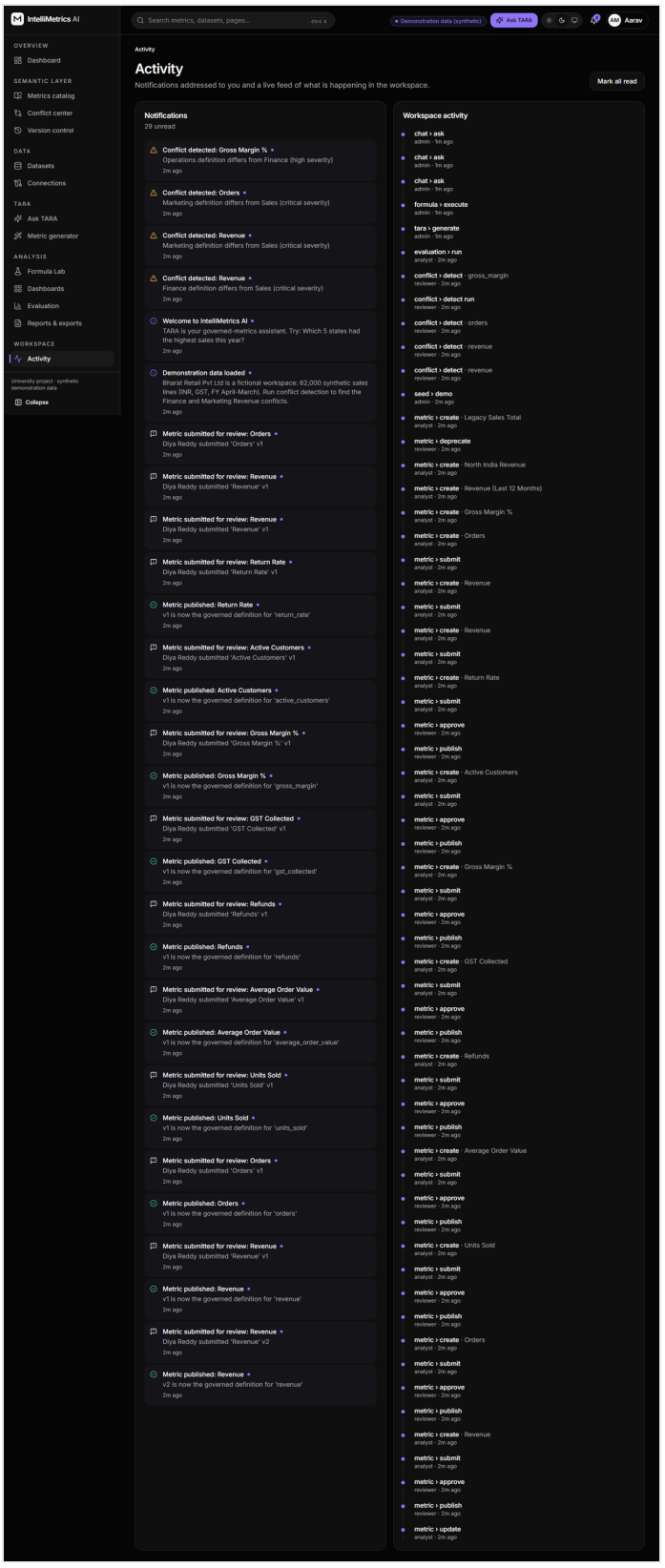
Application settings, light mode



Application settings, dark mode



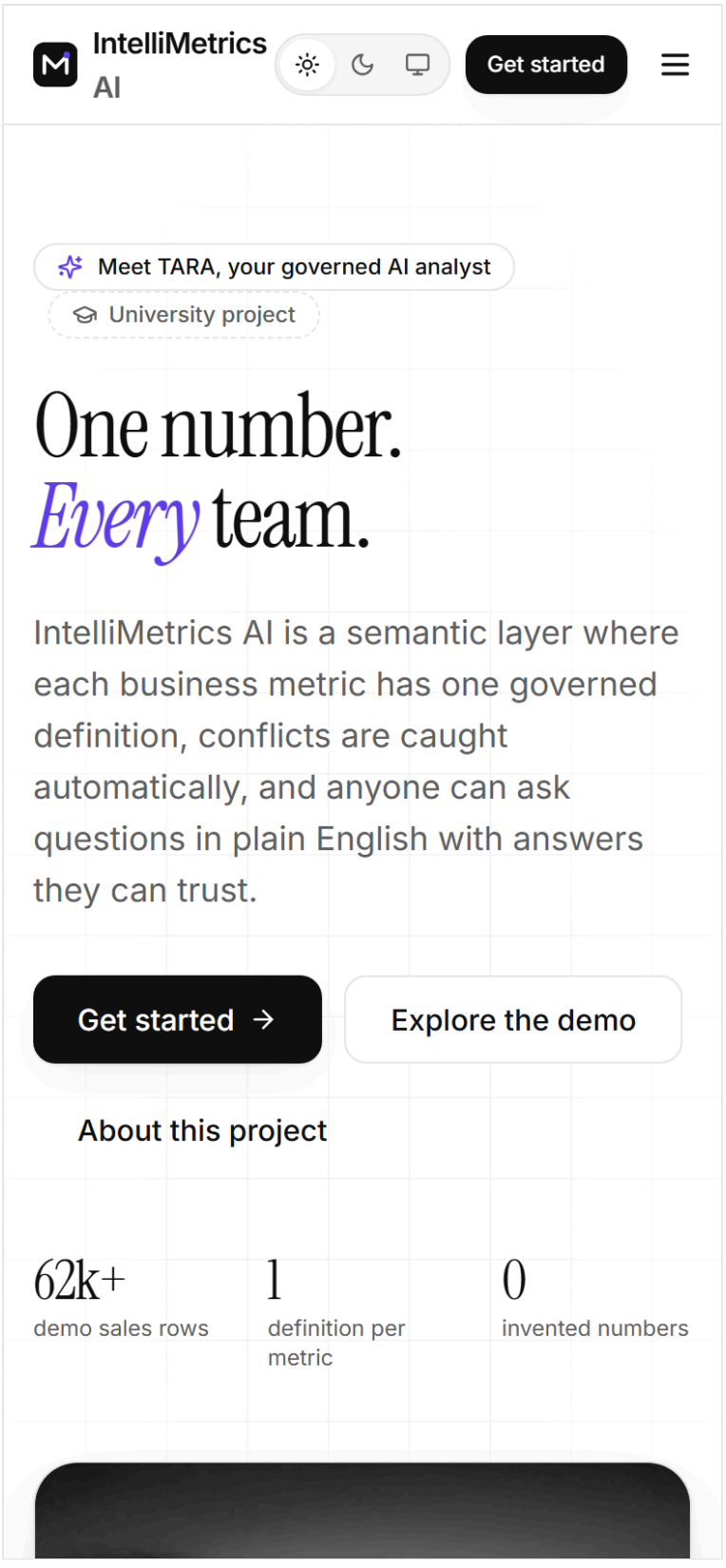
Notifications and activity, light mode



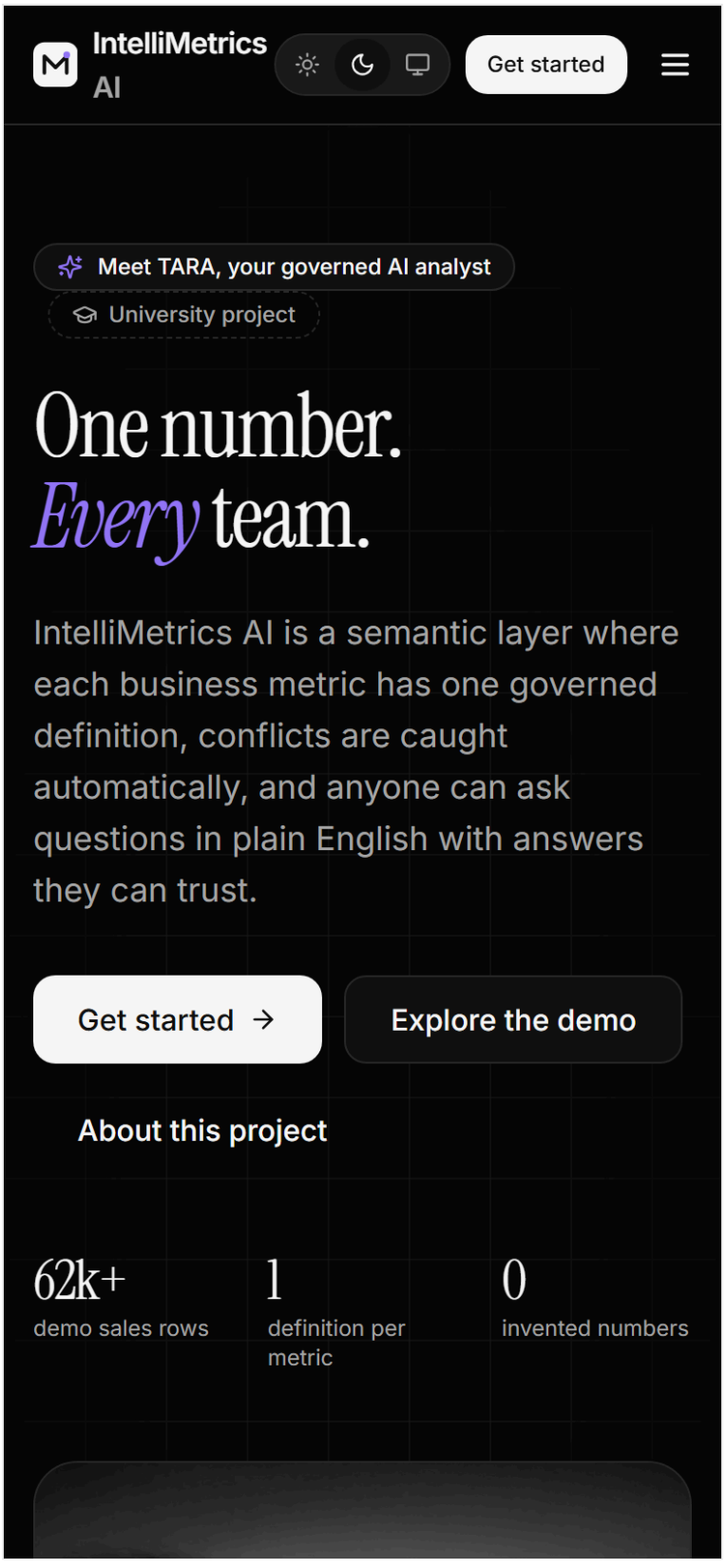
Notifications and activity, dark mode

11.13 Mobile layouts

Mobile: landing page

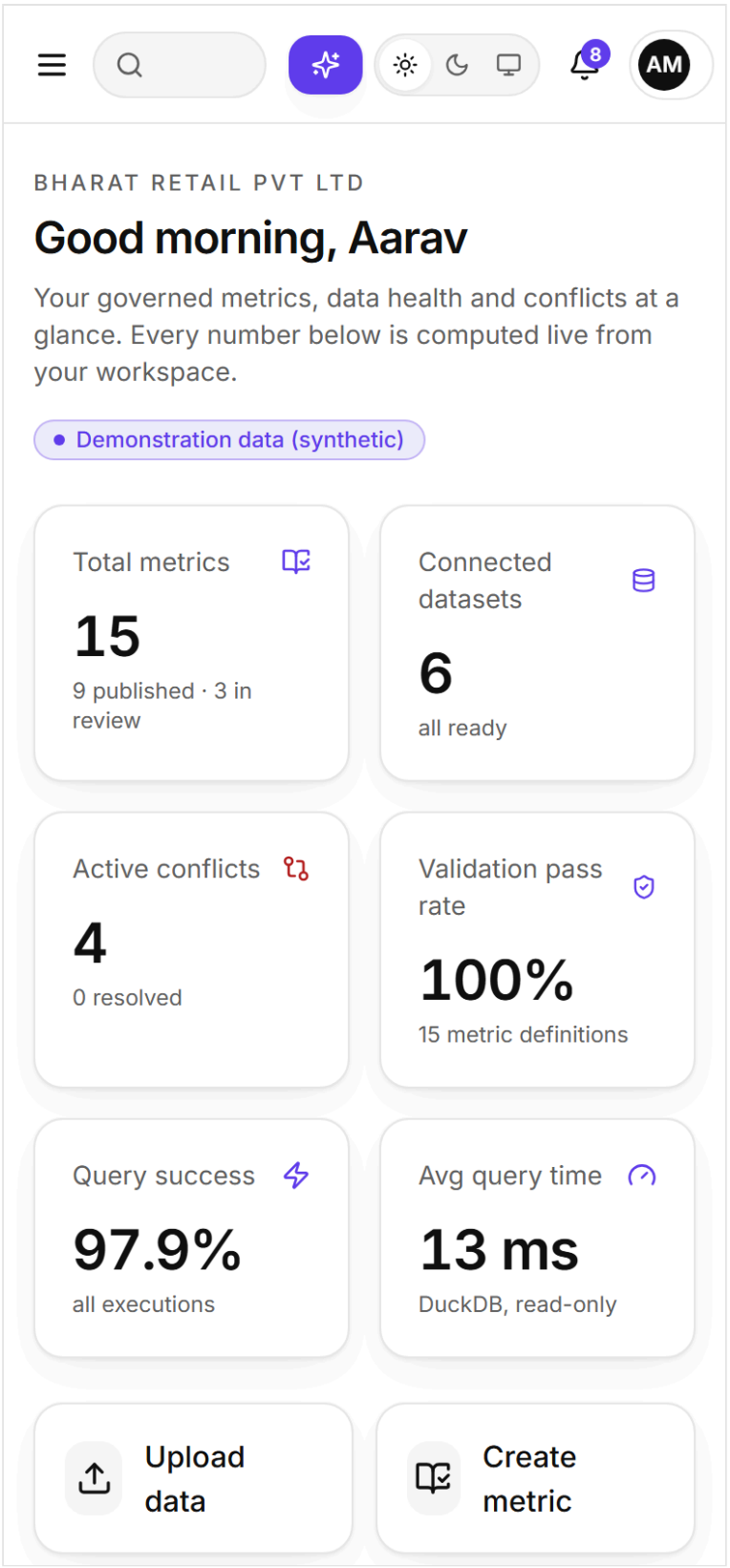


Mobile: landing page, light mode

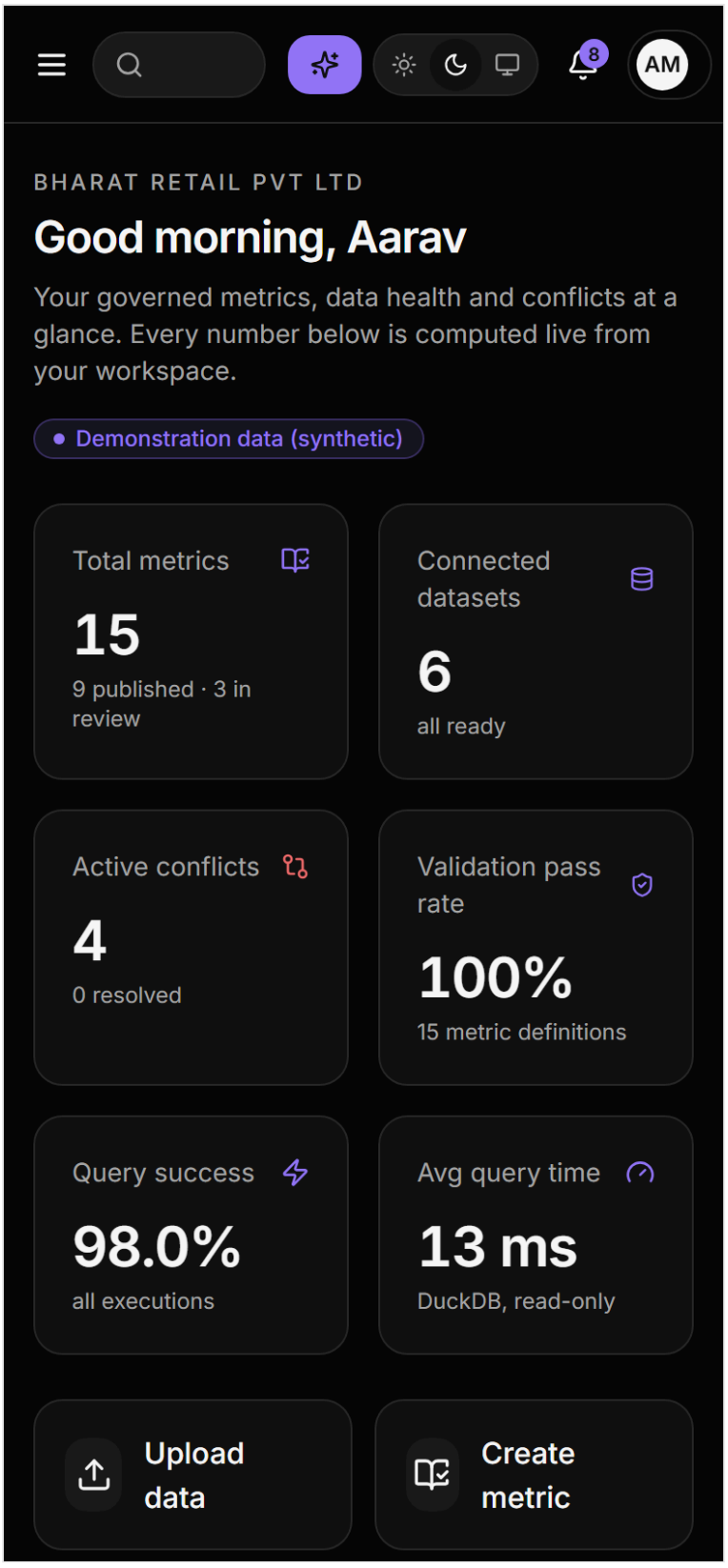


Mobile: landing page, dark mode

Mobile: dashboard

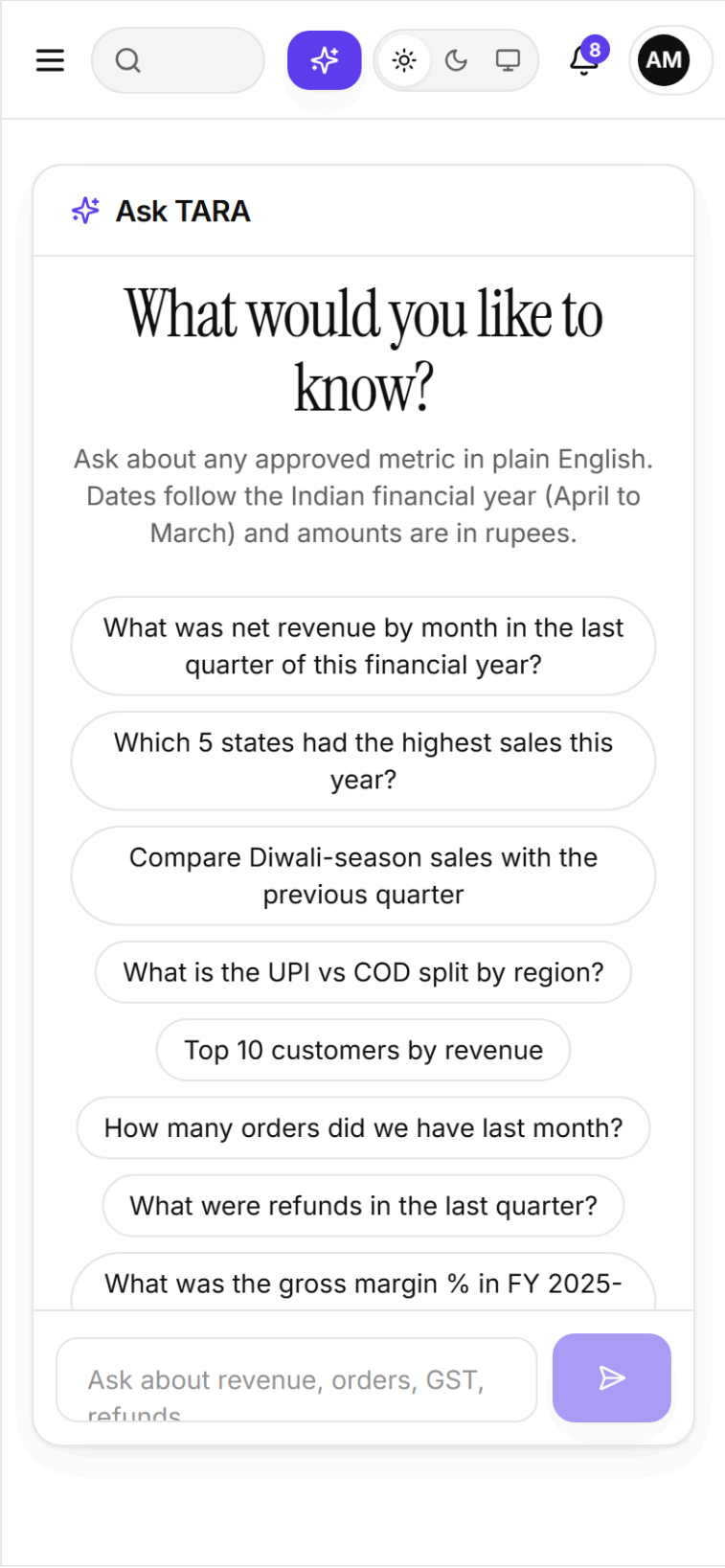


Mobile: dashboard, light mode

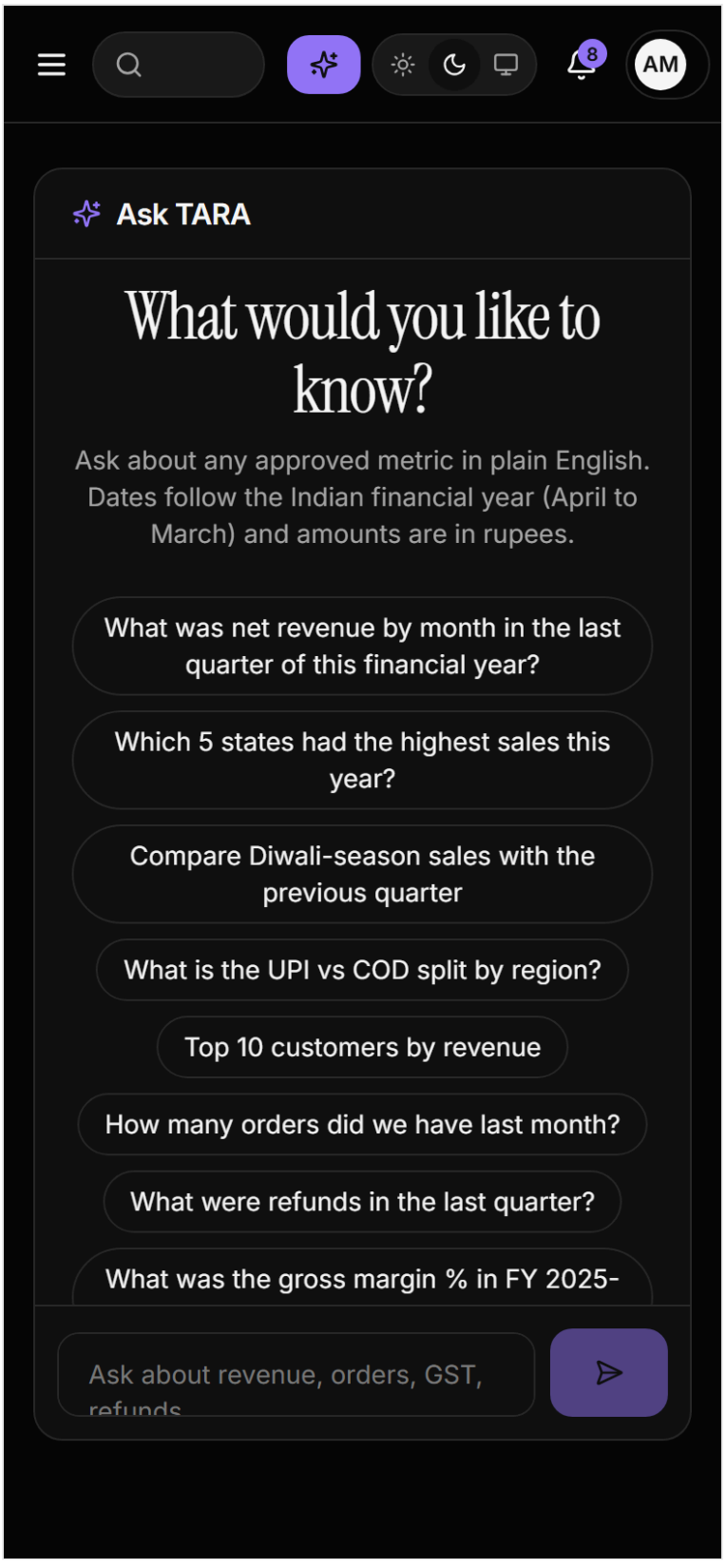


Mobile: dashboard, dark mode

Mobile: Ask TARA



Mobile: Ask TARA, light mode



Mobile: Ask TARA, dark mode

12 Why we built it

12.1 The business problem

The Indian retail example shows the problem precisely. For the same period, three departments can legitimately report three different “revenue” numbers:

Department	Definition	Formula	Includes GST	Subtracts refunds
Sales (official)	Taxable net sales	`SUM(net_sales)`	No	No
Finance	Net sales less refunds	`SUM(net_sales) - SUM(refund_amount)`	No	Yes
Marketing	Customer invoice value	`SUM(net_amount)`	**Yes**	No

Neither Finance nor Marketing is *wrong*; they are answering different questions with the same word. The platform makes the difference visible, forces a decision (rename, align, deprecate or consciously accept both), records who decided and why, and ensures that dashboards and TARA use only the published definition.

12.2 Who benefits

- **Business users** get trustworthy answers in plain English without learning SQL, with the definition and filters shown beside every number.
- **Analysts** write formulas once, get instant validation against the real schema and keep a history of every change.
- **Reviewers and finance controllers** see conflicts early and approve or publish with an audit trail.
- **Administrators** control roles, workspaces and settings, and can prove who changed what.

12.3 Why a governed layer plus AI

A semantic layer alone does not help a non-technical user ask a new question; an LLM alone cannot guarantee that it will use the organisation’s definition or even valid SQL. Combining them gives *both* ease of access and correctness: the AI chooses *which* governed metric and *which* breakdown, and never computes the value.

13 What we used and why

Technology	Version	Purpose in this project	Why chosen	Alternatives considered
React	18.3.1	Component-based UI for all pages	Largest ecosystem, hooks, mature testing tools; fits the many interactive widgets (grid, charts, chat)	Vue, Svelte, Angular
TypeScript	5.7.2	Static typing for the front end	Catches API-shape and prop errors at build time (`tsc --noEmit` is part of CI)	Plain JavaScript
Vite	6.0.5	Dev server, proxy and production build	Very fast HMR; simple proxy of `/api` ; small config	Create React App (deprecated), webpack, Next.js
Tailwind CSS	3.4.17	Styling through design tokens (CSS variables)	Utility classes + variables give a strict black/white brand with one accent and instant light/dark themes	CSS modules, Chakra UI, Material UI
shadcn-style components on Radix UI	dialog 1.1.2, tabs 1.1.1, dropdown 2.1.2	Accessible dialog, tabs and menu primitives wrapped in local components (`components/ui`)	Accessibility (focus trap, ARIA, keyboard) without a heavy theme; components are owned in the repo	MUI, Ant Design, Headless UI
Recharts	2.15.0	Line, bar, area, pie, grouped charts	Declarative React charts, themeable with CSS variables	Chart.js, ECharts, D3 directly
React Router	6.28.0	Routing, protected routes, 403/404	De-facto standard; nested layouts for public vs app shells	TanStack Router
TanStack Query	5.62.0	Server-state caching, polling (ingestion progress), mutations	Removes hand-written loading/error state; refetch intervals for job progress	Redux Toolkit Query, SWR
Framer Motion	11.15.0	Page transitions, hover lifts, animated counters	Declarative animation with reduced-motion support	CSS only, React Spring
lucide-react	0.468.0	Icon set	Consistent, tree-shakeable icons	Font Awesome, Heroicons
react-grid-layout	1.5.0	Drag and resize dashboard widgets	Purpose-built grid with persisted layouts	Gridstack, custom CSS grid
html-to-image + jsPDF	1.11.11 / 2.5.2	Dashboard export to PNG and PDF in the browser	Works from the rendered DOM without a server	Server-side headless rendering

Technology	Version	Purpose in this project	Why chosen	Alternatives considered
FastAPI	0.115.6	REST API, validation, OpenAPI docs at `/docs`	Type-driven validation (Pydantic), dependency injection for RBAC, automatic OpenAPI, async-ready	Flask, Django REST Framework, Node/Express
Uvicorn	0.34.0	ASGI server	Standard FastAPI server; multi-worker in Compose	Gunicorn + uvicorn workers, Hypercorn
Pydantic	2.10.4	Request/response models; validation of AI structured output	Strict schemas turn malformed AI JSON into handled errors	dataclasses, marshmallow
SQLAlchemy	2.0.36	ORM and query construction (2.0 typed style)	Portable across SQLite and PostgreSQL; unit-of-work; event hooks for immutability	Django ORM, raw SQL, Tortoise
Alembic	1.14.0	Schema migrations for every table	Autogenerate + reviewable revisions; PostgreSQL-specific trigger migration	Flyway, Liquibase
PostgreSQL	16 (Compose); 18.2 verified locally	System of record: metadata, versions, audit, conflicts	ACID, partial unique indexes (one live definition per key), triggers, JSON	MySQL, MongoDB
SQLite	bundled with Python	Zero-setup local development and fast tests	No server needed; same SQLAlchemy models	In-memory mocks
DuckDB	1.1.3	Execution engine for all metric queries over Parquet	In-process vectorised columnar analytics; read-only views; interruptible for timeouts	Querying PostgreSQL directly, Spark, ClickHouse, SQLite
Polars	1.18.0	Synthetic data generation, Parquet writing, chunk conversion	Fast, memory-efficient DataFrames and Parquet I/O	Pandas only
Pandas	2.2.3	Excel reading (via openpyxl)	Best Excel support	Polars + fastexcel
PyArrow	18.1.0	Streaming Parquet writer for database imports	Chunked writes without loading a table into memory	fastparquet
Celery	5.4.0	Background ingestion jobs	Mature task queue; separate worker scales independently	RQ, Dramatiq, FastAPI BackgroundTasks
Redis	7 (Compose); 3.0.504 verified locally	Celery broker/result store	Simple, fast, standard broker	RabbitMQ

Technology	Version	Purpose in this project	Why chosen	Alternatives considered
OpenAI / Gemini (REST via httpx)	httpx 0.28.1	Optional AI for query plans, metric generation and semantic conflict suggestions	Provider chosen by environment variable; key only on the server; JSON-mode outputs	Local open-source LLMs, a single provider
PyJWT + Argon2 (argon2-cffi)	2.10.1 / 23.1.0	httpOnly JWT access and rotating refresh tokens; password hashing	Argon2 is the current recommended password hash; httpOnly cookies keep tokens away from JavaScript	bcrypt, server sessions, localStorage tokens
cryptography (Fernet)	44.0.0	Encrypts stored database-connection passwords	Authenticated symmetric encryption, simple key handling	Storing secrets in plain text, external vault
openpyxl + ReportLab	3.1.5 / 4.2.5	Excel and PDF report generation	Pure-Python, no external binaries	WeasyPrint, pandas.to_excel only
Docker and Docker Compose	Compose file v3 syntax	Reproducible multi-service stack (postgres, redis, backend, worker, frontend) with persistent volumes	One-command start; mirrors production topology	Manual installs, Kubernetes
Nginx	1.27 (container)	Serves the SPA, reverse-proxies `/api`, keeps SSE unbuffered; TLS terminator on a VPS	Standard, tiny footprint	Caddy, Traefik
Pytest (+ pytest-cov)	8.3.4	Back-end unit, API, security and integration tests; coverage	Fixtures, parametrisation, TestClient	unittest
Vitest + Testing Library	2.1.8 / 16.1.0	Front-end component and library tests	Vite-native, fast; tests user-visible behaviour	Jest
Playwright	1.49.1	End-to-end browser tests and documentation screenshots	Reliable auto-waiting, traces, multiple themes and viewports	Cypress, Selenium
GitHub Actions	n/a	CI: pytest on SQLite and PostgreSQL, Vitest, build, Playwright, Docker build	Native to GitHub; service containers for PostgreSQL	GitLab CI, Jenkins
Mermaid	10.9.3 (docs tooling)	Diagrams in the reports, rendered to PNG/SVG	Diagrams-as-code that live next to the text	draw.io, Graphviz

14 Key design decisions and trade-offs

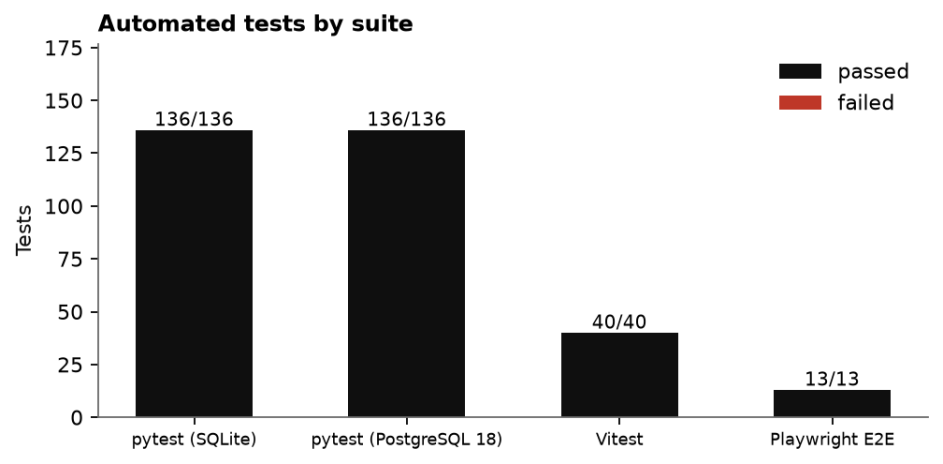
Decision	Alternatives considered	Why this choice	Trade-off accepted
Restricted formula grammar compiled to parameterised SQL	Accept SQL with a deny-list; let the LLM write SQL	Injection is impossible by construction; types and columns can be validated before execution; the same compiler serves the Lab, TARA, dashboards and the generator	Less expressive than SQL; no joins in a formula
Immutable versions + one live definition enforced by a partial unique index	Mutable rows with an audit log; soft versioning in application code only	History cannot be rewritten even by a bug; dashboards remain stable while a new version is reviewed; the database itself prevents two live definitions	Slightly more complex edit semantics (working copy vs live version)
Deterministic conflict detection first, AI only for semantic suggestions	LLM-only comparison	Repeatable, explainable, measurable precision/recall; AI output is clearly labelled and optional	Semantic overlaps between differently named metrics need an AI key to be suggested
TARA never fabricates: plan, validate, execute, then narrate from rows	LLM answers directly from data in the prompt	A wrong plan fails validation or returns an honest "cannot answer"; numbers always come from governed queries	Unusual phrasing may be refused instead of guessed
AI never auto-publishes or auto-executes	One-click AI publish	Human accountability and an approval trail	One extra review step
DuckDB over Parquet instead of querying PostgreSQL directly	Push every query to the source database; Spark	Fast columnar aggregation in-process; no load on source systems; simple deployment; read-only by construction	Data is a snapshot until re-ingested
Background ingestion (Celery/Redis; thread fallback)	Process uploads inside the request	Large files never block the web process or the browser; progress is visible	Extra infrastructure in production
Indian fiscal calendar in the planner and presets	Calendar year with a locale setting	"Last quarter" and "this year" have the meaning Indian finance teams expect; festival seasons give TARA realistic vocabulary	Region-specific logic that must be re-parameterised for other countries
SQLite for zero-setup development, PostgreSQL for production	PostgreSQL only	Contributors can run everything without a database server; the whole suite is also verified on PostgreSQL 18	Portable types only (string ids, JSON)
Frontend capability gates plus backend enforcement	Frontend-only checks	The backend is the source of truth; the frontend is for experience, not security	Capability names are duplicated by design (delivered via `/auth/me`)

15 Results

All figures in this section are measured by `scripts/benchmark.py`, the test runners and the in-product Evaluation module on Windows-10-10.0.26300-SP0 with Python 3.11.9 and 22 logical CPUs, run on 24-Oct-2026 18:11 IST. Where a number is an assumption (manual effort baselines) it is labelled so.

15.1 Test results

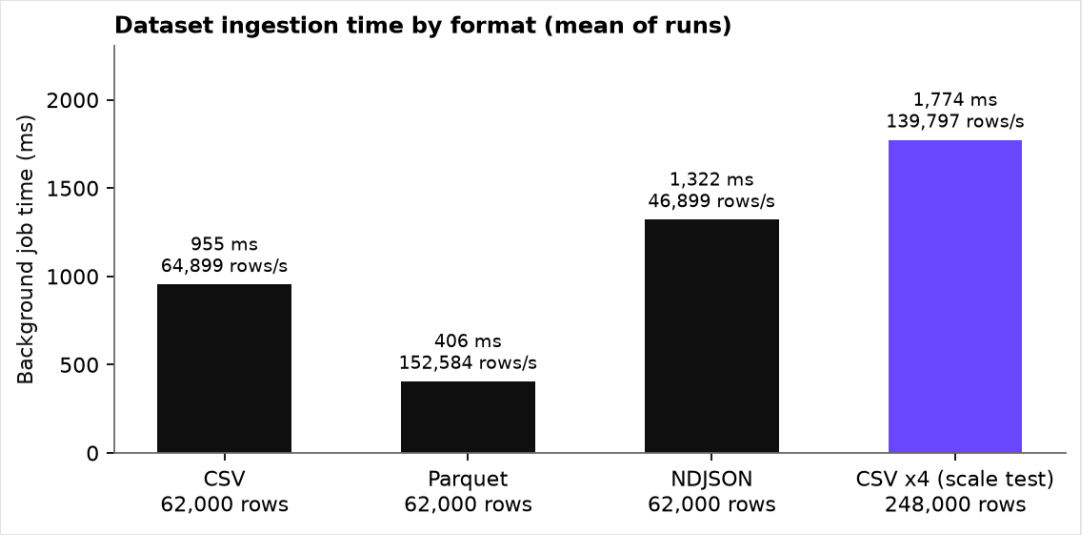
Suite	Tests	Passed	Failed	Duration
pytest, SQLite (API, services, security)	136	136	0	14.6 s
pytest, real PostgreSQL 18.2 (same suite, Alembic-built schema)	136	136	0	12.8 s
Vitest + Testing Library (components, lib)	40	40	0	1.4 s
Playwright E2E (Chromium)	13	13	0	48.4 s
Total	325	325	0	



Automated tests by suite

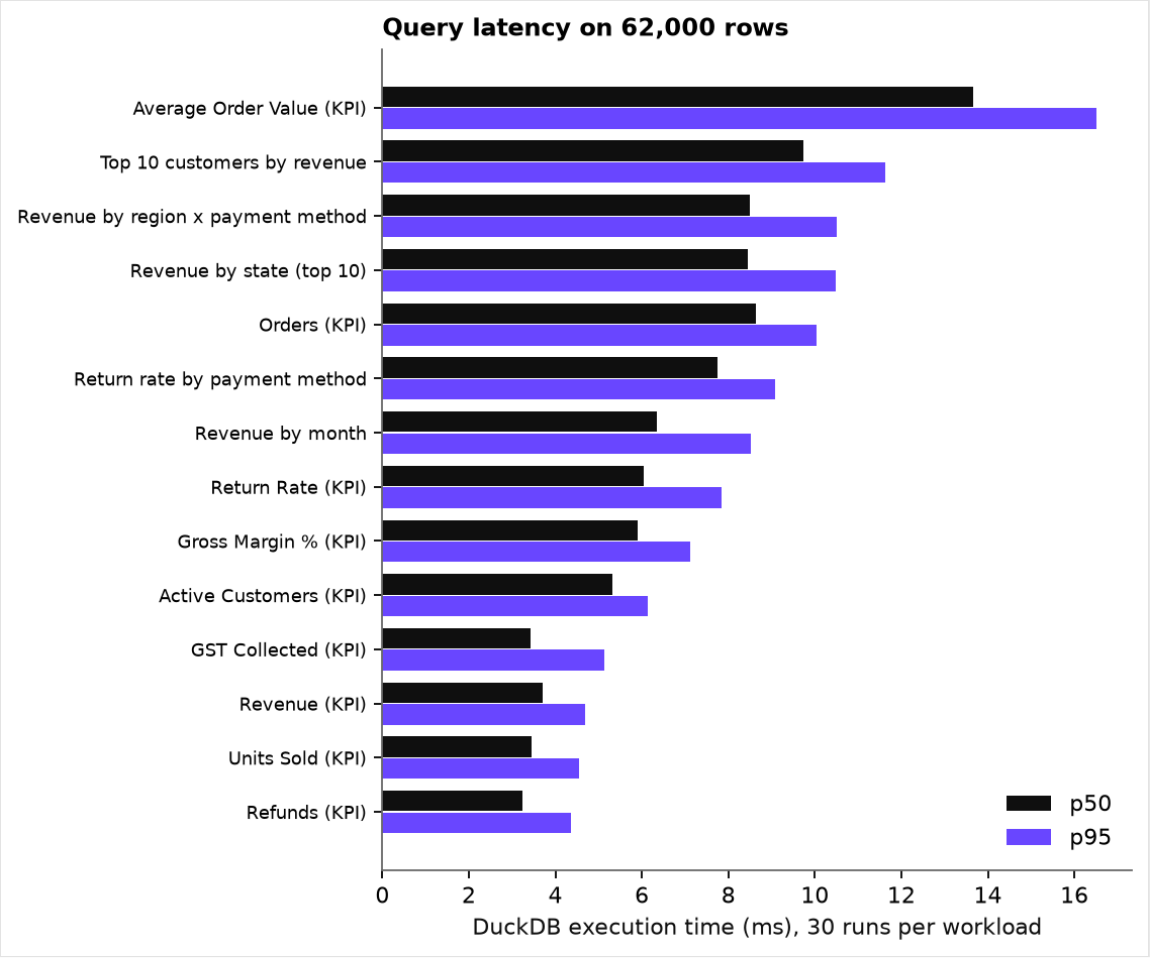
15.2 Ingestion and query performance

Format	Rows	File (MB)	Runs	Job time mean (ms)	Job p95 (ms)	Upload to ready (ms)	Rows / second
CSV	62,000	11.8	3	955	999	1,065	64,899
Parquet	62,000	1.7	3	406	420	489	152,584
NDJSON	62,000	31.2	3	1,322	1,376	1,504	46,899
CSV x4 (scale test)	248,000	47.6	1	1,774	n/a (1 run)	2,063	139,797



Dataset ingestion time by format

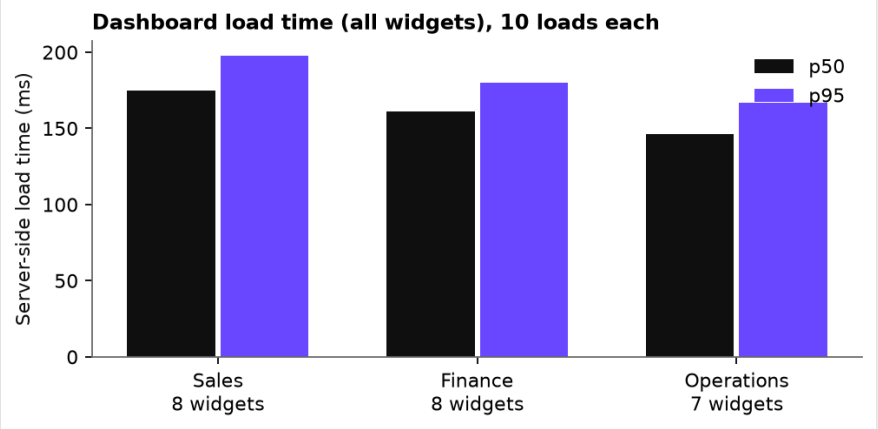
Workload (30 runs each)	Engine p50 (ms)	Engine p95 (ms)	HTTP round-trip p50 (ms)	HTTP p95 (ms)
Return Rate (KPI)	6.04	7.85	31.91	36.89
Active Customers (KPI)	5.33	6.14	25.59	35.6
Gross Margin % (KPI)	5.91	7.11	23.67	27.48
GST Collected (KPI)	3.42	5.13	21.78	26.45
Refunds (KPI)	3.23	4.37	24.17	32.13
Average Order Value (KPI)	13.67	16.51	41.28	50.94
Units Sold (KPI)	3.44	4.55	24.93	33.11
Orders (KPI)	8.64	10.03	29.45	32.08
Revenue (KPI)	3.71	4.68	24.28	28.27
Revenue by month	6.35	8.51	27.32	35.87
Revenue by state (top 10)	8.44	10.49	32.1	38.19
Revenue by region x payment method	8.5	10.5	30.33	33.91
Top 10 customers by revenue	9.73	11.62	30.27	33.49
Return rate by payment method	7.75	9.09	27.98	30.39
All 14 workloads (420 samples)	**6.37**	**12.77**	**27.85**	**40.17**



Query latency (DuckDB execution)

15.3 Dashboards and Ask TARA

Dashboard	Widgets	Server load p50 (ms)	p95 (ms)	HTTP p50 (ms)	Failed widgets (all loads)
Sales	8	174.95	197.51	186.23	0
Finance	8	161.05	179.76	170.62	0
Operations	7	146.1	166.81	156.05	0

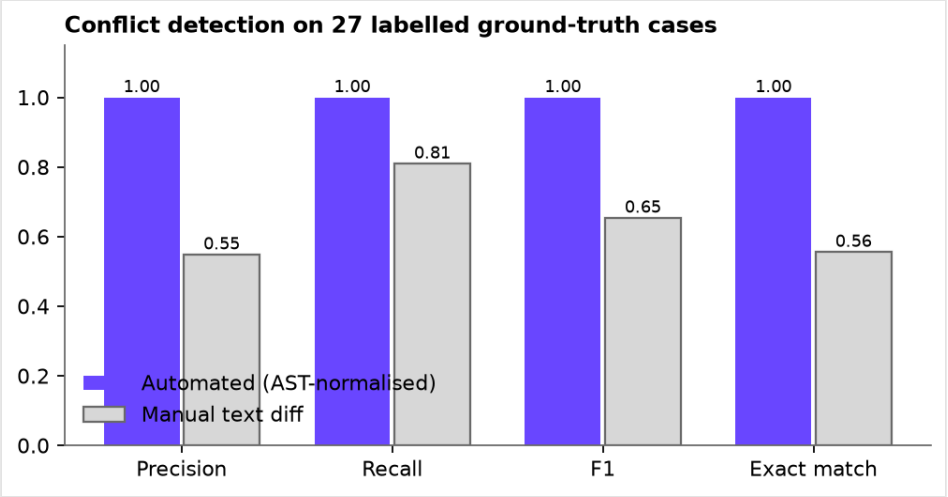


Dashboard load time

Question (10 runs each, offline planner)	Outcome	p50 (ms)	p95 (ms)
What was net revenue by month in the last quarter of this financial year?	answer	34.4	50.09
Which 5 states had the highest sales this year?	answer	37.36	40.91
Compare Diwali-season sales with the previous quarter	answer	50.01	53.84
What is the UPI vs COD split by region?	answer	38.28	48.05
Top 10 customers by revenue	answer	38.92	46.88
What is the weather in Paris?	unsupported	15.47	17.26
revenue by weather	unsupported	12.5	15.66

15.4 Quality measures from the Evaluation module

Measure	Result	Detail
Dataset processing (avg job time)	653 ms	16 successful jobs; 86,494 rows/s
Query latency (benchmark of every live metric)	avg 4.6 ms, p95 8.92 ms	9 metrics, full 62,000-row dataset
Validation pass rate	100%	23/23 metric definitions re-validated
Query success rate	100.0%	719/719 executions in the run history
Conflict detection precision / recall (automated)	1.00 / 1.00	27 labelled cases; TP 21, FP 0, FN 0
Conflict detection precision / recall (manual text diff baseline)	0.55 / 0.81	TP 17, FP 14, FN 4
AI suggestion acceptance rate (scripted offline session)	80%	8 accepted of 10 generations; 100% of outputs valid
Dashboard load time (avg of 5 stored dashboards)	167.2 ms	p95 230.2 ms; 5 dashboards
Metric creation: manual vs AI-assisted	360 s (assumed) vs 60.0 s	TARA generation measured at 0.0 s on average + 60 s assumed human review; estimated saving 83.3%
Conflict detection: manual vs automated	225 s (assumed) vs 0.48 ms	5 candidate pairs; manual review assumed at 45 s per pair



Conflict detection on labelled ground truth

The evaluation shows the intended behaviour of both detectors: the AST-normalised detector is correct on every labelled case, while the plain text comparison produces false positives on harmless variations (case, whitespace, operand order, a filter written in the formula versus the filter field) and therefore has lower precision.

15.5 Verified production topology

Docker was not available on the development machine, so the *components* of the production topology were verified directly instead:

- **PostgreSQL 18.2**: Alembic migrations `0001` and `0002` applied from an empty database; `alembic check` reports no drift; the full pytest suite (136 tests) passes against it; the partial unique index and the version-immutability trigger were confirmed with direct SQL.
- **Redis + Celery**: an ingestion job for 62,000 rows was queued by the API, executed by a separate Celery worker process on the `celery` runner in 748 ms and stored in PostgreSQL.
- **Connectors**: PostgreSQL 18.2 and MySQL 8.0.40 were each imported through the connection feature (30,000 rows per source); a write attempt through the import session is rejected by the database as read-only.

16 Limitations

- **Docker Compose was not executed** (Docker is not installed on the development machine). The Dockerfiles, Compose file and Nginx configuration are written to specification and are checked in CI, but the image builds were not run here.
- **No real OpenAI or Gemini call was made** (no API key). The client, structured-output validation, repair loop and fallbacks are covered by tests with a mocked provider; live behaviour and cost are unverified.
- The AI acceptance rate and the metric-creation time saving come from a *scripted* offline session and stated assumptions, not from a user study.
- All data is synthetic; accuracy of Indian calendar knowledge is limited to the festival windows encoded in `calendar_in.py`.
- The rate limiter is per process; a multi-instance deployment needs an edge limiter.
- E-mail delivery, single sign-on and multi-factor authentication are not implemented.

17 Conclusion

The project demonstrates that a governed semantic layer, a restricted and compiled formula language, deterministic conflict detection and an AI assistant that only drafts a query plan can be combined into a coherent, testable product. The Indian Revenue example is detected, explained, assigned, reviewed and resolved end to end, producing a new immutable version, and TARA then answers Indian-calendar questions (financial-year quarters, states, Diwali comparison, UPI versus COD) with numbers that match a direct query. The measured results show interactive performance on a 62,000-row dataset and complete automated verification across unit, API, component and browser levels.

18 Future scope

1. Run and tune TARA with live OpenAI/Gemini models; add evaluation against a labelled Indian question set and measure query-plan accuracy and refusal quality.
2. Joins and multi-dataset metrics through a governed entity model; semantic caching of frequent queries.
3. E-mail and Slack notifications; SSO (OIDC); multi-factor authentication; fine-grained dataset-level permissions.
4. Scheduled refresh and incremental ingestion; connectors for Snowflake, BigQuery and Google Sheets.
5. Regional-language TARA (Hindi, Telugu, Tamil) and voice input.
6. Anomaly detection and forecast metrics with explanation of variance by driver (price, volume, mix, GST).
7. Formal user study to quantify time saved and trust gained.

19 References

1. dbt Labs. *dbt Semantic Layer and MetricFlow documentation*. https://docs.getdbt.com/docs/build/about-metricflow
2. Cube Dev. *Cube semantic layer documentation*. https://cube.dev/docs
3. Google Cloud. *Looker: LookML reference*. https://cloud.google.com/looker/docs/lookml-terms-and-concepts
4. Raasveldt, M., and Mühleisen, H. (2019). DuckDB: an Embeddable Analytical Database. *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, pp. 1981-1984.
5. Yu, T., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *Proceedings of EMNLP 2018*.
6. Li, J., et al. (2023). Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD). *NeurIPS 2023 Datasets and Benchmarks Track*.
7. Gao, D., et al. (2024). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment*, 17(5).
8. Ji, Z., et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12).
9. OWASP Foundation. *SQL Injection Prevention Cheat Sheet* and *OWASP Top 10 (2021)*. https://owasp.org
10. Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. (1996). Role-Based Access Control Models. *IEEE Computer*, 29(2), 38-47.
11. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
12. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
13. Central Board of Indirect Taxes and Customs (CBIC), Government of India. *Goods and Services Tax: rates and notifications*. https://cbic-gst.gov.in
14. Government of India. *The Companies Act, 2013*, Section 2(41) (definition of "financial year").
15. FastAPI, SQLAlchemy 2.0, Alembic, Celery, Playwright and Vitest project documentation (versions in the technology table).